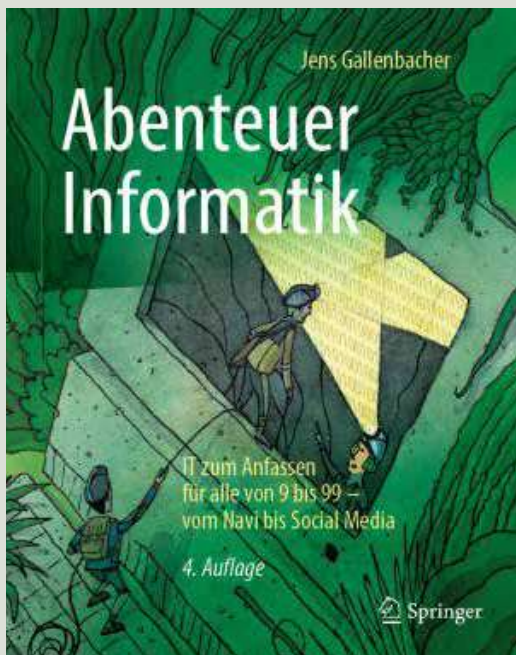


Abenteuer Informatik

Auszug der Kapitel

1, 2, 3, 8, 9, 11, 12, 14, 15



**IT zum Anfassen
für alle von 9 bis 99 –
vom Navi bis
Social Media**

Abenteuer Informatik

IT zum Anfassen für alle von 9 bis 99 –
vom Navi bis Social Media

4. Auflage

Auszug der Kapitel
1, 2, 3, 8, 9, 11, 12, 14, 15

Diese Kapitel werden unter der Lizenz
CC BY-NC-ND 4.0
veröffentlicht.

Sie dürfen daher dieses Dokument vervielfältigen und analog oder digital weiter verbreiten, aber nur unter bestimmten Bedingungen. Die wichtigsten sind:

Namensnennung - Sie müssen angemessene Urheber- und Rechteangaben machen und einen Link zu Lizenz beifügen.

Nicht kommerziell - Sie dürfen das Dokument nicht für kommerzielle Zwecke nutzen

Keine Bearbeitungen - Sie dürfen das Dokument nur in dieser unveränderten, vollständigen Form weitergeben und auch weder Ergänzungen noch Kürzungen vornehmen.

Die vollständigen Lizenzbedingungen folgen.

Durch die Ausübung der lizenzierten Rechte (wie unten definiert) erklären Sie sich rechtsverbindlich mit den Bedingungen dieser Creative Commons Namensnennung – Nicht kommerziell – Keine Bearbeitungen 4.0 International Public License ("Public License") einverstanden. Soweit die vorliegende Public License als Lizenzvertrag anzusehen ist, gewährt Ihnen der Lizenzgeber die in der Public License genannten lizenzierten Rechte im Gegenzug dafür, dass Sie die Lizenzbedingungen akzeptieren, und gewährt Ihnen die entsprechenden Rechte in Hinblick auf Vorteile, die der Lizenzgeber durch das Verfügbarmachen des lizenzierten Materials unter diesen Bedingungen hat.

Abschnitt 1 – Definitionen

a. Abgewandeltes Material bezeichnet Material, welches durch Urheberrechte oder ähnliche Rechte geschützt ist und vom lizenzierten Material abgeleitet ist oder darauf aufbaut und in welchem das lizenzierte Material übersetzt, verändert, umarrangiert, umgestaltet oder anderweitig modifiziert in einer Weise enthalten ist, die aufgrund des Urheberrechts oder ähnlicher Rechte des Lizenzgebers eine Zustimmung erfordert. Im Sinne der vorliegenden Public License entsteht immer abgewandeltes Material, wenn das lizenzierte Material ein Musikwerk, eine Darbietung oder eine Tonaufnahme ist und zur Vertonung von Bewegtbildern verwendet wird.

b. Urheberrecht und ähnliche Rechte bezeichnet das Urheberrecht und/oder ähnliche, dem Urheberrecht eng verwandte Rechte, einschließlich insbesondere des Rechts des ausübenden Künstlers, des Rechts zur Sendung, zur Tonaufnahme und des Sui-generis-Datenbankrechts, unabhängig davon, wie diese Rechte genannt oder kategorisiert werden. Im Sinne der vorliegenden Public License werden die in Abschnitt 2(b) (1)-(2) aufgeführten Rechte nicht als Urheberrecht und ähnliche Rechte angesehen.

c. Wirksame technische Schutzmaßnahmen bezeichnet solche Maßnahmen, die gemäß gesetzlichen Regelungen auf der Basis des Artikels 11 des WIPO Copyright Treaty vom 20. Dezember 1996 und/oder ähnlicher internationaler Vereinbarungen ohne entsprechende Erlaubnis nicht umgangen werden dürfen.

d. Ausnahmen und Beschränkungen bezeichnet Fair Use, Fair Dealing und/oder jegliche andere Ausnahme oder Beschränkung des Urheberrechts oder ähnlicher Rechte, die auf Ihre Nutzung des lizenzierten Materials Anwendung findet.

e. Lizenziertes Material bezeichnet das Werk der Literatur oder Kunst, die Datenbank oder das sonstige Material, welches der Lizenzgeber unter die vorliegende Public License gestellt hat.

f. Lizenzierte Rechte bezeichnet die Ihnen unter den Bedingungen der vorliegenden Public License gewährten Rechte, welche auf solche Urheberrechte und ähnlichen Rechte beschränkt sind, die Ihre Nutzung des lizenzierten Materials betreffen und die der Lizenzgeber zu lizenzieren berechtigt ist.

g. Lizenzgeber bezeichnet die natürliche(n) oder juristische(n) Person(en), die unter der vorliegenden Public License Rechte gewährt (oder gewähren).

h. Nicht kommerziell meint nicht vorrangig auf einen geschäftlichen Vorteil oder eine geldwerte Vergütung gerichtet. Der Austausch von lizenziertem Material gegen anderes unter Urheberrecht oder ähnlichen Rechten geschütztes Material durch digitales File-Sharing oder ähnliche Mittel ist nicht kommerziell im Sinne der vorliegenden Public License, sofern in Verbindung damit keine geldwerte Vergütung erfolgt.

i. Weitergabe meint, Material der Öffentlichkeit bereitzustellen durch beliebige Mittel oder Verfahren, die gemäß der lizenzierten Rechte Zustimmung erfordern, wie zum Beispiel Vervielfältigung, öffentliche Vorführung, öffentliche Darbietung, Vertrieb, Verbreitung, Wiedergabe oder Übernahme und öffentliche Zugänglichmachung bzw. Verfügbarmachung in solcher Weise, dass Mitglieder der Öffentlichkeit auf das Material von Orten und zu Zeiten ihrer Wahl zugreifen können.

j. Sui-generis Datenbankrechte bezeichnet Rechte, die keine Urheberrechte sind, sondern gegründet sind auf die Richtlinie 96/9/EG des Europäischen Parlaments und des Rates vom 11. März 1996 über den rechtlichen Schutz von Datenbanken in der jeweils gültigen Fassung bzw. deren Nachfolgeregelungen, sowie andere im Wesentlichen funktionsgleiche Rechte anderswo auf der Welt.

k. Sie bezeichnet die natürliche oder juristische Person, die von lizenzierten Rechten unter der vorliegenden Public License Gebrauch macht. Ihr bzw. Ihre hat die entsprechende Bedeutung.

Abschnitt 2 – Umfang

a. Lizenzgewährung

1. Unter den Bedingungen der vorliegenden Public License gewährt der Lizenzgeber Ihnen eine weltweite, vergütungsfreie, nicht unterlizenzierbare, nicht-ausschließliche, unwiderrufliche Lizenz zur Ausübung der lizenzierten Rechte am lizenzierten Material, um:

A. das lizenzierte Material ganz oder in Teilen zu vervielfältigen und weiterzugeben, jedoch nur für nicht kommerzielle Zwecke; und

B. nur für nicht kommerzielle Zwecke, abgewandeltes Material zu erstellen und zu vervielfältigen, es aber nicht weiterzugeben.

2. Ausnahmen und Beschränkungen. Es sei klargestellt, dass, wo immer gesetzliche Ausnahmen und Beschränkungen auf Ihre Nutzung Anwendung finden, die vorliegende Public License nicht anwendbar ist und Sie insoweit ihre Bedingungen nicht einhalten müssen.

3. Laufzeit. Die Laufzeit der vorliegenden Public License wird in Abschnitt 6(a) geregelt.

4. Medien und Formate; Gestattung technischer Modifikationen. Der Lizenzgeber erlaubt Ihnen, die lizenzierten Rechte in allen bekannten und zukünftig entstehenden Medien und Formaten auszuüben und die dafür notwendigen technischen Modifikationen vorzunehmen. Der Lizenzgeber verzichtet auf jegliche und/oder versichert die Nichtausübung jeglicher Rechte und Befugnisse, Ihnen zu verbieten, technische Modifikationen vorzunehmen, die notwendig sind, um die lizenzierten Rechte ausüben zu können, einschließlich solcher, die zur Umgehung wirksamer technischer Schutzmaßnahmen erforderlich sind. Im Sinne der vorliegenden Public License entsteht kein abgewandeltes Material, soweit lediglich Modifikationen vorgenommen werden, die nach diesem Abschnitt 2(a)(4) zulässig sind.

5. Nachfolgende Empfänger

A. Angebot des Lizenzgebers – Lizenziertes Material. Jeder Empfänger des lizenzierten Materials erhält automatisch ein Angebot des Lizenzgebers, die lizenzierten Rechte unter den Bedingungen der vorliegenden Public License auszuüben.

B. Keine Beschränkungen für nachfolgende Empfänger. Sie dürfen keine zusätzlichen oder abweichenden Bedingungen fordern oder das lizenzierte Material mit solchen belegen oder darauf wirksame technische Maßnahmen anwenden, sofern dadurch die Ausübung der lizenzierten Rechte durch Empfänger des lizenzierten Materials eingeschränkt wird.

6. Inhaltliche Indifferenz. Die vorliegende Public License begründet nicht die Erlaubnis, zu behaupten oder den Eindruck zu erwecken, dass Sie oder Ihre Nutzung des lizenzierten Materials mit dem Lizenzgeber oder den Zuschreibungsempfängern gemäß Abschnitt 3(a)(1)(A)(i) in Verbindung stehen oder durch ihn gefördert, gutgeheißen oder offiziell anerkannt werden.

b. Sonstige Rechte

1. Urheberpersönlichkeitsrechte, wie etwa zum Schutz vor Werkentstellungen, werden durch die vorliegende Public License ebenso wenig mitlizenziert wie das Recht auf Privatheit, auf Datenschutz und/oder ähnliche Persönlichkeitsrechte; gleichwohl verzichtet der Lizenzgeber auf derlei Rechte bzw. ihre Durchsetzung, soweit dies für Ihre Ausübung der lizenzierten Rechte erforderlich und möglich ist, jedoch nicht darüber hinaus.

2. Patent- und Kennzeichenrechte werden durch die vorliegende Public License nicht lizenziert.

3. Soweit wie möglich verzichtet der Lizenzgeber auf Vergütung durch Sie für die Ausübung der lizenzierten Rechte, sowohl direkt als auch durch eine Verwertungsgesellschaft unter welchem freiwilligen oder abdingbaren gesetzlichen oder Pflichtlizenzmechanismus auch immer eingezoogen. In allen übrigen Fällen behält sich der Lizenzgeber ausdrücklich jedes Recht vor, Vergütungen zu fordern, einschließlich für Nutzungen des lizenzierten Materials für andere als nicht kommerzielle Zwecke.

Abschnitt 3 – Lizenzbedingungen

Ihre Ausübung der lizenzierten Rechte unterliegt ausdrücklich folgenden Bedingungen

a. Namensnennung

1. Wenn Sie das lizenzierte Material weitergeben, müssen Sie:

A. die folgenden Angaben beibehalten, soweit sie vom Lizenzgeber dem lizenzierten Material beigefügt wurden:

i. die Bezeichnung der/des Ersteller(s) des lizenzierten Materials und anderer, die für eine Namensnennung vorgesehen sind (auch durch Pseudonym, falls angegeben), in jeder durch den Lizenzgeber verlangten Form, die angemessen ist;

ii. einen Copyright-Vermerk;

iii. einen Hinweis auf die vorliegende Public License;

iv. einen Hinweis auf den Haftungsausschluss;

v. soweit vernünftigerweise praktikabel einen URI oder Hyperlink zum lizenzierten Material;

B. angeben, falls Sie das lizenzierte Material verändert haben, und alle vorherigen Änderungsangaben beibehalten; und

C. angeben, dass das lizenzierte Material unter der vorliegenden Public License steht, und deren Text oder URI oder einen Hyperlink darauf beifügen.

Es sei klargestellt, dass Sie gemäß der vorliegenden Public License keine Erlaubnis haben, abgewandeltes Material weiterzugeben.

2. Sie dürfen die Bedingungen des Abschnitts 3(a)(1) in jeder angemessenen Form erfüllen, je nach Medium, Mittel und Kontext in bzw. mit dem Sie das lizenzierte Material weitergeben. Es kann zum Beispiel angemessen sein, die Bedingungen durch Angabe eines URI oder Hyperlinks auf eine Quelle zu erfüllen, die die erforderlichen Informationen enthält.

3. Falls der Lizenzgeber es verlangt, müssen Sie die gemäß Abschnitt 3(a)(1)(A) erforderlichen Informationen entfernen, soweit dies vernünftigerweise praktikabel ist.

Abschnitt 4 – Sui-generis-Datenbankrechte

Soweit die lizenzierten Rechte Sui-generis-Datenbankrechte beinhalten, die auf Ihre Nutzung des lizenzierten Materials Anwendung finden, gilt:

a. es sei klargestellt, dass Abschnitt 2(a)(1) Ihnen lediglich zu nicht kommerziellen Zwecken das Recht gewährt, die gesamten Inhalte der Datenbank oder wesentliche Teile davon zu entnehmen, weiterzuverwenden, zu vervielfältigen und weiterzugeben, und dies unter der Bedingung, dass Sie abgewandeltes Material nicht weitergeben;

b. sofern Sie alle Inhalte der Datenbank oder wesentliche Teile davon in eine Datenbank aufnehmen, an der Sie Sui-generis-Datenbankrechte haben, dann gilt die Datenbank, an der Sie Sui-generis-Datenbankrechte haben (aber nicht ihre einzelnen Inhalte) als abgewandeltes Material; und

c. Sie müssen die Bedingungen des Abschnitts 3(a) einhalten, wenn sie alle Datenbankinhalte oder wesentliche Teile davon weitergeben.

Es sei ferner klargestellt, dass dieser Abschnitt 4 Ihre Verpflichtungen aus der vorliegenden Public License nur ergänzt und nicht ersetzt, soweit die lizenzierten Rechte andere Urheberrechte oder ähnliche Rechte enthalten.

Abschnitt 5 – Gewährleistungsausschluss und Haftungsbeschränkung

a. Sofern der Lizenzgeber nicht separat anderes erklärt und so weit wie möglich, bietet der Lizenzgeber das lizenzierte Material so wie es ist und verfügbar ist an und sagt in Bezug auf das lizenzierte Material keine bestimmten Eigenschaften zu, weder ausdrücklich noch konkludent oder anderweitig, und schließt jegliche Gewährleistung aus, einschließlich der gesetzlichen. Dies umfasst insbesondere das Freisein von Rechtsmängeln, Verkehrsfähigkeit, Eignung für einen bestimmten Zweck, Wahrung der Rechte Dritter, Freisein von (auch verdeckten) Sachmängeln, Richtigkeit und das Vorliegen oder Nichtvorliegen von Irrtümern, gleichviel ob sie bekannt, unbekannt oder erkennbar sind. Dort, wo Gewährleistungsausschlüsse ganz oder teilweise unzulässig sind, gilt der vorliegende Ausschluss möglicherweise für Sie nicht.

b. Soweit wie möglich, haftet der Lizenzgeber Ihnen gegenüber nach keinem rechtlichen Konstrukt (einschließlich insbesondere Fahrlässigkeit) oder anderweitig für irgendwelche direkten, speziellen, indirekten, zufälligen, Folge-, Straf- exemplarischen oder anderen Verluste, Kosten, Aufwendungen oder Schäden, die sich aus der vorliegenden Public License oder der Nutzung des lizenzierten Materials ergeben, selbst wenn der Lizenzgeber auf die Möglichkeit solcher Verluste, Kosten, Aufwendungen oder Schäden hingewiesen wurde. Dort, wo Haftungsbeschränkungen ganz oder teilweise unzulässig sind, gilt die vorliegende Beschränkung möglicherweise für Sie nicht.

c. Der Gewährleistungsausschluss und die Haftungsbeschränkung oben sollen so ausgelegt werden, dass sie soweit wie möglich einem absoluten Haftungs- und Gewährleistungsausschluss nahe kommen.

Abschnitt 6 – Laufzeit und Beendigung.

a. Die vorliegende Public License gilt bis zum Ablauf der Schutzfrist des Urheberrechts und der ähnlichen Rechte, die hiermit lizenziert werden. Gleichwohl erlöschen Ihre Rechte aus dieser Public License automatisch, wenn Sie die Bestimmungen dieser Public License nicht einhalten.

b. Soweit Ihr Recht, das lizenzierte Material zu nutzen, gemäß Abschnitt 6(a) erloschen ist, lebt es wieder auf:

1. automatisch zu dem Zeitpunkt, an welchem die Verletzung abgestellt ist, sofern dies innerhalb von 30 Tagen seit Ihrer Kenntnis der Verletzung geschieht; oder

2. durch ausdrückliche Wiedereinsetzung durch den Lizenzgeber.

Es sei klargestellt, dass dieser Abschnitt 6(b) die Rechte des Lizenzgebers, Ausgleich für Ihre Verletzung der vorliegenden Public License zu verlangen, nicht einschränkt.

c. Es sei klargestellt, dass der Lizenzgeber das lizenzierte Material auch unter anderen Bedingungen anbieten oder den Vertrieb des lizenzierten Materials jederzeit einstellen darf; gleichwohl erlischt dadurch die vorliegende Public License nicht.

d. Die Abschnitte 1, 5, 6, 7 und 8 gelten auch nach Erlöschen der vorliegenden Public License fort.

Abschnitt 7 – Sonstige Bedingungen

a. Der Lizenzgeber ist nicht an durch Sie gestellte zusätzliche oder abweichende Bedingungen gebunden, wenn diese nicht ausdrücklich vereinbart wurden.

b. Jedwede das lizenzierte Material betreffenden und hier nicht genannten Umstände, Annahmen oder Vereinbarungen sind getrennt und unabhängig von den Bedingungen der vorliegenden Public License.

Abschnitt 8 – Auslegung

a. Es sei klargestellt, dass die vorliegende Public License weder besagen noch dahingehend ausgelegt werden soll, dass sie solche Nutzungen des lizenzierten Materials verringert, begrenzt, einschränkt oder mit Bedingungen belegt, die ohne eine Erlaubnis aus dieser Public License zulässig sind.

b. Soweit wie möglich soll, falls eine Klausel der vorliegenden Public License als nicht durchsetzbar anzusehen ist, diese Klausel automatisch im geringst erforderlichen Maße angepasst werden, um sie durchsetzbar zu machen. Falls die Klausel nicht anpassbar ist, soll sie von der vorliegenden Public License abgeschieden werden, ohne dass die Durchsetzbarkeit der verbleibenden Bedingungen tangiert wird.

c. Auf keine Bedingung der vorliegenden Public License wird verzichtet und kein Verstoß dagegen soll als hingenommen gelten, außer der Lizenzgeber hat sich damit ausdrücklich einverstanden erklärt.

d. Nichts in der vorliegenden Public License soll zu einer Beschränkung oder Aufhebung von Privilegien und Immunitäten führen, die dem Lizenzgeber oder Ihnen insbesondere aufgrund rechtlicher Regelungen irgendeiner Rechtsordnung oder Rechtsposition zustehen, oder dahingehend interpretiert werden.

Prof. Dr.-Ing. Jens Gallenbacher
Didaktik der Informatik
abenteuer@gallenbacher.de

Ergänzendes Material zu diesem Buch finden Sie auf
www.abenteuer-informatik.de

Dieser Auszug ist entnommen aus dem Buch
Abenteuer Informatik, 4. Auflage, erschienen bei Springer-Nature
ISBN 978-3-662-53964-4
ISBN 978-3-662-53965-1 (eBook)
DOI 10.1007/978-3-662-53965-1

Inhalt

Einleitung

V

Kapitel 1 – Sag mir wohin ...

1

Lösen Sie das Problem des kürzesten Weges: Welchen Weg muss ich nehmen, um mit der kürzesten Strecke bzw. am schnellsten von Ort A nach Ort B zu kommen? Kleine Krabbeltiere werden Ihnen dabei behilflich sein, auch den Geheimnissen der Navi-Profis im Internet auf die Spur zu kommen.

Kapitel 2 – Ordnung muss sein!

49

Einstein und Freud bevorzugten einhellig das Chaos auf ihrem privaten Schreibtisch. Damit ein Computer aber für uns schnell Informationen auffindet, müssen diese sortiert werden. Lernen Sie, wie der Computer vom Menschen gelernt hat.

Kapitel 3 – Ich packe meinen Koffer und ...

85

Wenn ein Informatiker einen Rucksack füllt, dann sollte der zur Verfügung stehende Platz bestens ausgenutzt sein! Vollziehen Sie die Kunst des Packens und dabei auch noch die der dynamischen Programmierung nach.

Kapitel 8 – Erkennungsdienst

111

Spiel oder Ernst? Identifizieren Sie mit Spürsinn anhand einfacher Fragen Ihr Gegenüber. Können Sie sich besser tarnen, wenn Sie dann im Fokus der Enthüllungen stehen? Auf jeden Fall ermitteln Sie dabei ganz automatisch ein paar wesentliche Grundlagen der Informatik.

Kapitel 9 – Paketpost

129

Bücher, Schuhe oder auch einfach die Pizza für den kleinen Hunger zwischendurch – kaum eine Lieferung würde bei uns ankommen, wenn nicht vorher ganz andere Pakete sicher ihren Weg durch den Dschungel des Internets gefunden hätten. Erfahren Sie hier, wie das auch in chaotischen Verhältnissen immer wieder gelingt.

Kapitel 11 – Ordnung im Chaos

153

Hatte Einstein doch recht und Chaos ist besser als Ordnung? Informatiker nutzen seit Langem ein Verfahren, das Informationen scheinbar chaotisch anordnet, sie jedoch trotzdem auf Anhieb auffindbar macht. Vielleicht eine neue Möglichkeit, die Wohnung zu organisieren (Achtung: Ärger mit dem Partner ist vorprogrammiert ...)

Kapitel 12 – Mit Sicherheit

177

Safeknacker sind unmodern geworden. Ihre Kollegen des IT-Zeitalters bedienen sich nur noch eines Computers und des Internets. Nur wer die Prinzipien hinter modernen Sicherheitsmechanismen begreift, kann diese so einsetzen, dass die Diebe kaum noch eine Chance haben! Aber hinter was sind sie eigentlich her – in den sozialen Netzwerken hinterlassen wir doch kein Geld, nur Daten ... genau!

Kapitel 14 – InformaGik

223

Nicht alles ist fauler Zauber: Lernen Sie die Magie der Fehlerkorrekturmechanismen kennen, die zum Beispiel dafür sorgen, dass sich auch verstaubte und verkratzte DVDs noch einwandfrei abspielen lassen.

Kapitel 15 – Allmächtiger Computer!?

239

Computer ohne Grenzen? Wohl kaum! Auch für die schlauen Kisten gibt es weiße Flecken auf der Landkarte: Nicht alles ist von ihnen berechenbar. Erfahren Sie anhand spannender Puzzlespiele, welche Dinge niemals per Computer ermittelbar sein werden.

Einleitung

Unsere Welt ist gestaltet! Zunehmend dominieren nicht mehr die Kräfte der Natur das Bild des Alltags, sondern Elemente, die aus menschlicher Kreativität und menschlichem Erfindungsgeist rühren. Informationstechnik ist dabei unzweifelhaft der dominierende Faktor: Sie steckt in normalen Haushaltsgeräten so selbstverständlich wie in Kleidung und Spielen.

Digitale Lebenswelten sind emanzipiert von den Naturgesetzen und daher ihre Gestaltungsmöglichkeiten schier unbegrenzt. Sie stellen daher aber auch sowohl für ihre Nutzer als auch die Gesellschaft eine große Herausforderung dar: Demokratie bedeutet für alle, bei essentiellen Veränderungen und Weichenstellungen mitzuentcheiden. Das kann entweder aktiv ausgeübt werden, durch konkrete Gestaltung von Elementen oder aber passiv, durch informierte politische Beteiligung.

Beides erfordert Einsicht in die zugrundeliegenden Prinzipien der Gestaltung und die daraus resultierende Technologie der Informationstechnik. Informatik ist der Schlüssel zum Verständnis der Informationstechnik und damit der Schlüssel zum Verstehen unserer modernen Umwelt!

Die gute Nachricht: Informatik ist nicht nur cool, sondern auch eine Disziplin, die von Menschen nach menschlichen Prinzipien gestaltet ist! Überraschend viele Prinzipien sind Ihnen also schon bekannt. Schemata großer Datenbanksysteme beruhen auf den gleichen Vorgehensweisen, wie sie auch Grundschülerinnen und Grundschüler beim Sortieren von Spielkarten ganz implizit entwickeln, andere Ideen können wir uns aus der Natur abschauen. Für das Verständnis komplexer Zusammenhänge haben wir eigene Anschauungen entwickelt, wovon die Sprache der Mathematik nur eine ist.

Das Buch „Abenteuer Informatik“ hilft Ihnen dabei, diese Potentiale zu erkennen und die Informatikerin bzw. den Informatiker in sich zu finden. Dabei werden einerseits alltägliche Geräte wie das Navi im Auto beleuchtet, andererseits bewusst spielerische Herangehensweisen gewählt.

Ein Prinzip durchzieht dabei allerdings alle Kapitel: Das wörtliche „Begreifen“ steht nicht nur am Anfang, sondern ist das zentrale Element. Das geht am besten ohne Computer: mit Papier, Bleistift, einer Schere und den Vorlagen aus dem Buch.

Sie lesen hier einen Ausschnitt mit neun Kapiteln des Buches. Dank der TU Wien, die die Kapitel als Grundlage für Zugangstests für die Informatik-Studiengänge nutzt, konnten sie mit einer Open-Access-Lizenz versehen für alle kostenlos zugänglich gemacht werden.

Um Inkonsistenzen zu vermeiden wurden dabei die Original-Buchkapitel soweit es ging beibehalten. Das hat folgende Implikationen, die Sie beim Lesen beachten sollten:

Das Format des gedruckten Buches ist 193mm x 242mm. Dieses Format wurde in dieser PDF-Datei beibehalten. Wenn Sie diese drucken, werden Sie wahrscheinlich auf DIN-A4 drucken, was mit 210mm x 297mm deutlich größer ist. Ich empfehle, dabei keine Vergrößerungen und Verkleinerungen vorzunehmen, damit Sie mit den angebotenen Bastelbögen konsistent bleiben. Wenn Sie möchten, können Sie natürlich die Seiten oben bzw. unten beschneiden, um dem original-Buch möglichst nahe zu kommen.

Der Satz ist auf Doppelseiten ausgelegt, es gibt also „linke“ Seiten mit geraden Nummern und „rechte“ Seiten mit ungeraden Nummern. Durchgehend sind am Rand die Marginalienspalten mit zusätzlichen Informationen zu finden, aber auch im Inhalt wird manchmal davon ausgegangen, dass Sie linke und rechte Seite gleichzeitig sehen. Ich empfehle daher, beim Lesen in elektronischer Form die Einstellung „Zweiseitenansicht“ zu wählen. Wenn Sie das Dokument drucken, sollten Sie es doppelseitig drucken und binden, so dass sich die Seitennummern außen befinden. Damit das besser klappt, gibt es spezielle PDF-Vorlagen für A4- und A3-Druck.

Soweit die technischen Hinweise... Beim Lesen von „Abenteuer Informatik“ können Sie eine mehr passive oder mehr aktive Rolle wählen. Selbstverständlich ist es spannend und auch lehrreich, auf wichtige Erkenntnisse der Informatik selbst zu kommen – zum Beispiel nach einem eigenhändig durchgeführten Experiment.

Sehr oft finden Sie daher eine Frage, einen Denkanstoß oder einen Arbeitsauftrag in blauer Schrift. Direkt dahinter steht das „Denk-Köpfchen“.



Es soll Sie kurz daran hindern, hier schon weiterzulesen. Sie können sich nun entscheiden, ob Sie selbst versuchen, zu einer Lösung zu kommen oder diese einfach im nächsten Abschnitt lesen. Lassen Sie sich davon leiten, wie spannend das Thema für Sie ist.

Eine Quelle zusätzlicher Information ist die kleine linke bzw. rechte Spalte des Buches. Hier finden Sie kurze Lebensläufe wichtiger Persönlichkeiten, Anekdoten und anderes Wissenswerte im Zusammenhang mit dem Haupttext. Das Verständnis des Kapitels ist allerdings auch ohne diese Spalte gewährleistet. Sie können sich daher diesen Teil für ein zweites Durchlesen aufsparen oder ihn als willkommene Ablenkung ansehen.

Den Anfang neuer Kapitel finden Sie im Buch übrigens recht einfach, wenn Sie es von der Seite betrachten: Das große Bild auf der linken Seite ist auch hier zu erkennen und lässt sich auf diese Weise als eine Art Index nutzen.

Wichtige Begriffe

Im Text sind immer wieder grundlegende Begriffe und Methoden der Informatik kurz erklärt. Um sie leichter wiederzufinden, stehen sie in blauen Kästchen. Wenn Sie möchten, legen Sie sich doch ein eigenes Merkblatt mit den Inhalten dieser Kästchen an. Ganz hinten im Buch sind allerdings ein paar wichtige Schlagwörter in einem Glossar zusammengefasst auch übersichtlich nachzuschlagen.

Die Reihenfolge der Kapitel ist recht lose. Auch wenn Sie das Buch nicht von vorne nach hinten durchlesen, sondern sich von Ihrer „Abenteuerlust“ leiten lassen, kann man die meisten Texte verstehen! Manchmal ist allerdings ein Hinweis auf eines der vorhergehenden Kapitel gegeben, das dann eine wichtige Grundlage beinhaltet. Mein Tipp ist, auf jeden Fall das erste Kapitel, vielleicht auch noch das zweite Kapitel zu Beginn zu lesen. Danach können Sie unbeschwert stöbern.

Kommen wir zu einem sehr wichtigen Teil, der dieses Buch auch von den allermeisten anderen Informatik-Büchern unterscheidet: der Bastelbogen! Er ermöglicht das

wörtliche „Begreifen“ ohne zu viel Aufwand in der Herstellung eigener Materialien. Es genügt meistens, diese auszuschneiden.

Jedes Kapitel enthält daher am Ende entsprechende Kopiervorlagen. Manche Spielmaterialien – wie die Karten der binären Magie – brauchen zwingend Vorder- und Rückseite. Da selbst Kopierer und Drucker mit Duplex-Einheit oft nicht schaffen, die beiden Seiten wirklich deckungsgleich zu Papier zu bringen, habe ich diese Vorlagen so angelegt, dass man sie nach dem Ausschneiden an der durchgezogenen Linie noch an der gestrichelten Linie faltet und entweder zusammenklebt oder gefaltet laminiert.



Abbildung
Gedruckte Bastelbögen

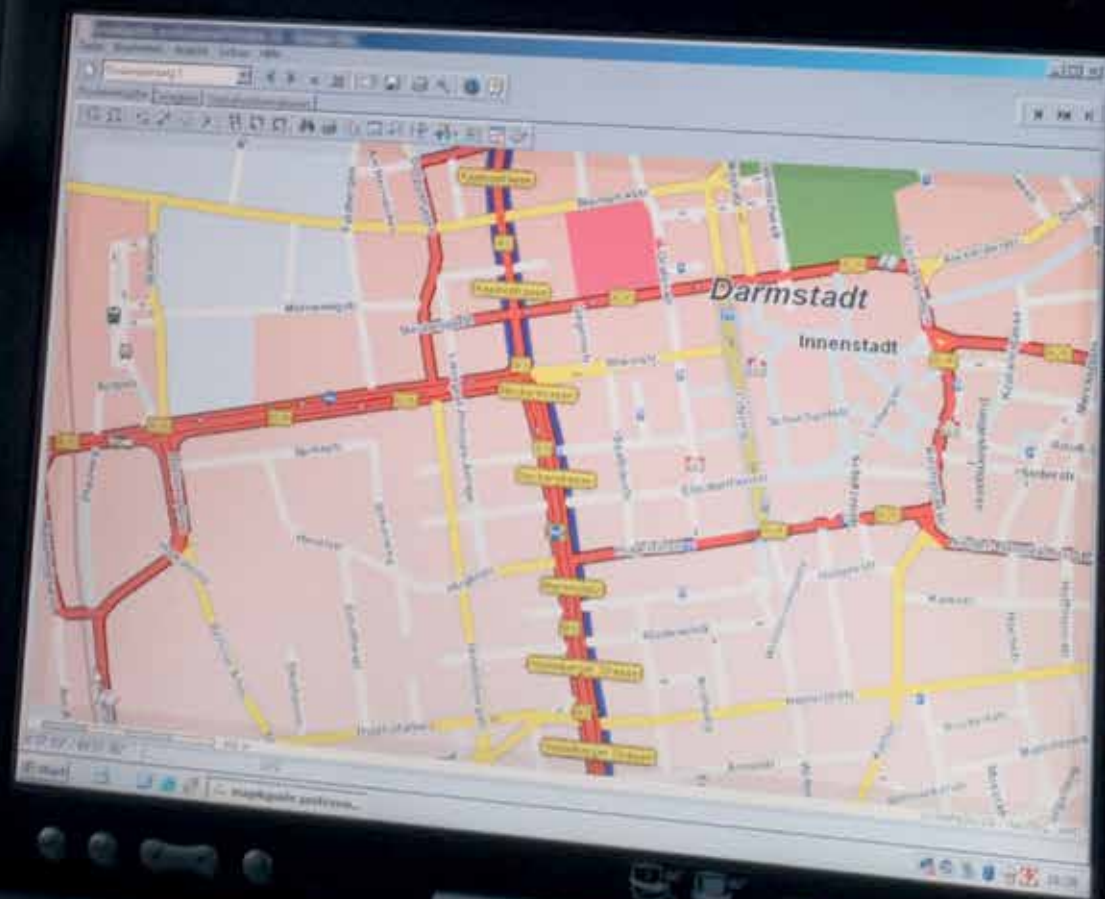
Noch einfacher ist selbstverständlich, die fertigen Bastelbögen zu benutzen. Um noch mehr Inhalt in das Buch zu bringen und die elektronische Ausgabe zu ermöglichen, sind diese nicht mehr integraler Bestandteil, sondern gesondert zu beziehen. Das erleichtert auch Lehrerinnen und Lehrern, gleich einen ganzen Klassensatz zu bekommen. Leserinnen und Lesern, die dieses Buch in einer Bücherei ausgeliehen haben, können so ebenfalls unbeschwert schneiden und kleben. Die Bedingungen für den Bezug sind zum Zeitpunkt des Drucks noch nicht ganz geklärt, bitte informieren Sie sich über die Webseite

www.abenteuer-informatik.de

Diese ist auch die zentrale Seite für Korrekturen und zusätzliche Informationen zum Buch sowie der gleichnamigen Ausstellung, die inzwischen in Universitäten, Schulen und Science-Centern zu finden ist.

Damit genug der Nutzungshinweise – ich wünsche Ihnen viel Spaß bei Ihrem ganz persönlichen Abenteuer Informatik!


Jens Gallenbacher



1. Sag mir wohin ...

Einführung

Routenplaner gehören heute schon fast zum Alltag: Viele Autos haben sie bereits eingebaut und auch wer keinen im Fahrzeug hat, lässt sich den günstigsten Weg zu seinem Ziel oft auf dem heimischen PC ermitteln und druckt ihn aus.

Versuchen wir doch gleich einmal das nachzuvollziehen: Nehmen Sie einen großen Straßenatlas und ermitteln Sie die günstigste Strecke von Stockheim nach Weilheim!

Zu viel Arbeit? Kein Atlas? Also gut – ich hatte in der Einleitung ja versprochen, dass alle notwendigen Materialien hier im Buch zu finden sind. Daher arbeiten wir erst einmal mit der folgenden kleinen Welt nach Abbildung 1.1.

Die Karte zeigt Orte, zwei Autobahnen, größere und kleinere Landstraßen. Die roten und blauen Zahlen geben dabei immer die Länge der Straße an, wenn man sie mit dem Auto fährt. Man kann sehen, dass kleine Straßen meistens viel länger sind, als sie scheinen, weil die zahlreichen Kurven und Berge nicht eingezeichnet sind.

Welches ist denn nun der günstigste Weg von Imstadt nach Oppenheim?

Erster Ansatz wäre, zur nächsten Autobahn zu fahren. Aber geht das am besten über Pappstadt oder die Auffahrt Budingen? Außerdem führt ja von Pappstadt aus auch eine direkte Landstraße zum Ziel. Die ist aber wohl gewunden und ziemlich lang. Oder doch lieber die gelbe Straße zum Flughafen und von dort die Autobahn nach Oppenheim nehmen?

Versuchen Sie, die Lösung selbst herauszufinden. Hinweis: Sie müssen 24,6 km zurücklegen.

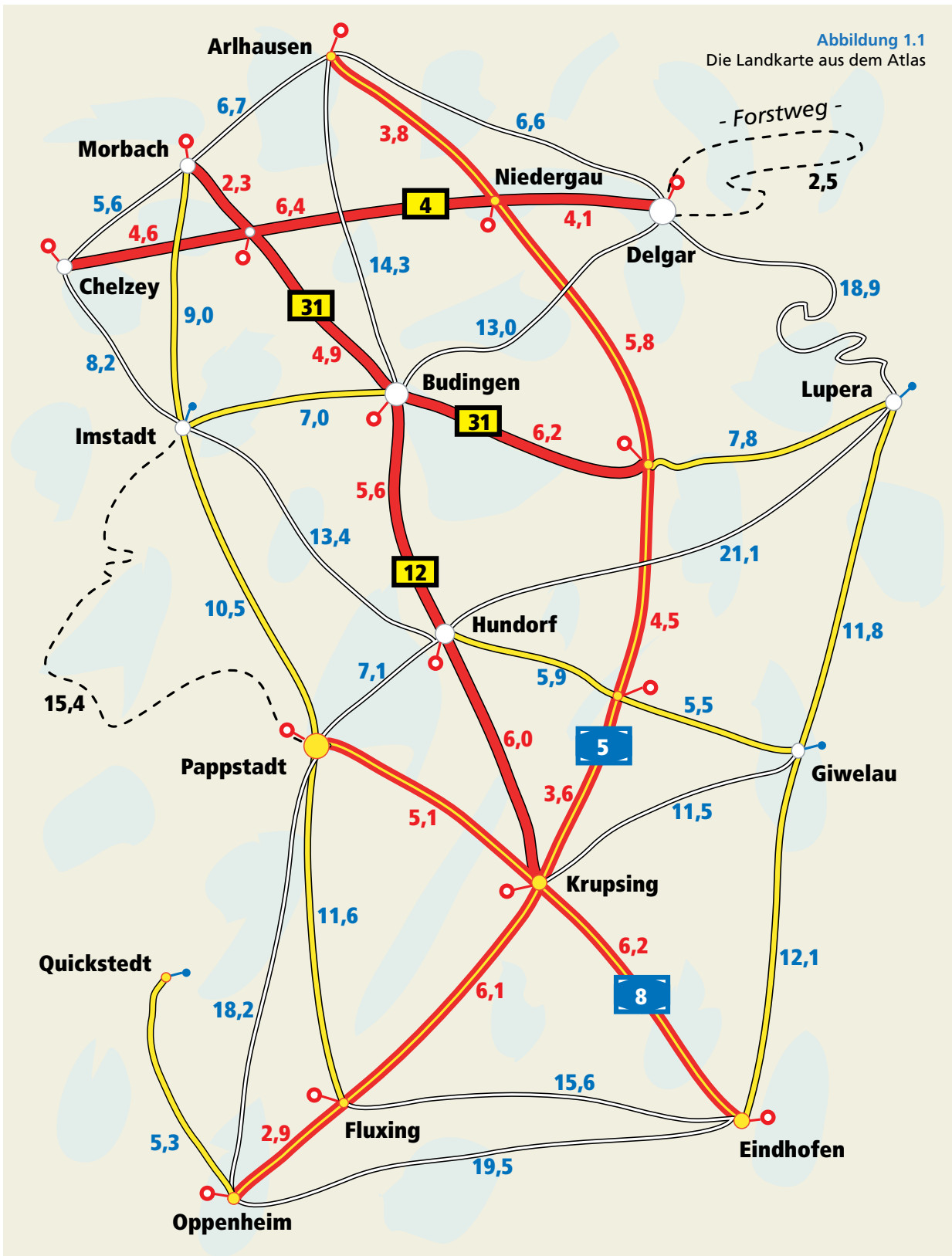
Geschafft? Gut! Dann lehnen Sie sich zurück und genießen Sie den Erfolg.

Wie sind Sie vorgegangen? Sie haben wahrscheinlich alle möglichen Wege durchprobiert und die Entfernung zum Ziel ermittelt. Dann haben Sie sich für den günstigsten Weg entschieden.

Dieses Verfahren gibt es auch bei Computern – es hat sogar einen Namen: die Brute-Force-Methode, also etwa „Brutale Macht“. Warum? Weil auf diese Weise etwas umfangreichere Aufgaben nur mit extrem großer Rechenkraft gelöst werden können. Überlegen Sie einmal, wie viele verschiedene Wege Sie schon bei der gegebenen, sehr übersichtlichen Karte durchspielen mussten. Stellen Sie sich nun vor, wie das mit 1000 und mehr Städten wäre – hier wären normale Rechner gar nicht mehr zur Lösung fähig.

Außerdem besitzt ein Rechner keine Intelligenz: Während Sie beim Durchprobieren unbewusst alle absurden und unwahrscheinlichen Möglichkeiten verwerfen, muss er diese durchrechnen.

Abbildung 1.1
Die Landkarte aus dem Atlas



Vorüberlegungen

Wie kommt ein Informatiker nun zu einer besseren Lösung?

Zunächst sollte man mit der Methode der Abstraktion arbeiten!

Methode der Abstraktion

In zur Verfügung stehender Information stecken sowohl relevante als auch unwesentliche Anteile. Durch Abstraktion reduzieren Sie die Information auf das für die aktuelle Problemlösung Wesentliche; Dadurch können Sie sich besser auf Ihre Aufgabe konzentrieren.



Man könnte auch sagen: Werfen Sie alles Überflüssige über Bord und konzentrieren Sie sich auf das Wesentliche. Was ist aber bei der gestellten Aufgabe wesentlich?

Alle gegebenen Informationen stecken in der Karte. Welche Typen von Informationen kann man erkennen? Erstellen Sie eine Liste, bevor Sie weiterlesen.



Auch **abstrakte Malerei** ist die Konzentration auf das Wesentliche. Der Künstler stellt die Aspekte in den Mittelpunkt, die ihn bewegen, zum Beispiel ein Gefühl oder ein Ereignis. Details wie die realistische Darstellung treten dadurch in den Hintergrund. Hier abgebildet ist das „Blaue Pferd“ von Franz Marc (1911).

Die Informationen der Karte sind in folgender Tabelle zusammengefasst.

	wichtig?
Namen der Städte	☐
Position der Städte	☐
Größe der Städte	☐
Verlauf der Straßen	☐
Länge der Straßen	☐
Namen und Nummern der Straßen	☐
Straßentyp	☐
Straße führt von ... nach ...	☐
Landschaftliche Information	☐

An dieser Stelle fällt Ihnen eventuell auch bereits auf, dass unsere bisherige Formulierung der Aufgabe, den „günstigsten“ Weg zu finden, nicht präzise genug ist. Genau genommen suchen wir den (Strecken-)kürzesten Weg!

Überlegen Sie weiter, welche dieser Informationen wir benötigen, um den kürzesten Weg zwischen zwei Städten zu suchen. Markieren Sie für jede Information, ob diese Ihrer Meinung nach für die Aufgabenstellung wichtig oder nicht wichtig ist. Der kürzeste Weg soll sich hierbei auf die zu fahrende Strecke beziehen, nicht auf die gefahrene Zeit.



Linie 1



Abstraktion begegnet uns ständig! Wegweiser, Straßenschilder, Fahrpläne, Infobroschüren und viele andere Dinge des Alltags sind so aufbereitet, dass die nötigen Informationen möglichst offen und gut erkennbar dargestellt sind.

Mein Ergebnis ist folgendes:

	wichtig?
Namen der Städte	☒ Wenn man nicht weiß, welche Stadt wie heißt, kann auch nicht der kürzeste Weg zwischen Imstadt und Oppenheim bestimmt werden.
Position der Städte	☐ Es ist uns egal, wo sich die Städte genau befinden. Relevant sind nur die Straßen zwischen den Städten.
Größe der Städte	☐ Kommt in unserer Aufgabenstellung nirgendwo vor.
Verlauf der Straßen	☐ Es kommt nur auf die Streckenlänge an, nicht auf den Verlauf.
Länge der Straßen	☒ Um die Reisedistanz zu summieren, benötigen wir die einzelnen Strecken zwischen den Orten.
Namen und Nummern der Straßen	☐ Zumindest zur Bestimmung der kürzesten Strecke irrelevant.
Straßentyp	☐ Da es nur auf die Entfernungen, nicht auf Zeit ankommt, ist egal, ob Autobahn oder Feldweg gefahren wird.
Straße führt von ... nach ...	☒ Wir benötigen die Information, von welcher Stadt zu welcher anderen eine Straße führt.
Landschaftliche Information	☐ Offensichtlich ...

Nun kann die Karte neu gezeichnet werden, und zwar so, dass möglichst alle irrelevanten und damit störenden Informationen fehlen. Versuchen Sie es einmal selbst, bevor Sie weiterlesen!

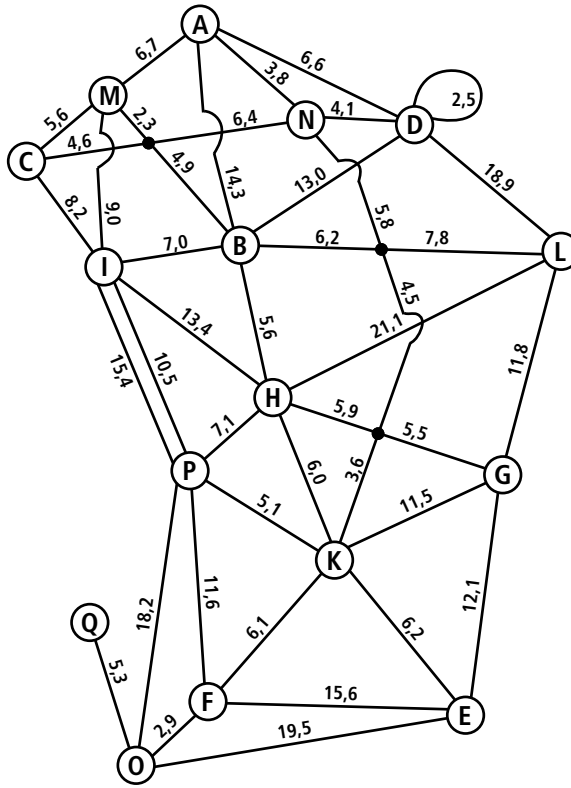


Aus Gründen der Übersichtlichkeit habe ich hier in Abbildung 1.2 die Städte auf ihren ersten Buchstaben reduziert. Das ist jedoch nicht unbedingt notwendig. Wenn Ihre Lösung die ausgeschriebenen Namen enthält, können Sie auch damit weiterarbeiten.

Man sieht, dass es noch ein paar Besonderheiten gibt: An vier Stellen kreuzen sich zwei Straßen, ohne dass es Auf- oder Abfahrten von einer zur anderen gäbe (z. B. eine Autobahnbrücke über einer Landstraße). Dies ist mit einem Bogen dargestellt. Darüber hinaus gibt es weitere drei Stellen, an denen sich zwei Straßen schneiden und es Auf- und Abfahrten gibt – gekennzeichnet mit einem Punkt. Außerdem ist unklar, welches nun die „Straße zwischen I und P“ ist, denn hierfür gibt es zwei Kandidaten.

Informatiker wollen sich jedoch prinzipiell nicht mit Sonderfällen beschäftigen und lieben es daher übersichtlich: Zu viele spezielle Fälle und Unterscheidungen machen das Denken schwierig. Wie bei Mathematikern, die einen Bruch erst einmal auf den gleichen Nenner bringen, wird hier auch versucht, ein Problem möglichst gleichförmig darzustellen.

Abbildung 1.2
Abstraktere Form der Landkarte



Methode der Gleichformung

Versuchen Sie, die verschiedenen Facetten eines Problems auf die gleichen Grundelemente zurückzuführen. Dadurch wird einerseits das Problem übersichtlicher und andererseits benötigt man weniger Lösungsansätze: Für gleichförmige Teilprobleme kann der gleiche Lösungsansatz verwendet werden.

Können Sie die Karte auf diese Weise noch einfacher gestalten?

Überlegen Sie, welches die Eigenschaften der „normalen“ Elemente sind und ob man die speziellen Elemente nicht auch als normale Elemente darstellen kann.



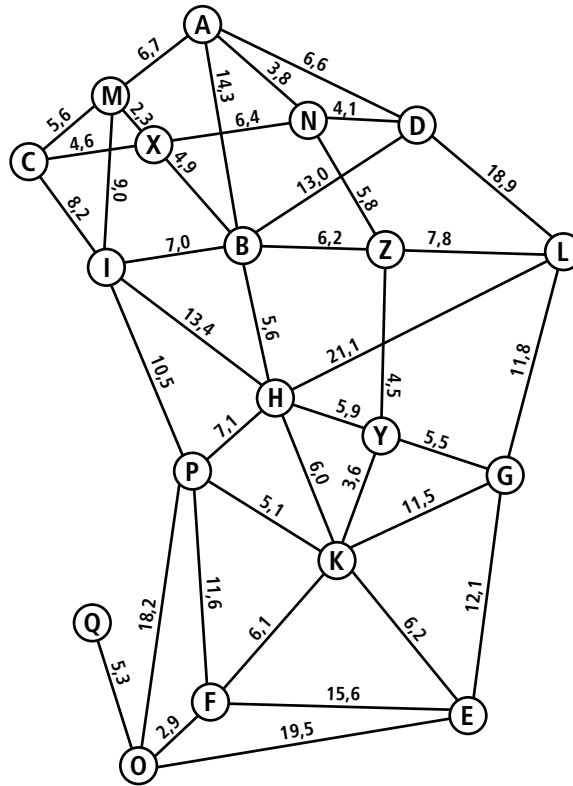
Auf der Karte sind Städte als Kreise eingezeichnet. Ohne dass dies speziell vermerkt ist, kann man offenbar bei Städten problemlos von einer Straße auf eine angrenzende Straße wechseln.

Genau das soll auch an den mit Punkt gekennzeichneten Stellen möglich sein. Also tun wir einfach so, als ob sich dort Städte befinden. Um sie nicht mit den anderen Städten zu verwechseln, nennen wir sie $\textcircled{X}$, $\textcircled{Y}$ und $\textcircled{Z}$.

An allen weiteren Kreuzungspunkten zwischen zwei Straßen ist jetzt kein Wechsel mehr möglich. Es ist daher auch keine Unterscheidung, also auch keine Kennzeichnung durch Bögen mehr notwendig.

Für die Bestimmung des kürzesten Weges kommt von zwei möglichen Strecken nur die kürzeste in Frage. Rundfahrten wie von Delgar aus und zurück spielen hier keine Rolle. Beides kann daher in unserer Arbeitskarte berücksichtigt werden. Den fertigen Plan zeigt Abbildung 1.3.

Abbildung 1.3
Landkarte mit durch virtuelle
Orte ersetzten Knotenpunk-
ten

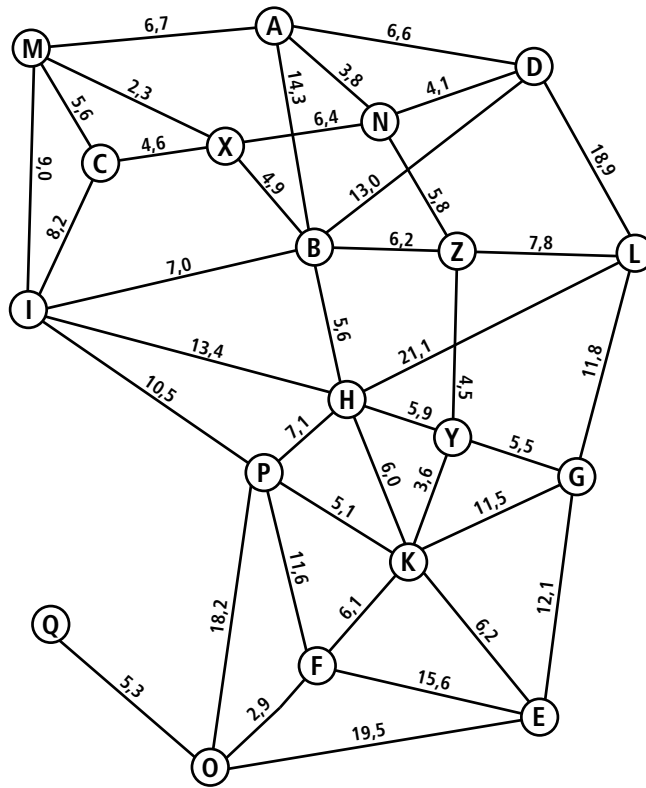


Leonhard Euler (1707 – 1783)
Euler hat schon 1736 als Erster
Wegeprobleme durch abstrakte
Darstellung vereinfacht, als
er das Königsberger Brücken-
problem löste: „Gibt es einen
Rundweg, der alle sieben
Brücken über den Fluss Pregel
genau einmal überquert und
wieder zum Ausgangspunkt
führt?“ Euler bewies, dass es
keinen solchen Weg geben
kann.

Die Städte sind immer noch auf ihrer geographischen Position eingezeichnet. Dadurch ergeben sich an manchen Stellen Ballungszentren. Die Straßenführung wird unübersichtlich. Die geographische Position der Städte haben wir jedoch weiter vorne als irrelevant deklariert. Um es Ihnen so einfach wie möglich zu machen, habe ich die Karte in Abbildung 1.4 etwas entzerrt.

Bitte überprüfen Sie, dass die Inhalte immer noch übereinstimmen, also die Verbindungen zwischen den Städten gleich sind und die gleiche Längenangabe aufweisen. Lediglich die Darstellung hat sich geändert. Sie können auch erkennen, dass es sich noch um die gleiche Karte handelt wie am Anfang des Kapitels – lediglich mit weniger Detailinformationen.

Mit dieser Karte werden wir ab jetzt weiterarbeiten, daher ist sie am Ende des Kapitels als Abbildung 1.K1 nochmals besonders groß abgedruckt – zum Beispiel als Kopiervorlage. Sie können sich vorstellen, dass wir zur Bestimmung des kürzesten Weges auch viel rechnen müssen, daher habe ich es uns zur Übung auch noch etwas einfacher gemacht und die einzelnen Wegstrecken gerundet.



Dijkstra und die Ameisen

Wie ermitteln wir denn aber nun den kürzesten Weg von Imstadt nach Oppenheim?

Der direkte Ansatz, immer ganze Wege zu betrachten, ist ja bereits aufgrund des hohen Aufwands gescheitert. Versuchen wir etwas anderes – vielleicht können wir ja hier von der Natur lernen:

Ein Stamm Ameisen hat auf der Suche nach Futter ein ähnliches Problem: Eine Kundschafterin findet ein großes Stück Fleisch. Welchen Weg sollen die Arbeiterinnen nehmen, um die Beute am schnellsten zu sichern?

Setzen wir also den Stamm Ameisen auf unseren Ausgangspunkt Imstadt bzw. ①. Fünf Wege führen von dort weg, also teilen sich unzählige Ameisen auf, um diese zu erkunden. Wir nehmen jetzt einmal an, dass alle Ameisen gleich schnell sind, zum Beispiel 1 km pro Minute (okay, für unser Problem setzen wir offenbar Turbo-Ameisen ein ...).

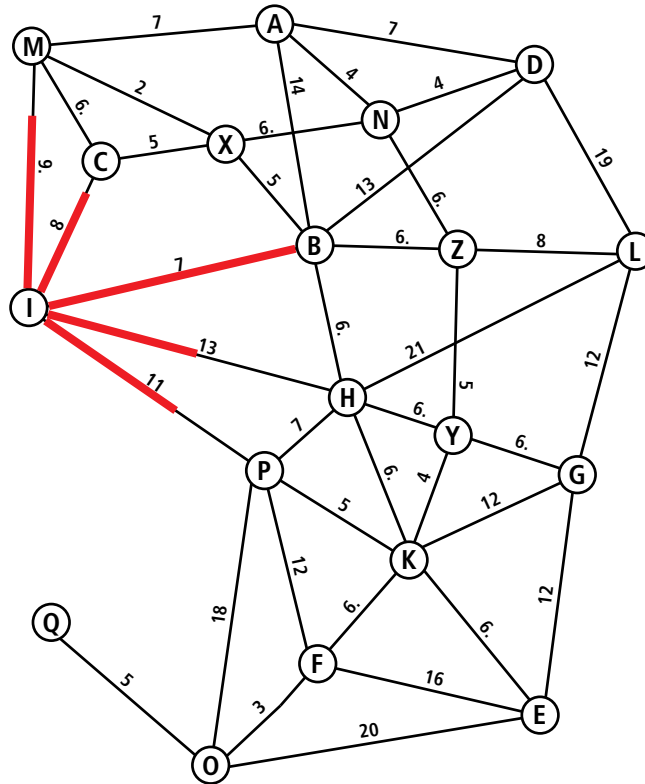
Mit dem Finger auf der Landkarte verfolgen wir den Weg der Ameisen. Abbildung 1.5 zeigt in Rot ihren Fortschritt nach 7 Minuten.

Die Ameisen haben ② erreicht. Auf den anderen Strecken sind sie noch auf dem Weg. Bitte lassen Sie sich nicht davon täuschen, dass die Strecke mit 7 km am längsten aussieht: Wir verwenden ja explizit keinen maßstabstreuen Plan!

Was haben wir dadurch bisher von den Ameisen gelernt?



Abbildung 1.5
Die Ameisen auf dem Weg
von Imstadt aus



Genau! Um von ① nach ② zu kommen, gibt es garantiert keinen günstigeren Weg als den mit 7 km. Die Ameisen haben ja sämtliche bisher für sie möglichen Wege ausprobiert und sind nach 7 km zuerst bei ② angekommen.

Wie geht es jetzt weiter? Die Ameisen, die bisher nirgendwo angekommen sind, setzen ihren Weg einfach fort. Die Ameisen bei **(B)** teilen sich erneut auf: Wieder sind fünf Wege möglich. Den bisherigen Erfolg dokumentieren sie, indem sie die bisher zurückgelegte Strecke bei **(B)** vermerken. Abbildung 1.6 zeigt den Plan der Ameisen.

Nach insgesamt 8 Minuten kommt der nächste Ameisentrupp bei ③ an. Die Insekten sehen, dass sie die Ersten sind, markieren die Strecke, verzeichnen die Anzahl der bisher gelaufenen Kilometer und teilen sich auf die zwei bei ③ weitergehenden Wege auf.

Am Ende der neunten Minute kommt dann auch der Trupp bei (M) als Erster an. Auch dieser Weg wird vermerkt (siehe Abbildung 1.7). Von hier aus sind drei weitere Strecken zu erkunden.

So weit verlief alles nach dem gleichen Schema. Die kürzesten Strecken zu den Städten **(B)**, **(C)** und **(M)** stehen nun fest.

Vielleicht haben Sie bemerkt, dass nun Ameisentrupps sowohl von **M** als auch von **C** ausgehend unterwegs sind – zwischen beiden Städten auf Kollisionskurs.

Was passiert jetzt, wenn sie sich irgendwo auf der Strecke dazwischen begegnen? Welche Informationen können sie austauschen? Bringt ihnen das etwas für ihr Ziel, das Gelände zu erkunden?



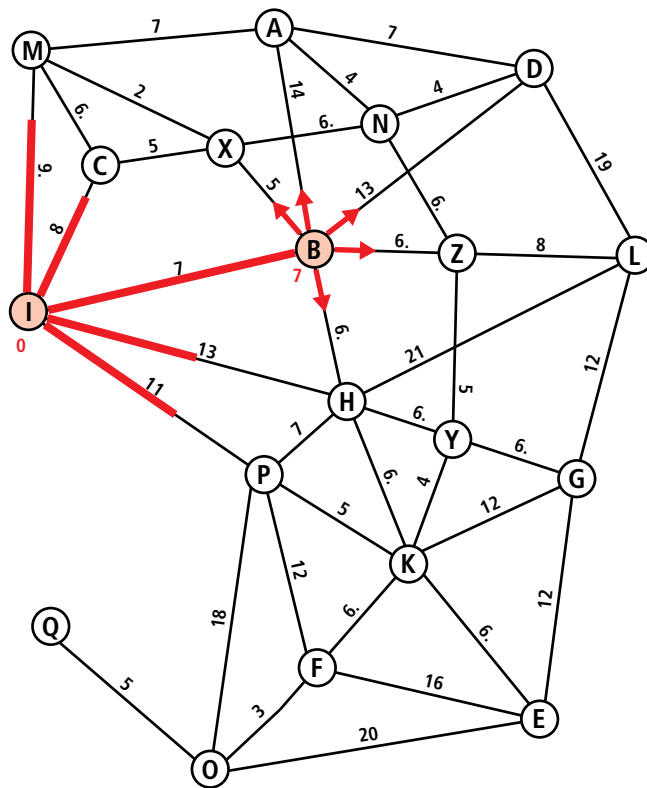


Abbildung 1.6
Die Ameisen bewegen sich von Budingen aus in alle Richtungen.

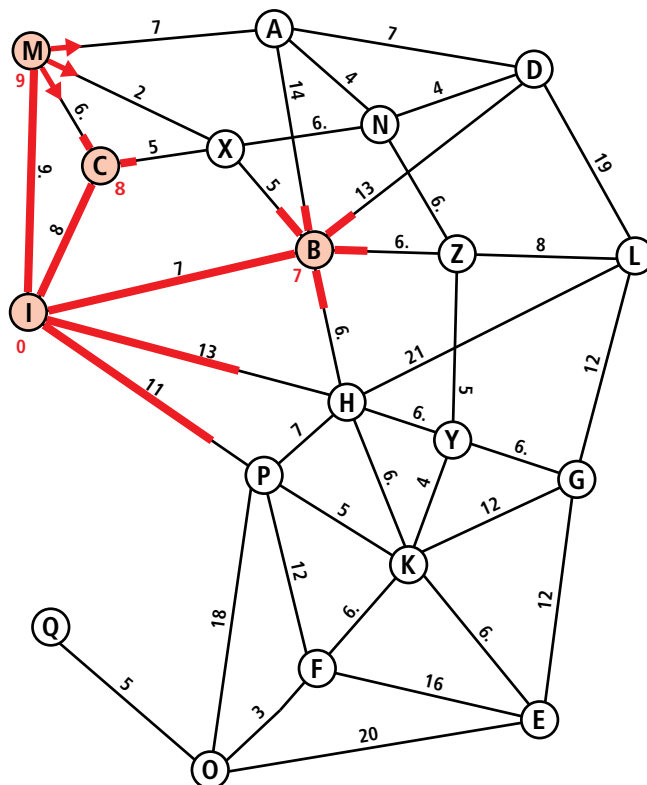


Abbildung 1.7
Drei weitere Wege werden von Morbach aus erkundet.

Richtig! Der Trupp von ③ weiß, dass dieses Ziel bereits erreicht ist, die kürzeste Strecke dorthin also feststeht. Der Trupp, der von ④ kommt, kann das Gleiche von seinem Ausgangspunkt berichten. Es ist also sinnlos, weiterzumarschieren. Die Strecke wird als „unbrauchbar“ markiert, die Ameisen können wieder zu ihrem Stamm zurück, sie sind sozusagen aus dem Rennen. Abbildung 1.8 zeigt das gleich mit einem dicken roten Kreuz.

Als Nächstes kommen zwei Erkundungstrupps gleichzeitig an: In der 11. Minute erreichen die Ameisen ⑤ und ⑥. ⑤ erreichen sie auf direktem Wege von ① aus. Bei ⑥ kommt der Trupp an, der über ④ unterwegs ist (9 km bis ④ plus 2 bis ⑥).

Wieder teilen sich die Ameisen auf. Von ⑥ gibt es nur noch einen erfolgversprechenden Weg, bei den anderen treffen sie recht schnell auf Kameraden und geben die Strecke auf. Die von ⑤ ausgehenden Touren sind alle noch offen. Abbildung 1.9 zeigt den aktuellen Stand.

Haben Sie das Prinzip verstanden?

Statt immer nur einen Weg auszuprobieren und wieder zu verwerfen, wenn sich ein besserer gefunden hat, erkunden die Ameisen gleichzeitig alle sich bietenden Möglichkeiten.

Kommen sie als Erste bei einer Stadt an, wissen sie, dass der genommene Weg der kürzeste ist, denn sonst wäre ja ein anderer Erkundungstrupp bereits da (zur Erinnerung: Alle bewegen sich mit der gleichen Geschwindigkeit.)

Treffen die Ameisen irgendwo auf Artgenossen, dann wissen sie, dass ihre Reise zu Ende ist, weil sie sich gegenseitig ein „Ich bin schon da“ berichten können. Andere haben also das anvisierte Ziel früher erreicht.

Führen Sie zur Übung das Verfahren noch zu Ende und zeichnen Sie den Weg der Ameisen, die gefundenen Strecken sowie verworfene Wege in die Landkarte ein!

Können Sie sich vielleicht gleichzeitig auch schon vorstellen, wie ein Computer das Ameisenprinzip adaptiert?



Die Lösung wird in Abbildung 1.10 dargestellt. Haben Sie ein anderes Ergebnis? Kein Problem: Die Beispielaufgabe ist bewusst so gestellt, dass es zu manchen Orten unterschiedliche Wege mit gleicher kürzester Strecke gibt. So kommt man etwa zu ⑧ sowohl über ①③⑧ als auch direkt über ①⑧ in jeweils 13 km. Falls Ihre Lösung jedoch Abweichungen in den rot eingetragenen Entfernungen aufweist, sollten Sie diese nochmal kontrollieren.

Welche Informationen haben wir dadurch jetzt eigentlich gewonnen?

Um von Imstadt zu einem beliebigen anderen Ort zu kommen, folgen Sie dem Pfad der Ameisen:

Von Imstadt nach Oppenheim kommt man so am günstigsten über Pappstadt, Krupsing und Flughafen. Die Gesamtstrecke beträgt 25 km. Ein günstigerer Weg existiert nicht!

Sie haben aber nicht nur die ursprüngliche Aufgabe gelöst, sondern sozusagen als Abfallprodukt noch die kürzesten Wege von Imstadt zu allen weiteren Städten ermittelt.

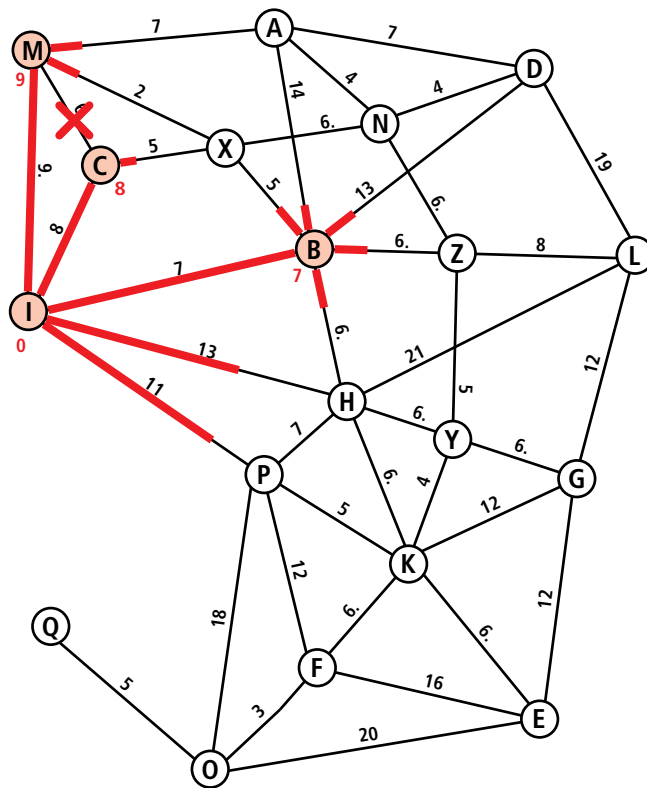


Abbildung 1.8
Der Weg zwischen Morbach und Chelzey wird als unbrauchbar markiert.

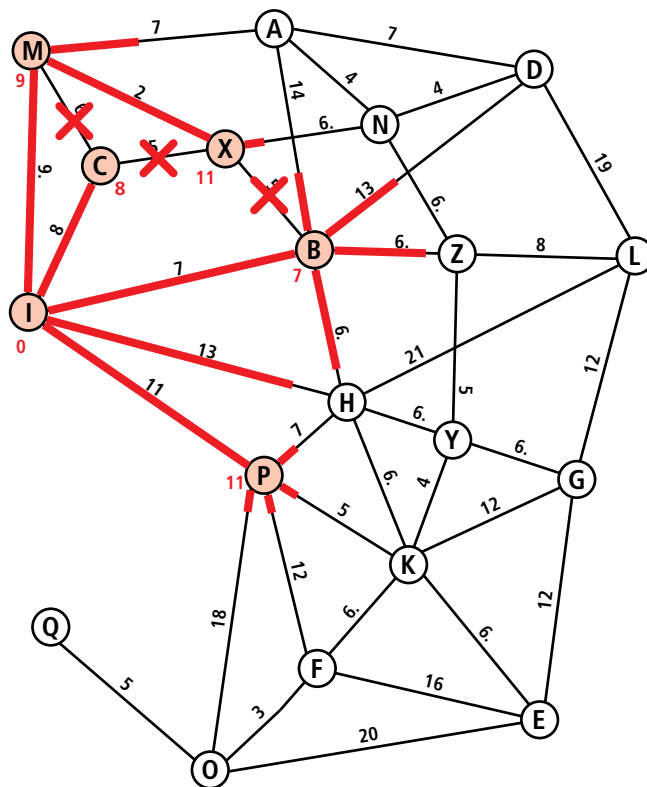
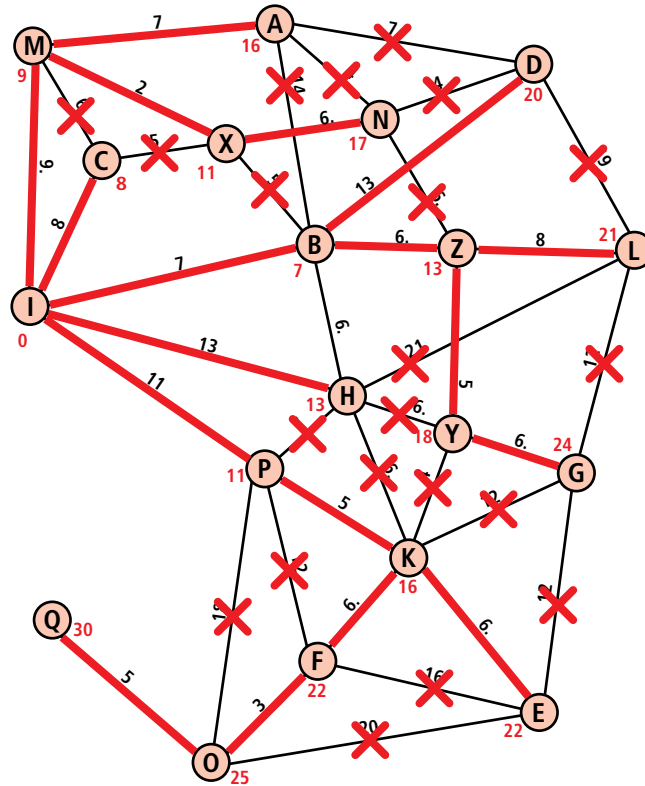


Abbildung 1.9
Weiterer Fortschritt der Ameisen

Abbildung 1.10
Die fertig von den Ameisen
erkundete Landkarte



So fährt man zum Beispiel nach Giwelau am besten über Buding und die beiden Autobahnkreuze, die wir mit **Z** und **Y** bezeichnet haben.

Beachten Sie bitte auch die „0“, die nun der Vollständigkeit halber bei Imstadt steht, denn der Weg von Imstadt nach Imstadt beträgt ja ganz offensichtlich 0 km.

Warum ist das Ameisen-Prinzip für einen Informatiker interessant?

- Es führt in absehbarer Zeit zum Ziel. Da die Ameisen ständig in Bewegung sind und keine Strecke doppelt gehen, müssen sie recht bald alles erkundet haben (maximal nach der Zeit, die dem kürzesten Weg zur am weitesten entfernten Stadt entspricht).
- Es werden immer wieder die gleichen, sehr einfachen Anweisungen benutzt, um die Ameisen zu steuern:
 - ❶ Teile den Trupp auf und folge allen Routen.
 - ❷ Wenn ein Ort erreicht wird: kürzeste Strecke dorthin gefunden, weiter bei ❶.
 - ❸ Wenn man einem anderen Trupp begegnet: Strecke verwerfen, Ende.

Solche Verfahren lassen sich (normalerweise) gut und einfach für den Computer umsetzen. Eine entsprechend formulierte Vorschrift nennt man dann Algorithmus.

Algorithmus

Ein Algorithmus ist eine Handlungsvorschrift zur Lösung eines Problems bzw. einer Kategorie von Problemen. Diese Handlungsvorschriften lassen sich im Allgemeinen in ein Computerprogramm umsetzen. Hierfür müssen sie hinreichend genau formuliert sein.

Algorithmus

Wussten Sie, dass die Herkunft dieses Begriffes nie richtig geklärt wurde? Entweder kommt er vom griechischen „arithmos“, was Zahl bedeutet, oder er ist der veränderte Name des persisch-arabischen Mathematikers „Al-Charismi“, der – englisch ausgesprochen – in etwa wie „algorithm“ klingt. Erste Algorithmen wurden übrigens bereits lange vor der Zeitenwende hauptsächlich von griechischen Mathematikern entwickelt: Etwa im 4. Jahrhundert v. Chr. das euklidische Verfahren zur Bestimmung des größten gemeinsamen Teilers zweier Zahlen oder im 3. Jahrhundert v. Chr. das Sieb des Eratosthenes zur Ermittlung von Primzahlen. Beide Algorithmen werden heute noch in Computern eingesetzt!

Wie lösen also unsere Routenplaner im Auto das Problem des kürzesten Weges? Enthalten sie einen Ameisen-Simulator und spielen die genaue Vorgehensweise nach?

Sicherlich wäre das vorteilhafter als die ganz zu Anfang beschriebene „Brute-Force-Methode“. Trotzdem ist der Informatiker hier erneut gefragt, das gefundene Verfahren für den Computer zu optimieren.

Können Sie sich vorstellen, wie?

Das Ameisen-Prinzip liegt uns Menschen sehr, weil wir uns gut vorstellen können, wie die kleinen Krabbeltiere die Pfade erkunden. Dadurch erschließt sich auch recht deutlich, warum das Verfahren wirklich die günstigsten Wege hervorbringt.

Wann immer wir so ein „aus dem Leben gegriffenes“ Verfahren im Computer benötigen und es daher in Bits und Bytes umsetzen, hilft wieder das Prinzip der Abstraktion:

Was ist vom Ameisen-Prinzip für die Problemlösung relevant? Bedenken Sie: Computer benötigen keine Anschauung, diese Anteile sind also zu eliminieren.

Ob Edsger W. Dijkstra, zuletzt Professor in Texas, 1959 über eine Ameisenstraße lief, ist unbekannt. Er hat jedoch bereits zu diesem Zeitpunkt ein Verfahren vorgestellt, das unser Ameisen-Prinzip im Computer zur Berechnung eines kürzesten Weges nutzt. Es ist bis heute nach ihm benannt.

Doch versuchen Sie zunächst einmal selbst, ein solches Verfahren herauszufinden. Vielleicht kommen Sie ja sogar auf eine bessere Methode, die dann nach Ihnen benannt wird.



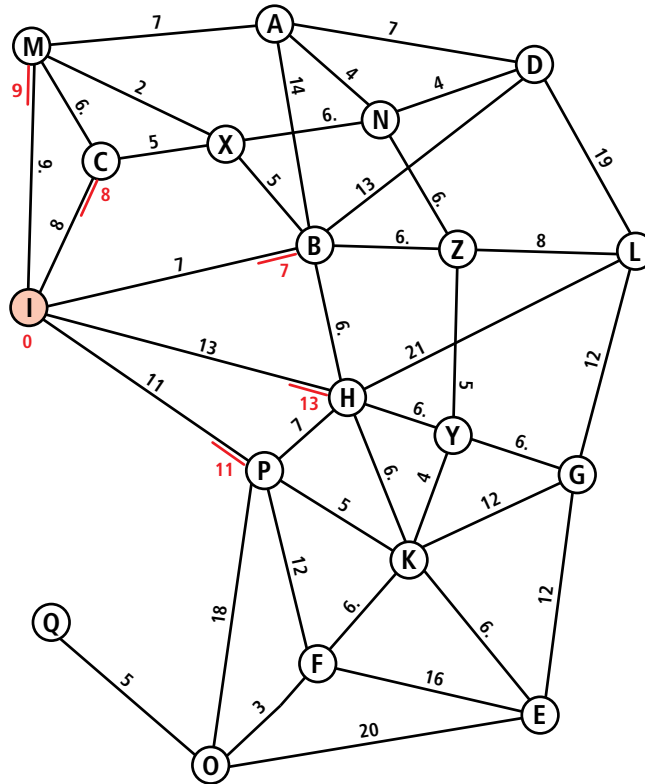
Der Dijkstra-Algorithmus

Fangen wir noch einmal ganz von vorne an: Sie sitzen in Imstadt und wollen nach Oppenheim.

Die Ameisen unseres letzten Experiments liefen nun einfach in alle anderen direkt erreichbaren Städte, um zu ermitteln, wie lange sie dorthin unterwegs sind. Die hierfür notwendigen Zeiten braucht ein Computer jedoch gar nicht zu ermitteln, er liest einfach die Strecken ab. Wie aus Abbildung 1.11 ersichtlich, sind dies ja einfach die verzeichneten Längen der Verbindungslinien zwischen den Städten.

In der Graphik ist außerdem bei allen Zielpunkten markiert, woher diese kommen, damit die beschriebene Entfernung zutrifft.

Abbildung 1.11
Potentiell vom Start in Im-
stadt erreichbare Städte und
die Weglängen dorthin



Wie ging es mit den Ameisen weiter? Der Trupp, der zuerst bei einer Stadt ankam, markierte die genommene Strecke als „günstig“ und verteilte sich auf die umliegenden Strecken.

Für den Computer ist nun gar kein Problem, diese Stadt zu bestimmen: Es ist diejenige, die mit der kleinsten Zahl bezeichnet ist, also **(B)** mit der Zahl 7. Abbildung 1.12 zeigt das, der Knoten ist mit roter Farbe als „besucht“ markiert.

Von dieser Stadt aus werden wiederum die Entfernungen zu allen Nachbarn bestimmt. Da die Ameisen jedoch bis nach **(B)** bereits 7 km unterwegs waren, müssen diese addiert werden. Versuchen Sie es! Stoßen Sie auf ein Problem?

Genau! Für **(X)**, **(A)**, **(D)** und **(Z)** können wir die Methodik einfach durchführen, aber was ist mit **(H)**? Hier steht bereits ein Wert! Wie ist hier zu verfahren?

Ziehen wir die Ameisen zurate: Der Weg von **(I)** über **(B)** nach **(H)** ist 13 km lang, der direkte Weg von **(I)** nach **(H)** ebenfalls 13 km. Die mögliche neue Strecke ist zwar nicht schlechter, aber auch nicht besser – wir können die alte Markierung behalten.

Daher gilt für das Dijkstra-Verfahren die folgende Regel für den Fall, dass an einer Nachbarstadt bereits eine Zahl steht:

- Wenn die Zahl, die neu hingeschrieben werden soll, kleiner ist, dann wird die alte durch die neue Zahl ersetzt und dementsprechend auch die Marke, woher die Ameisen kommen.
- Wenn die neue Zahl größer oder gleich ist, passiert gar nichts.

Ergebnis ist daher die Karte aus Abbildung 1.13.

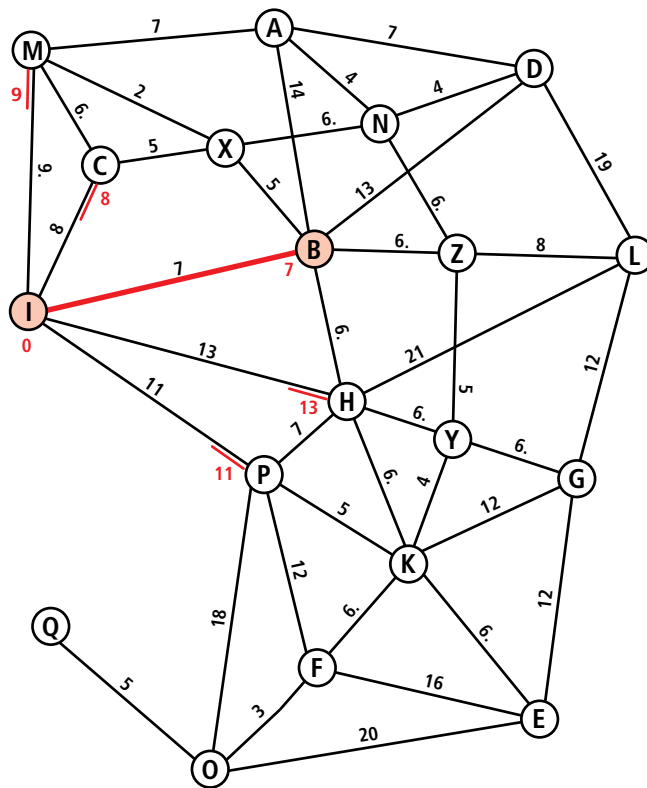


Abbildung 1.12
Der erste kürzeste Weg von Imstadt aus wurde ermittelt.

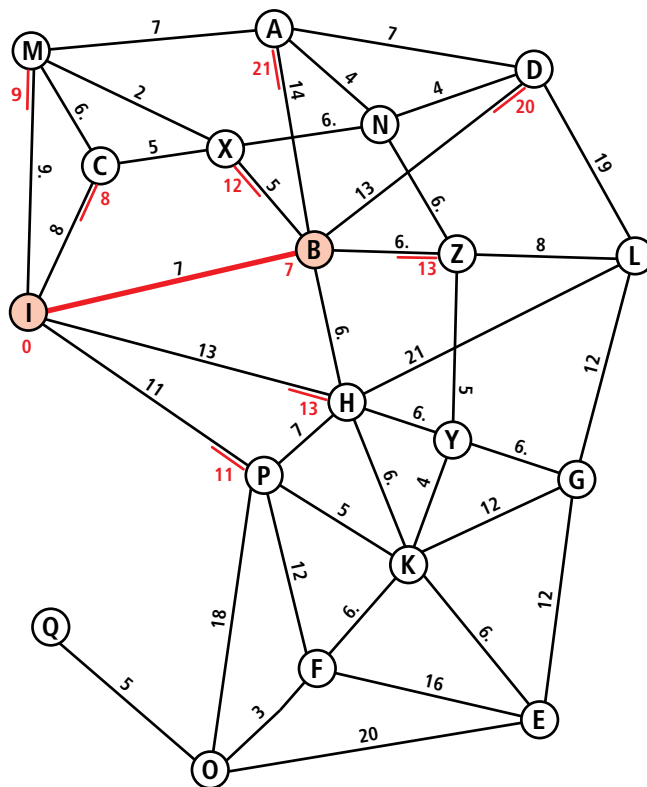
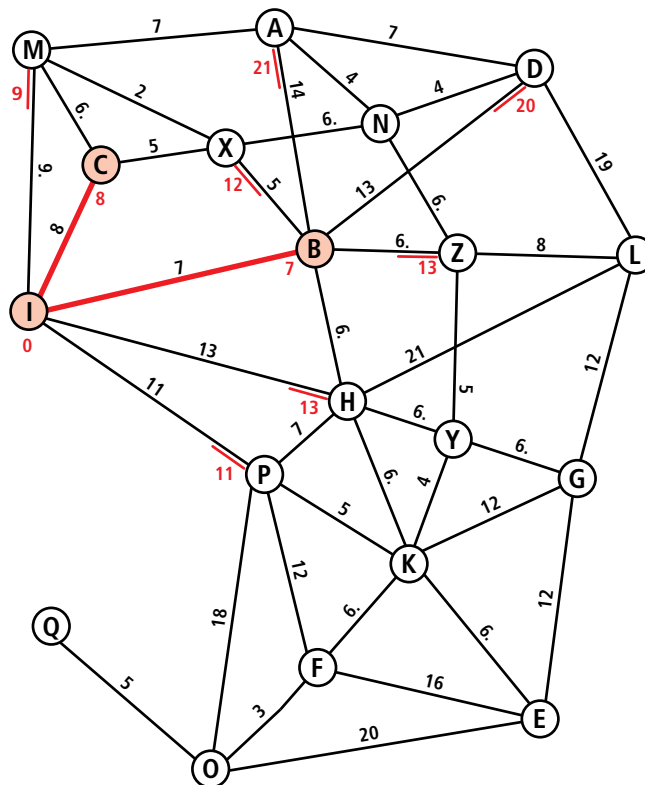
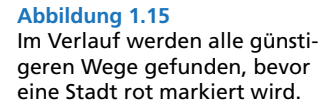


Abbildung 1.13
Erreichbare Städte sind durch eine rote Zahl mit der gefundenen Weglänge markiert.

Algorithmen spielen in der Informatik eine sehr große Rolle: Ein Großteil der vorkommenden Aufgaben aus Wirtschaft und Wissenschaft lässt sich mit Algorithmen

Erneut wurde ein kürzester Weg gefunden.

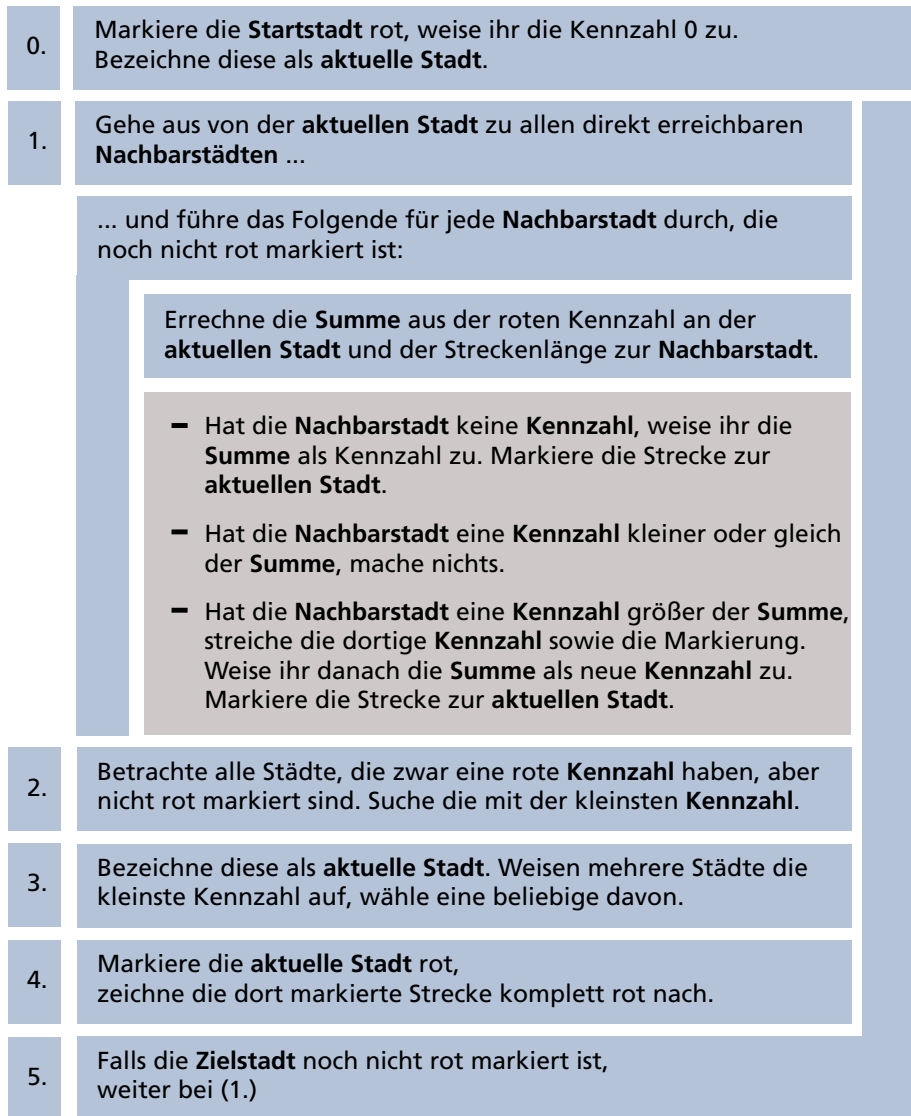




Es gibt auch diverse formale Methoden, die Algorithmen zu Papier zu bringen. Eigentlich geht es jedoch darum, die Beschreibung so genau zu machen, dass jemand anderes den Algorithmus danach durchführen kann.

An einem Blockdiagramm wie diesem erkennt man besonders gut Wiederholungen von Anweisungsfolgen. Es ist aber genauso gut, einfach Text zu schreiben oder verschiedene Anweisungen mit Pfeilen zu verbinden. Alles ist erlaubt, was Ihnen ermöglicht, sich strukturiert Gedanken über den Ablauf zu machen. Wenn das Ergebnis dann auch noch für andere lesbar und verständlich ist – umso besser!

Abbildung 1.16
Algorithmus als Blockdiagramm



Edsger Wybe Dijkstra (1930 – 2002) arbeitete nach der Ausbildung zunächst als Professor an der Universität in Eindhoven und wechselte 1984 nach Texas. Neben seinen Arbeiten zur Berechnung des kürzesten Weges hat er auch einen wesentlichen Beitrag zur Einführung der strukturierten Programmierung geleistet. Dafür erhielt er 1972 auch den Turing-Preis, der das Pendant des Nobelpreises für Informatiker ist.

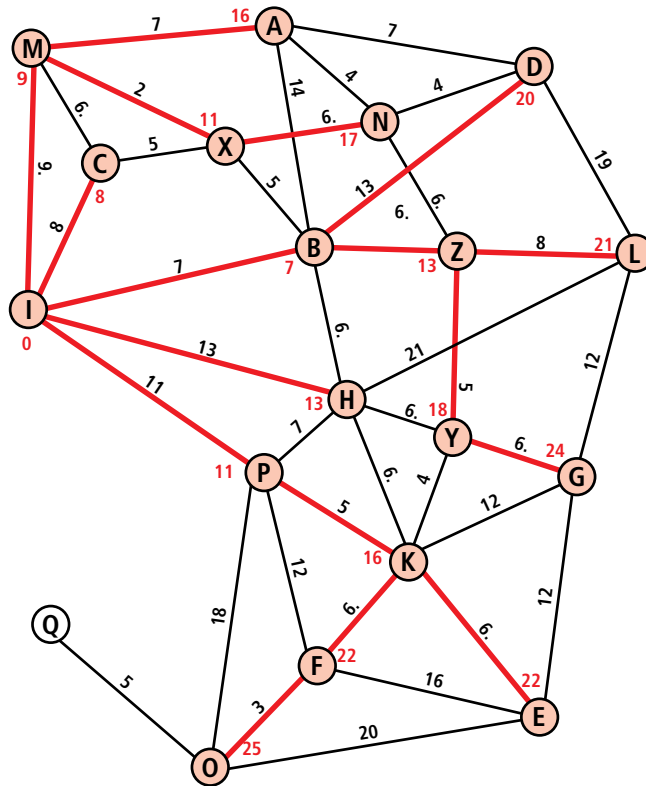
Das Ergebnis der Weiterführung sehen Sie in Abbildung 1.17. Erkennen Sie die Gemeinsamkeiten mit dem Ergebnis bei Nutzung des Ameisen-Prinzips?

Bemerken Sie, dass alle Städte bis auf ☉ rot markiert sind? In unserer Algorithmusbeschreibung haben wir festgelegt, dass wir aufhören dürfen, wenn die Zielstadt erreicht ist. Auf diese Weise sparen wir uns eventuell, einige Städte zu betrachten. Und auch wenn trotzdem fast alle Städte rot markiert sind, hat die Durchführung nicht allzu viel Zeit in Anspruch genommen.

Üben Sie nun mit dem neuen Algorithmus und bestimmen Sie den kürzesten Weg von Lupera nach Eindhoven. Wie wäre es, wenn Sie dafür die genaueren Wegstreckenangaben nutzen, die am Ende des Kapitels in Abbildung 1.K2 als Kopiervorlage zu finden sind?



Abbildung 1.17
Fertige Landkarte nach Durchlauf des Dijkstra-Algorithmus



Wenn Sie alles richtig gemacht haben, sollte in etwa Abbildung 1.18 herauskommen. Die durchgestrichenen roten Zahlen und Markierungen sind absichtlich noch sichtbar, um den Verlauf besser nachvollziehen zu können. Ein Routenplaner würde Ihnen die Strecke als wahrscheinlich präsentieren wie in Abbildung 1.19.

Weil es ab und zu Verwechslungen gibt: Bitte denken Sie daran, dass das dargestellte rote Netz nur die kürzesten Wege von Lupera aus repräsentiert! Wenn Sie es zum Beispiel nutzen würden, um über rote Strecken von Imstadt nach Fluxing zu gelangen, wäre dieser Weg über Budingen, die Kreuze ② und ③ sowie Krupsing mit 27,4 km viel zu weit. Aus Abbildung 1.K2 können Sie ablesen, dass dies auch in 21,7 km zu schaffen ist.

Dijkstra bestimmt immer die kürzesten Wege von einem Startpunkt zu vielen anderen Punkten, ist aber nicht geeignet, um daraus Schlüsse auf die Situation mit anderen Startpunkten zu ziehen.

Herzlichen Glückwunsch – Sie wissen nun, wie so ein Routenplaner funktioniert! Da Sie sicher ganz heiß darauf sind, Ihr neues Wissen gleich praktisch anzuwenden, hier gleich die nächste Aufgabenstellung ...

Abbildung 1.18

In 22,1 km kommt man von
Lupera nach Eindhofen.

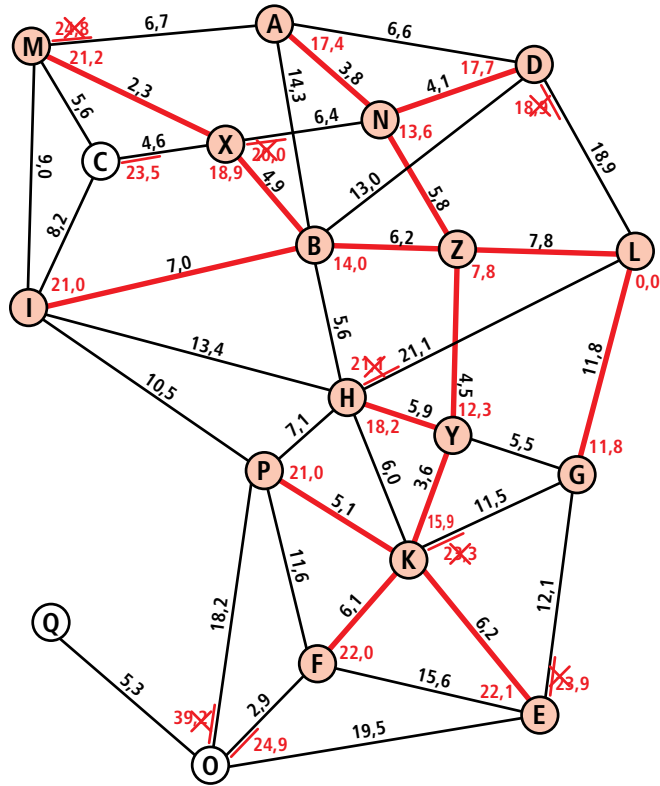
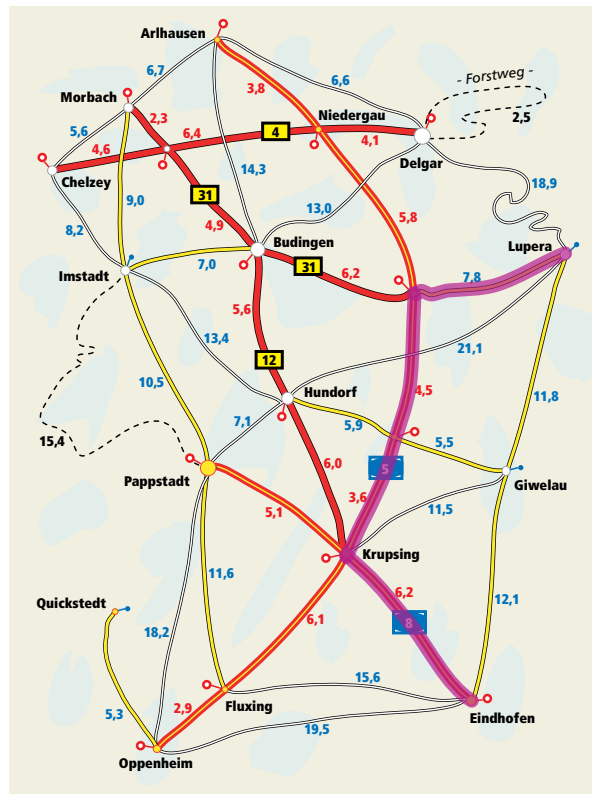


Abbildung 1.19

Ein Navigationsgerät würde
das Ergebnis eventuell auf
diese Weise visualisieren.



Die Schilda-Rallye

Sehen Sie hier in Abbildung 1.20 den Touristen-Stadtplan von Schilda, mit seinen acht Hotels und vier Pensionen.

Wie es so kommt, hat der Stadtrat vor Kurzem beschlossen, alle Straßen zu Einbahnstraßen zu machen (ganz im Gegensatz zu den traditionellen Gepflogenheiten sind trotzdem noch alle Orte erreichbar).

Die Fahrzeuge von Schilda-Taxi warten vor den Hotels Adler und Gozo sowie auf dem Parkplatz der Pension Capitol. Aufgrund der neuen Verkehrsführung benötigen die Fahrer einen Plan, wie sie auf dem kürzesten Weg von ihrem Standort zu allen anderen Hotels kommen. Eine Entfernungstabelle ist auch zur Berechnung der neuen Tarife notwendig.

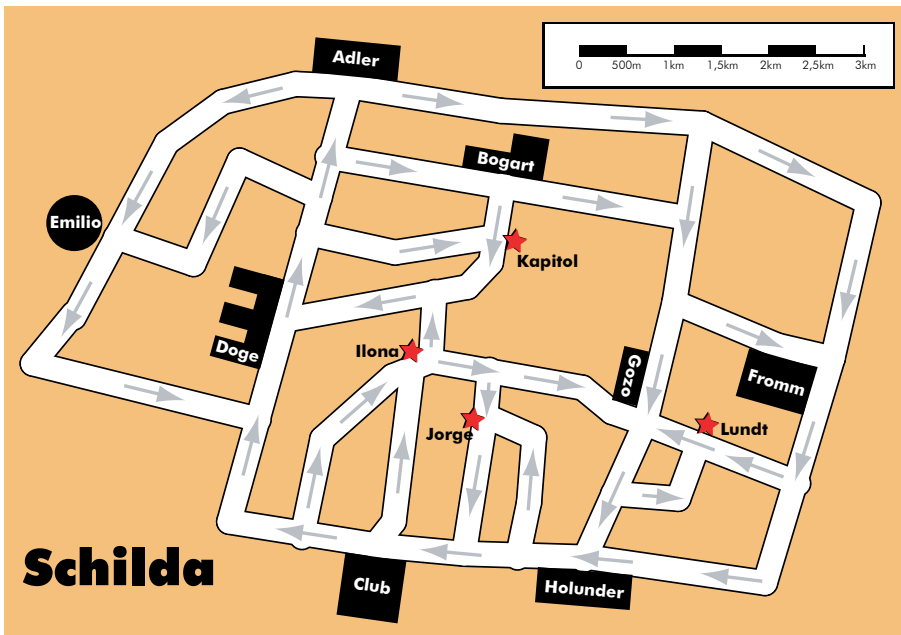


Abbildung 1.20
Stadtplan von Schilda

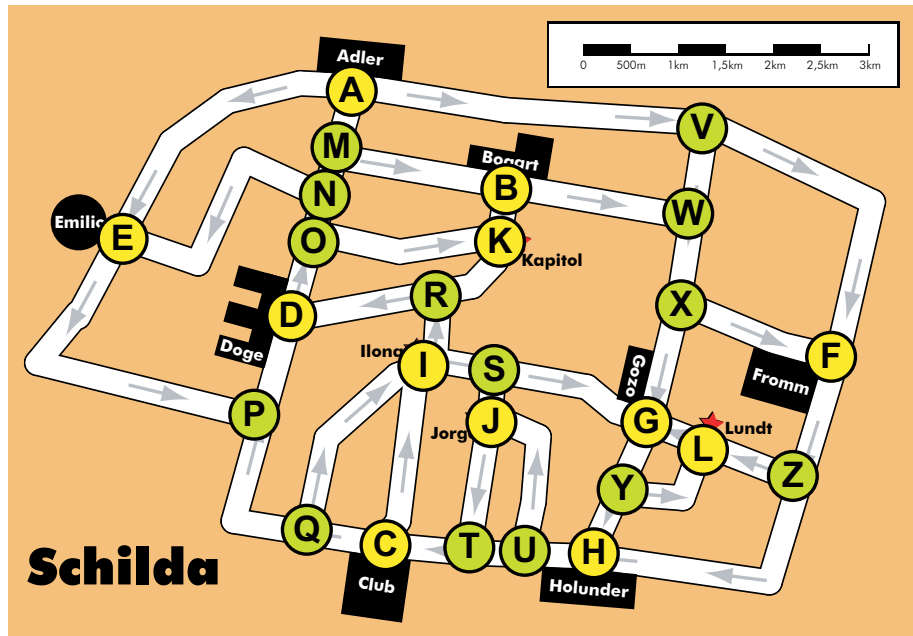
Ihre Aufgabe: Erstellen Sie drei Tabellen mit den Entfernungen der Hotels Adler und Gozo sowie der Pension Capitol zu allen anderen Gästehäusern.

Tipp: Gehen Sie zunächst genauso vor, wie Sie es gerade gelernt haben. Einen Unterschied gibt es allerdings. Erkennen Sie ihn? Versuchen Sie das Problem so weit zu lösen, wie Sie selbst kommen. Wenn es wirklich nicht mehr weitergeht, schauen Sie in die hier vorgeschlagene Lösung.



Zunächst wird wieder eine Abstraktion benötigt. Die Definition von Orten ist der erste Schritt. Selbstverständlich sind die Positionen aller Hotels relevante Orte. Relevant sind jedoch auch alle Orte, an denen sich die Straße teilt, also ein Abbiegen möglich ist.

Abbildung 1.21
 Stadtplan von Schilda mit
 markierten Straßenkreuzungen



In dieser Lösung nach Abbildung 1.21 habe ich die Orte für die Hotels (ihrem Namen entsprechend) A bis L genannt und in der Karte gelb markiert, Straßenkreuzungen ohne Hotel M bis Z mit einer hellgrünen Markierung. Andere Nomenklaturen sind selbstverständlich genauso möglich.

Haben Sie weitere Biegungen als „Orte“ markiert, etwa zwischen E und P oder zwischen V und F? Das ist kein Fehler, jedoch nicht notwendig, denn ein Auto kann sowieso nur dem Straßenverlauf folgen.

Welches ist der nächste Schritt? Stellen Sie einen Plan auf, ähnlich dem Plan für das letzte Routenplaner-Problem!



Der Unterschied zum letzten Problem sind die Einbahnstraßen – diese wichtige Information muss auch in einer abstrakteren Darstellung der Karte erfasst werden, aber das stellt ja kein Hindernis dar. Abbildung 1.22 auf der nächsten Doppelseite zeigt die Lösung.

Erinnern Sie sich noch an das Prinzip der Ameisen? Es lässt sich immer noch anwenden – nur müssen die Ameisen nun die Regeln der Einbahnstraßen beachten, also immer nur in Pfeilrichtung laufen. Für den Dijkstra-Algorithmus bedeutet dies, dass „Nachbarorte“ immer nur solche sind, die in Pfeilrichtung erreichbar sind.

Fangen Sie damit an, die Wege-Tabelle vom Hotel Adler aus zu erstellen!



Sie sind ja ganz sicher selbst auf die Lösung in Abbildung 1.23 auf der nächsten Doppelseite gekommen ...

Üben Sie nun weiter, indem Sie die Entfernungen vom Hotel Gozo zu jedem anderen Hotel und von der Pension Kapitoll aus bestimmen.



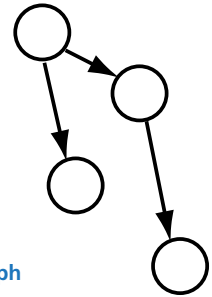
Die Lösung für das Hotel Gozo entnehmen Sie Abbildung 1.24.

Die Entfernungen von der Pension Kapitoll aus entnehmen Sie Abbildung 1.25.

Auch diese Abbildungen finden Sie auf der nächsten Doppelseite – als kleiner Schutz vor dem „Spicken“.

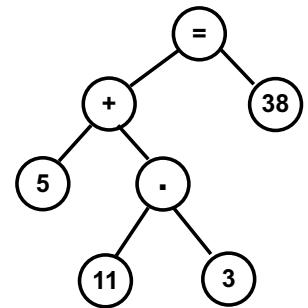
Was steckt dahinter?

Die besprochenen Probleme gehören in das Umfeld der sogenannten Graphenalgorithmen. Ein Graph besteht hierbei aus einer Menge von Knoten und einer Menge von Kanten, die zwischen den Knoten verlaufen. Graphen werden oft wie in diesem Beispiel dargestellt: die Knoten als Kringel und die Kanten als Linien oder Pfeile dazwischen. Man unterscheidet ungerichtete Graphen, also solche, bei denen die Verbindung zweier Knoten in beide Richtungen geht (ohne Pfeil), und gerichtete Graphen (mit Pfeil).



Graph

Wozu ist aber ein Graph gut? Wie andere Modelle der Informatik auch kann der Graph einen Ausschnitt der Wirklichkeit modellieren, um diese einfacher zu verstehen und zu analysieren. Prominentes Beispiel für einen Graphen ist die Landkarte bzw. der Stadtplan, wie hier im Kapitel gezeigt. Knoten sind hierbei die Orte, Kanten die Verbindungswege. Genauso könnte der Graph jedoch auch ein Computernetzwerk darstellen. Auch andere abstrakte Konzepte wie Produktionspläne oder eine mathematische Gleichung lässt sich per Graph sehr übersichtlich darstellen – in diesem Fall $5 + (11 \cdot 3) = 38$.



Mathematische Gleichung

Sowohl Knoten als auch Kanten können Markierungen tragen. Im Beispiel der Landkarte sind das Ortsnamen für die Knoten und Weglängen für die Kanten. Aufgrund der vielfältigen Anwendungen von Graphen gibt es auch eine Menge von Algorithmen, die auf Graphen arbeiten und Aufgaben – in der Informatik nennen wir sie oft „Probleme“ – mit Hilfe einer übersichtlichen Darstellung als Graph lösen. Der besprochene Dijkstra-Algorithmus ist hier ein sehr prominenter Vertreter.

Was zeichnet einen guten Algorithmus aus? Einerseits muss er natürlich die gestellte Aufgabe lösen. Wichtiges Kriterium ist jedoch außerdem, dass er die gestellte Aufgabe schnell löst und so auch bei großen Problemen nicht in die Knie geht.

Man rechnet daher aus, wie stark der Zeitbedarf mit der sogenannten Problemgröße ansteigt. Auch dies kann man am Beispiel der Landkarte sehr schön demonstrieren.

Eine brauchbare Problemgröße ist die Anzahl der Städte auf der Landkarte.

Rekapitulieren wir zunächst den anfänglich kurz erwähnten Brute-Force-Algorithmus zur Bestimmung des kürzesten Weges:

„Bestimme alle möglichen Wege vom Start zum Ziel und suche davon den kürzesten.“

Im schlechtesten Fall müssen also von einem Graphen alle von einem Punkt ausgehenden Wege bestimmt werden. Außerdem sind im schlechtesten Fall alle Knoten mit

Abbildung 1.22
Stadtplan von Schilda

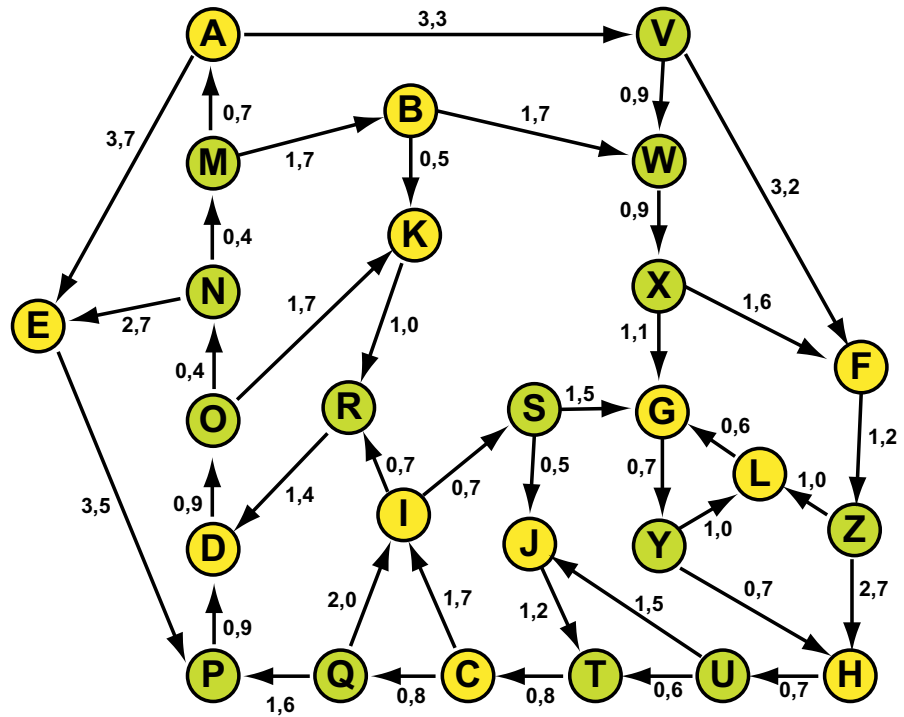
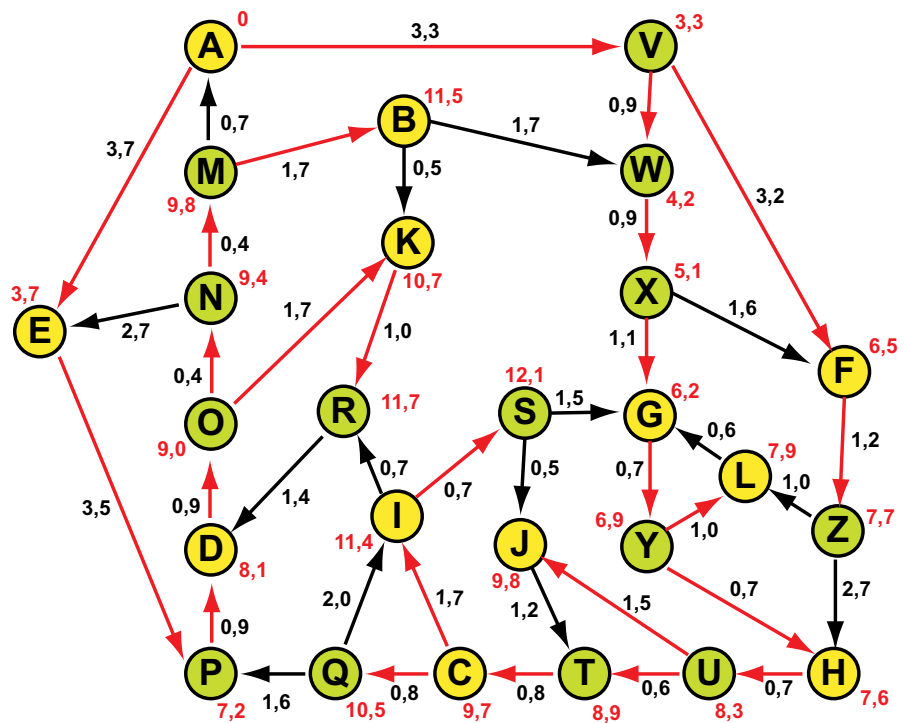


Abbildung 1.23

Kürzeste Entfernungen vom
Hotel Adler. Sie ergeben fol-
gende Tabelle:

Hotel	Entfernung
Adler	0,0
Bogart	11,5
Club	9,7
Doge	8,1
Emilio	3,7
Fromm	6,5
Gozo	6,2
Holunder	7,6
Ilona	11,4
Jorge	9,8
Kapitol	10,7
Lundt	7,9



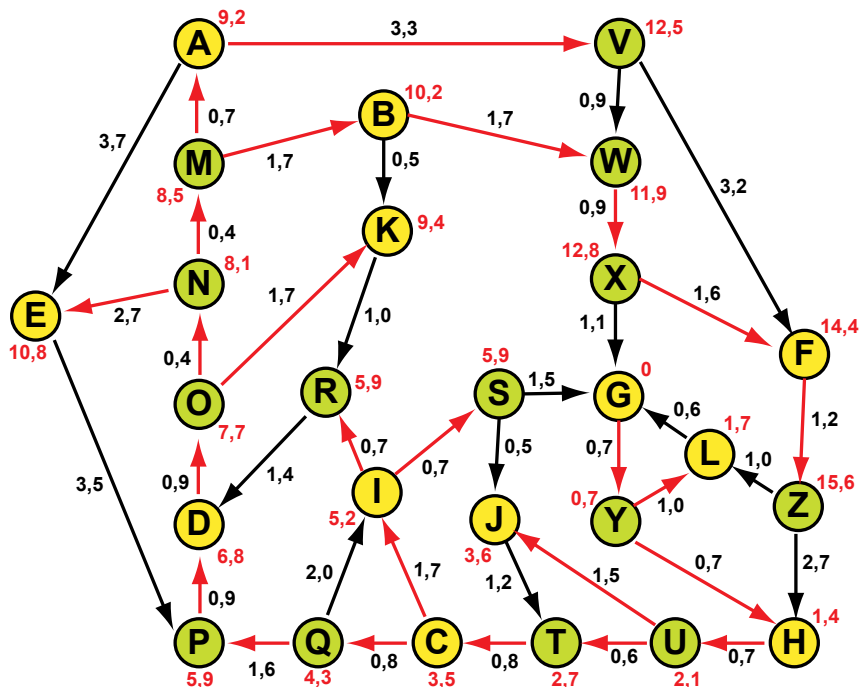


Abbildung 1.24
Kürzeste Wege vom Hotel Gozo. Sie ergeben folgende Tabelle:

Hotel	Entfernung
Adler	9,2
Bogart	10,2
Club	3,5
Doge	6,8
Emilio	10,8
Fromm	14,4
Gozo	0,0
Holunder	1,4
Ilona	5,2
Jorge	3,6
Kapitol	9,4
Lundt	1,7

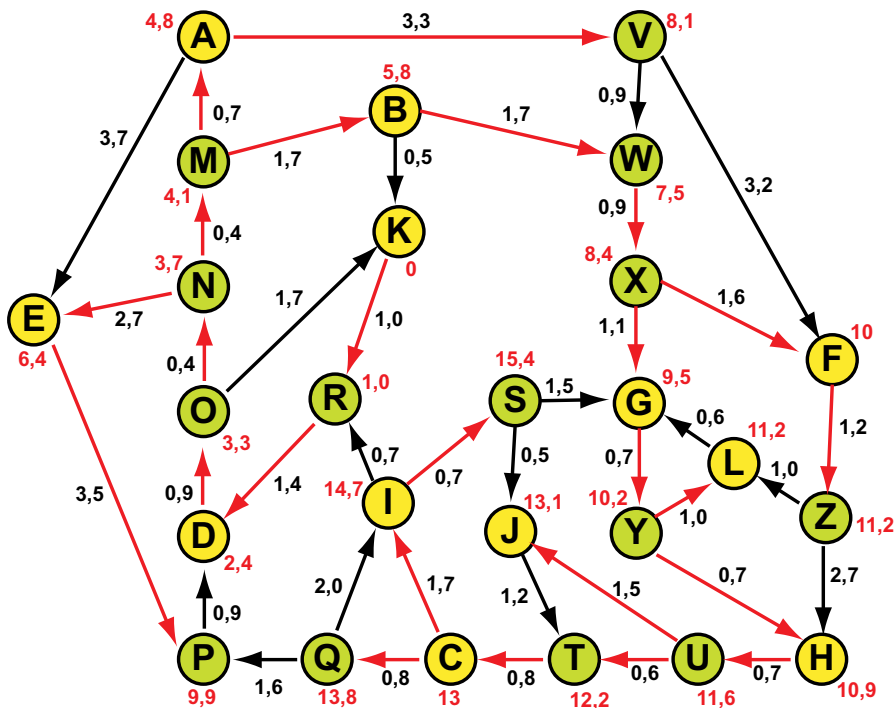
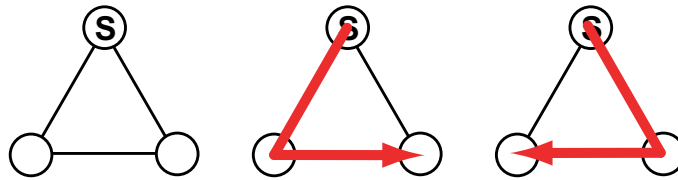


Abbildung 1.25
Kürzeste Wege vom Hotel Capitol. Sie ergeben folgende Tabelle:

Hotel	Entfernung
Adler	4,8
Bogart	5,8
Club	13
Doge	2,4
Emilio	6,4
Fromm	10,0
Gozo	9,5
Holunder	10,9
Ilona	14,7
Jorge	13,1
Kapitol	0,0
Lundt	11,2

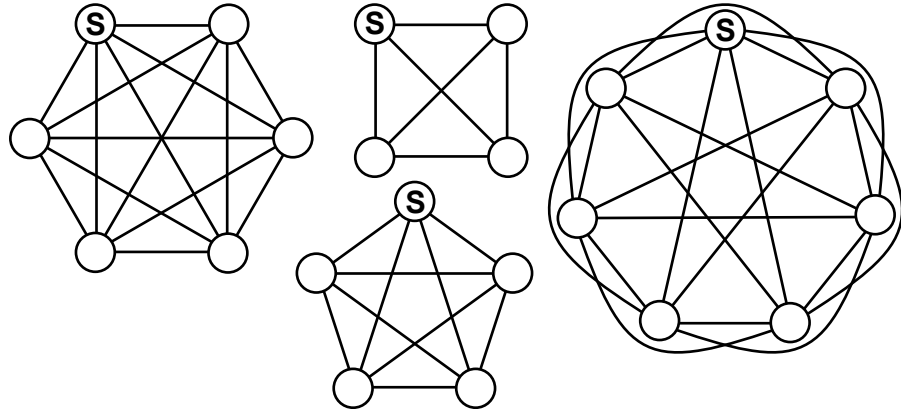
Abbildung 1.26
Von S gehen zwei unterschiedliche Wege aus.



allen anderen Knoten verbunden (der Fachausdruck dafür ist „vollständiger Graph“). Für drei Knoten ist es noch kein Problem, die Anzahl möglicher Wege vom Startpunkt S aus zu bestimmen. Abbildung 1.26 zeigt die zwei Lösungen.

Versuchen Sie das jetzt auch mit den Graphen aus Abbildung 1.27 mit vier bis sieben Knoten.

Abbildung 1.27
Vollständige Graphen mit vier bis sieben Knoten. Bei einem vollständigen Graphen ist jeder Knoten mit jedem anderen durch eine Kante verbunden.



Knoten	Schritte
3	4
4	18
5	96
6	600
7	4.320
8	35.280
9	322.560
10	3.265.920
15	1.220.496.076.800
20	2.311.256.907.767.808.000
50	29.805.811.337.679.110.482.740.356.002.743.473.467.490.088.737.581.301.760.000.000.000

Falls Sie angefangen haben, die Lösung zeichnerisch zu bestimmen, werden Sie bereits festgestellt haben, wie viel länger die Suche dauert, wenn auch nur ein Knoten hinzukommt. Man kann die Anzahl der Möglichkeiten auch rechnerisch bestimmen:

Für den Graphen mit vier Knoten gilt, dass man ihn aus zwei Komponenten zusammensetzen kann: einem einzelnen Knoten **S** plus einem Graphen mit drei Knoten **○** kann dann mit dem Graphen verbunden werden, wie in Abbildung 1.28 dargestellt.

Da der „bekannte“ Graph drei Knoten besitzt, kann vom „neuen Knoten **S**“ auf drei Arten ein Weg zum bekannten Graphen begonnen werden, nämlich jeweils zu einem der Knoten. Stellen Sie sich vor, dass im 3er-Graphen dann auf die bereits ermittelte Weise ein Weg gesucht wird.

Daher ist die Anzahl möglicher Wege im 4er-Graphen $3 \cdot 2 = 6$. Für den 5er-Graphen gibt es nach dem gleichen Schema vier Möglichkeiten, Wege vom neuen Knoten zum 4er-Graphen zu beginnen. Die Anzahl der Wege insgesamt beträgt daher $4 \cdot 6 = 24$. Auf diese Weise kann man ableiten, dass in einem vollständigen Graphen mit n Knoten $(n - 1)!$ verschiedene Wege von einem gesetzten Startpunkt ausgehen.

Da für jeden der Wege $n - 1$ Streckenabschnitte eingerechnet werden müssen, bedarf es für den Brute-Force-Algorithmus ungefähr $(n - 1) \cdot (n - 1)!$ Berechnungen, um den kürzesten Weg zu finden. Die nebenstehende Tabelle enthält die Anzahlen für ein paar Problemgrößen.

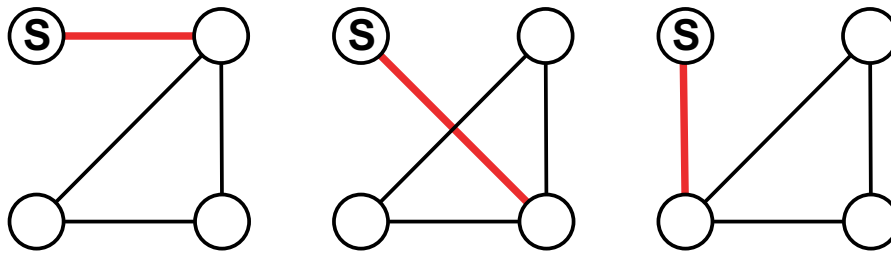


Abbildung 1.28
Der Startknoten S kann auf drei Arten mit dem vorhandenen Graphen verbunden werden.

Für 100 Knoten sind im schlimmsten Fall bereits

92.392.953.289.504.711.154.882.246.467.704.033.485.808.808.581.737.805.253.907.03
4.256.265.423.993.297.616.452.852.049.336.394.953.103.391.160.941.618.951.520.668
.673.358.807.695.360.000.000.000.000.000.000.000

Berechnungen vonnöten. Um das einzuschätzen: Ein Googol wurde bereits in der ersten Hälfte des 20. Jahrhunderts definiert als Obergrenze für Zahlen, die natürliche Vorgänge beschreiben, denn mit 10^{100} ist ein Googol größer als die geschätzte Zahl von Elementarteilchen im gesamten Universum. Für die 100 Knoten kommen wir immer noch auf

9.239.295.328.950.471.115.488.224.646.770.403.348.580.880.858.173.780.525.390
Googol

oder 9.239 Nonillionen Googol Berechnungen.

Wir können das aber noch weiter auf die Spitze treiben: Wenn auch nur alle 12.903 Gemeinden Deutschlands (Stand: Ende 2003) als Knoten in die Suche einbezogen werden, ergibt sich für die Anzahl der Berechnungen eine unglaubliche Zahl, die ich Ihnen nicht in ihrer vollen Schönheit vorenthalten möchte. Sehen Sie selbst in Abbildung 1.29.

Anders ausgedrückt, handelt es sich um etwa $9,88 \cdot 10^{47438}$.

Wenn wir diese Zahl in der normalen Schriftart hier abgedruckt hätten, würde sie 14 Seiten vollständig eng beschrieben füllen. Um sich diese Summe wenigstens einigermaßen vorstellen zu können, möchte ich sie mit einer Analogie verdeutlichen.

Wenn sich die verfügbaren Rechenleistungen wie momentan jedes Jahr fast verdoppeln, haben im Jahre 2020 alle Rechner der Erde zusammengekommen etwa die Möglichkeit, 128 Exaflops durchzuführen, also 128 Trillion Fließkommaoperationen pro Sekunde. Nehmen wir an, wir vereinigen alle Rechner zu einer gigantischen Maschine, die sich dann um unser Wegeproblem kümmert. Dann würde sie immer noch etwa $4 \cdot 10^{47412}$ Jahre für das Ergebnis benötigen.

Das Alter des Universums wird heute auf etwa 12 Milliarden Jahre geschätzt. Die zusammengezogene Macht der Rechner würde also immer noch etwa $333 \cdot 10^{47397}$ mal so lange benötigen, wie das gesamte Universum alt ist – ausgedrückt mit der Einheit Googol von vorher über eine Million Googol Google von Universumszeitaltern.

Was soll an diesem Beispiel verdeutlicht werden? Es gibt Aufgabenstellungen, die wir zwar mit schierer Rechenleistung lösen können, wenn sie sehr klein sind (so klein, dass wir meistens gar keinen Computer benötigen), aber nicht mehr, wenn sie relevante Größenordnungen annehmen.

In diesem Fall wächst der Aufwand zur Lösung exponentiell an, was bedeutet: Wenn n die Problemgröße ist, kann man den Aufwand nur noch mit x^n nach oben hin abschätzen (mit einem beliebigen x).

Optimal wäre ein linearer Aufwand. Das bedeutet, dass zur Lösung eines Problems der Größe n der Aufwand $x \cdot n$ beträgt (mit konstantem x). Leider gibt es kaum Algorithmen, die so gut sind. Betrachten wir einmal den Aufwand von Dijkstra:

Vom Startknoten aus werden alle benachbarten Knoten (also die, zu denen eine Verbindung besteht) betrachtet. Bei einem vollständigen Graphen mit n Knoten sind das $(n - 1)$. Ein Beispiel mit 7 Knoten sehen Sie in Abbildung 1.30.

Danach ist der Startknoten vollständig „aus dem Rennen“, er wird nicht mehr betrachtet. Nun wird der Knoten mit der kleinsten Markierung gesucht und von dort geht es wiederum zu allen benachbarten Knoten – das sind $(n - 2)$. Danach ist auch dieser „aus dem Rennen“.

Dies setzt sich fort. Daher ist der Aufwand für den Dijkstra-Algorithmus:

$$(n - 1) + (n - 2) + (n - 3) + \dots + (1)$$

Wir können dies wiederum in einer Tabelle wie nebenstehend dokumentieren.

Bei 12.903 Knoten ergeben sich 83.237.253 Rechenschritte. Das erledigt bereits ein normales Smartphone ohne große Schwierigkeiten in kürzester Zeit. An der Tabelle kann man ablesen, dass die Rechenzeit auch schneller als linear mit der Problemgröße anwächst, aber lange nicht so stark wie bei Brute-Force.

Sie erkennen, dass in der Informatik manchmal scheinbare Lösungen nicht praktikabel sind, weil sie bei realen Aufgabenstellungen zu viel Rechenzeit beanspruchen. Ein Informatiker muss daher fast immer die sogenannte Komplexität einer Lösung abschätzen. Die Komplexität gibt an, wie stark die Rechenzeit in Relation zur Problemgröße anwächst. Im Falle von Brute-Force ist die Komplexität n^n , während Dijkstra eine Komplexität von n^2 aufweist.

Die Informatik unterscheidet dabei verschiedene Klassen von Problemen:

Alles, was sich in polynomieller Laufzeit lösen lässt, gehört zur Klasse „P“. Das bedeutet, die Laufzeit steigt im Verhältnis zur Problemgröße in einem Verhältnis an, das

Knoten	Schritte
3	3
4	6
5	10
6	15
7	21
8	28
9	36
10	45
15	105
20	190
50	4.950
12.903	83.237.253

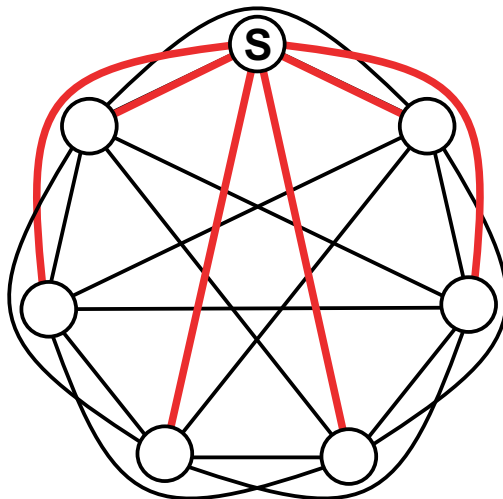


Abbildung 1.30

Im vollständigen Graphen mit 7 Knoten hat jeder Knoten (also auch der Startknoten) 6 Nachbarn.

sich durch ein Polynom ausdrücken lässt. Die Problemgröße steht also im Fall von Hoch-Rechnung immer unten.

Probleme mit polynomieller Laufzeit könnten etwa n^2 , $n^4 + n^2 + n$, $n \log n$ oder auch n^{100} als Komplexität haben.

Darüber hinaus gibt es die Klasse der „Nicht Praktisch berechenbaren“ Probleme oder „NP“. Diese haben eine Laufzeit, die sich nicht durch ein Polynom ausdrücken lässt. Dies bedeutet, dass in der Komplexitätsfunktion die Problemgröße irgendwo in einem Exponenten auftaucht.

NP-Komplexitäten sind 2^n , $n!$ oder n^n .

Allerdings haben NP-Probleme noch eine gute Seite: Man kann eine Antwort raten und dann in polynomieller Zeit überprüfen, ob diese Antwort korrekt ist.

Zu welcher Klasse von Problemen gehört das gerade besprochene Problem des kürzesten Weges? Selbstverständlich in die Klasse „P“, denn mit dem Dijkstra-Algorithmus existiert eine Lösung, die in polynomieller Laufzeit arbeitet.

Übrigens gilt dies nicht für das alte „Traveling Salesman“-Problem:

„Ein Vertreter aus Darmstadt muss Kunden in den Städten Berlin, Chemnitz, Frankfurt, Hamburg, Hannover, Marburg, München und Stuttgart besuchen. Bestimmen Sie eine Route, die von seinem Wohnort durch alle genannten Städte und zurück führt und die kürzer ist als 1000 km!“

Für dieses Problem gibt es immer noch keinen Algorithmus mit polynomieller Laufzeit. Vielleicht finden Sie ja einen ... Allerdings ist auch einzusehen, dass es sich um ein NP-Problem handelt: Rät man eine Strecke, kann man in polynomieller Zeit feststellen, ob sie der Bedingung „kürzer als 1000 km“ genügt.

Eine weitere Klasse von Problemen ist nicht in polynomieller Zeit berechenbar, und selbst wenn man eine Lösung rät, kann man die Richtigkeit nicht in polynomieller Zeit überprüfen. Diese Erkenntnisse gehen jedoch weit über die praktischen Anwendungen der Informatik hinaus, daher möchte ich hier im Buch nicht weiter darauf eingehen.

Eine der größten Fragen der Mathematik und Informatik ist immer noch kurz gesagt „P = NP?“, also ob prinzipiell alle Probleme der NP-Klasse auch in polynomieller Zeit lösbar sind (und nur noch kein entsprechender Algorithmus gefunden wurde) oder ob es Probleme gibt, die grundsätzlich nicht in polynomieller Zeit lösbar sind. Es gibt immer wieder Veröffentlichungen in dieser Richtung, aber sie haben sich bisher als informationstechnische Pendants der Kernfusion im Reagenzglas herausgestellt (also als Flop).

Wenn Ihnen jetzt P und NP noch völlig unklar sind, dann machen Sie sich keine Sorgen – sehr vielen Informatikern geht es ähnlich!

Der schnellste Weg und weitere Anwendungen

Kann man nun den Dijkstra-Algorithmus immer nur zur Berechnung kürzester Wege verwenden?

P=NP?

Die Lösung dieser Frage ist momentan der Heilige Gral der Informatik – oft gesucht, (bisher) nicht gefunden.

Genauer beschäftigen wir uns mit dieser Frage später im Buch.

Und wenn Sie Informatiker sind und „NP“ daher statt „nicht praktisch lösbar“

als „nichtdeterministisch polynomiell lösbar“ aus-schreiben, haben Sie selbst-verständlich recht! Auf allen real existierenden Computern läuft das aber auf die Frage „praktisch berechenbar“

hinaus und die Frage P = NP wäre auf jeden Fall mit einer Lösung des Problems des Handlungsreisenden in polynomieller Zeit ein für alle Male geklärt.

Ein Informatiker versucht natürlich, für einen so schönen Algorithmus weitere Anwendungen zu finden, denn es ist ja effektiver, für neue Aufgaben bereits bekannte (und bewährte) Verfahren zu adaptieren, als das Rad immer wieder neu zu erfinden.

Betätigen Sie sich doch einmal in dieser Disziplin und versuchen Sie, die folgenden Aufgabenstellungen mit dem Dijkstra-Algorithmus zu lösen.

Wie sieht es etwa aus, wenn wir für unsere erste Karte nicht den kürzesten, sondern den schnellsten Weg suchen?

Dabei gehen wir zur Berechnung davon aus, dass wir mit dem Auto auf der Autobahn im Schnitt 130 km/h schaffen, auf der Landstraße (rot und gelb) jeweils 100 km/h und auf den kleinen Gemeindestraßen (weiß) nur 50 km/h. Zusätzlich veranschlagen wir für jede Ortsdurchfahrt pauschal 8 Minuten (der Einfachheit halber auch bei den Orten, durch die wir auf der Autobahn fahren, auch hier gibt es ja oft Tempolimits aufgrund Lärmschutz).

Finden Sie den schnellsten Weg von Imstadt zu jeder anderen Stadt!



Erstellen Sie einfach eine Karte, auf der an den Straßen nicht die Strecke, sondern die Fahrzeit in Minuten steht. Dazu teilen Sie die Strecke durch die angenommene Geschwindigkeit. Für die Strecke zwischen Imstadt und Budingen ergibt sich auf diese Weise:

$$7 \text{ km} : 80 \frac{\text{km}}{\text{h}} = 7 \text{ km} : \frac{80 \text{ km}}{60 \text{ min}} = 7 \text{ km} \cdot \frac{60 \text{ min}}{80 \text{ km}} = 5,25 \text{ min}$$

Außerdem müssen für jede Ortsdurchfahrt 8 min addiert werden (also für jeden Ort an einem Ende der Strecke 4 min, da wir mangels anderer Information davon ausgehen, dass das Ein- und Ausfahren gleich lange benötigt).

Daher muss man für die besagte Strecke 13,25 min veranschlagen.

Achtung: Strecken, die an einem Ende die Nicht-Orte (X), (Y) und (Z) haben, bekommen natürlich nicht die zusätzliche Zeit für die Ortsdurchfahrt angerechnet.

Abbildung 1.31 zeigt die modifizierte Karte.

Mit dieser Karte kann dann der ganz normale Dijkstra-Algorithmus durchgeführt werden. Das Ergebnis sehen Sie in Abbildung 1.32.

Wenn Sie das Ergebnis mit der Lösung für die kürzeste Wegstrecke nach Abbildung 1.17 vergleichen, stellen Sie fest, dass es recht ähnlich, aber doch nicht identisch ist.

Das ist Informatik: Probleme lösen

Das bisherige Kapitel hat es Ihnen vielleicht schon demonstriert: Informatik macht Spaß, wenn man Aufgaben mit viel Kreativität und Intuition löst. Der wichtigste Schritt zur Lösung ist dann schon getan, wenn man sich etwas zutraut und nicht vor der Herausforderung zurückschreckt. Sie haben das gemacht, indem Sie bis hierhin durchgehalten haben! Verschiedene Untersuchungen haben gezeigt, dass in der Informatik die Tendenz zum Aufgeben für „interessierte Laien“ besonders hoch ist: In der

Abbildung 1.31
Karte mit den angenomme-
nen Wegen in Minuten

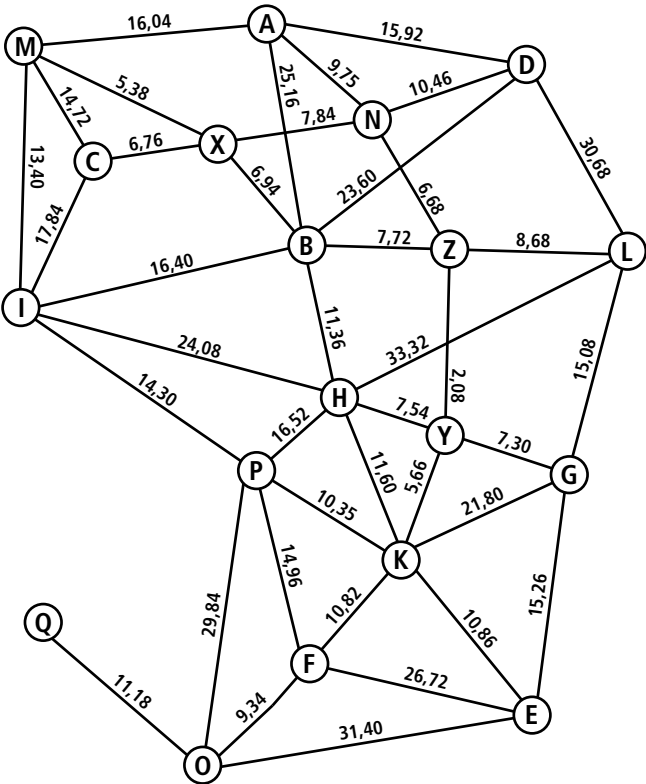
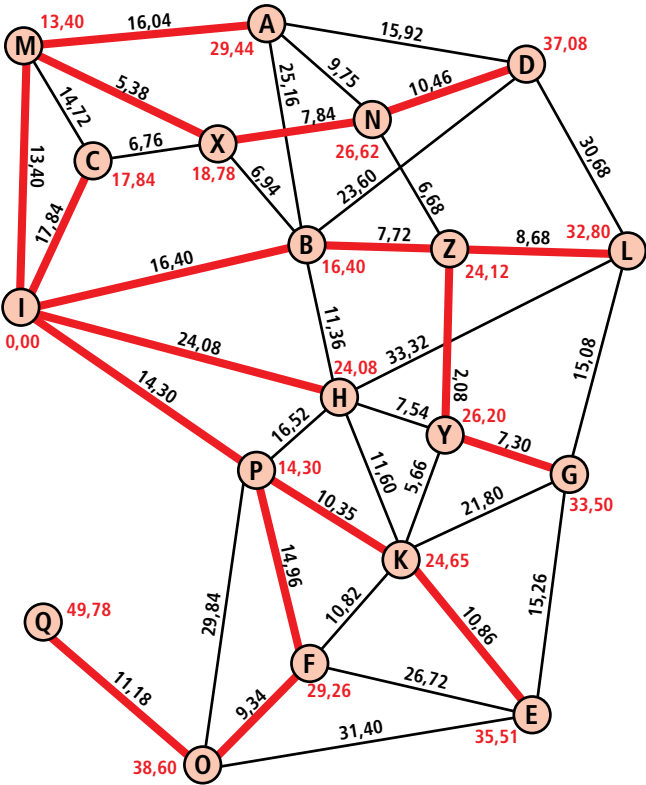


Abbildung 1.32
Die Karte mit allen zeitkürzes-
ten Wegen von Imstadt aus



öffentlichen Wahrnehmung hat sich das Klischee des typischen „Nerds“ etabliert, der zwar sozial eher unterbelichtet, aber eben dadurch hoch kompetent für die speziellen Anforderungen der Informatik ist – eine für „Normalos“ unüberwindliche Hürde ...

Dieses Buch soll auch zeigen, dass man sich besonders in der Informatik der Lösung sehr vieler Aufgaben durch konstruktive, gestaltende Herangehensweise sehr gut nähern kann. Der Unterschied zwischen „Normalen“ und „Hochleistern“ besteht häufig nur darin, dass letztere in der Lage sind, einen Ansatz zu finden und diesen konsequent zu verfolgen.

In der Informatik nutzen wir statt „Aufgabe“ gerne einen erweiterten Begriff, den ich in diesem Kapitel bereits verwendet habe, ohne ihn zu definieren: Ein Problem unterscheidet sich von einer einfachen Aufgabe dadurch, dass der Bearbeiter kein Standardverfahren zur Bewältigung kennt. Die Lösung eines Problems ist also immer mit einem schöpferischen Moment verbunden und dem produktiven Denken zuzuordnen. Manchmal fällt es gerade dabei jedoch schwer, den Anfang zu finden. Eine „Ankerlinie“, an der man sich zur Lösung der Aufgabe entlanghangeln kann, stellt daher das allgemeine Problemlöseschema sein, wie in der nebenstehenden Abbildung dargestellt.

Wenn Sie nun denken, dass das doch der oben aufgeführten Definition eines Problems widerspricht, weil es damit genau kein Standardverfahren zur Lösung geben darf, so haben Sie natürlich recht: Es handelt sich auch nicht um ein Lösungsverfahren, sondern nur um einen Vorschlag für die Herangehensweise, eine Lösung zu finden. Im Prinzip haben wir im Verlauf des ersten Kapitels exakt dieses Problemlöseschema verfolgt, um kürzeste Wege zu finden.

Sie merken vielleicht bereits: Dieser Abschluss des ersten Kapitels ist etwas trockener als der Beginn und wird im Folgenden hauptsächlich die gewonnenen Erkenntnisse nochmal auf eine andere Art darstellen, um sie zu reflektieren und dann auch zu erweitern. Da hier jedoch weniger direkt experimentiert wird: Entscheiden Sie selbst, ob Sie dies bereits jetzt lesen möchten oder diesen Abschnitt zunächst überspringen. Ich verspreche Ihnen, dass hier keine Grundlagen vorkommen, die zum Verständnis des weiteren Buches nötig wären!

Ordnen wir nun die Vorgehensweise beim Nachvollziehen des Dijkstra-Algorithmus in das Problemlöseschema: Zunächst haben wir eine Abstraktion vorgenommen. In diesem Fall konnten wir uns des Standardwerkzeugs „Graph“ bedienen, um die Komplexität der Aufgabenstellung auf das für die Problemlösung minimale Maß zu reduzieren. Der Graph ist ein entscheidendes geistiges Werkzeug, in der Informatik gibt es aber noch viele weitere Standardwerkzeuge für unterschiedliche Arten der Modellierung. In den meisten Fällen ist die Anwendung zwar nicht nach „Schema F“, aber doch sehr mechanisch intuitiv durchführbar.

Der nächste Schritt ist derjenige, der tatsächlich den „kreativen Funken“ benötigt. Effektiv haben wir Menschen normalerweise eine sehr gute Intuition zur Lösung eines Problems – zumindest die Brute-Force-Methode des Alles-Ausprobierens ist im Regelfall leicht vorstellbar. Diese lässt sich auch sehr einfach modellieren und formaler als Algorithmus fassen. Als Darstellung eignet sich dafür alles, was man selbst übersichtlich und lesbar findet. Das kann Text sein, ein Blockdiagramm wie in Abbildung 1.16 oder z. B. ein sogenanntes Struktogramm wie in Abbildung 1.33. Es ist am Beispiel recht selbsterklärend, bei Bedarf finden Sie jedoch genügend Quellen, die die genaue Bedeutung der unterschiedlichen graphischen Elemente aufzeigen.

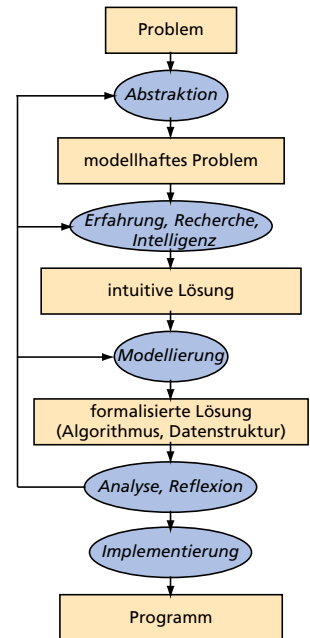
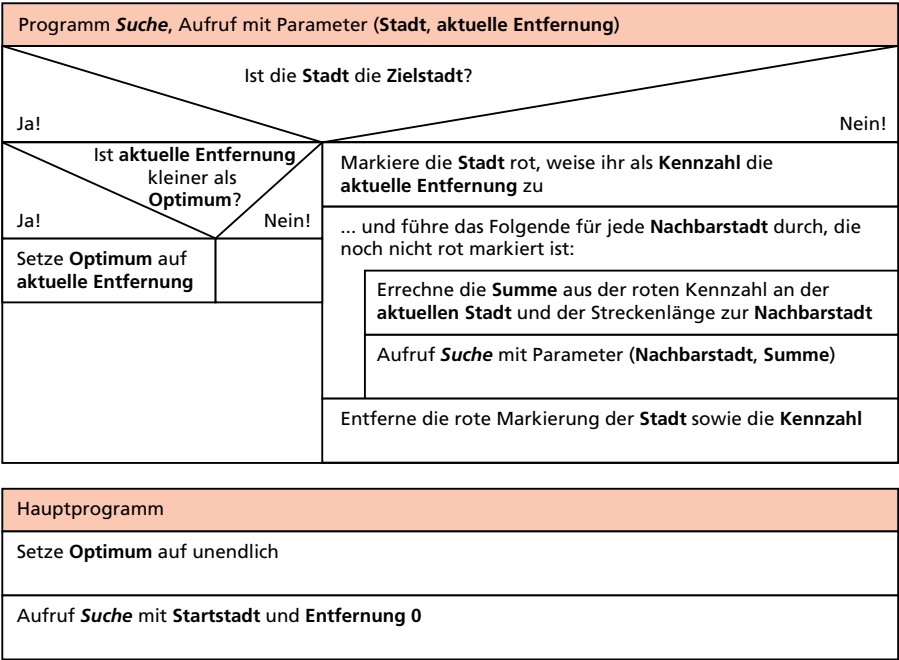


Abbildung 1.33
Struktogramm für die
Brute-Force-Methode, den
kürzesten Weg zu finden



Nächster Schritt war dann die Analyse. Der wichtigste Aspekt dabei ist die Einschätzung, ob man mit den zur Verfügung stehenden Ressourcen überhaupt eine Chance hat, die Lösung des Problems auf die vorgeschlagene Weise zu erreichen. Das läuft auf eine Komplexitätsabschätzung hinaus, die anhand des modellierten Algorithmus machbar ist, besonders unter der Perspektive einer Worst-Case-Betrachtung. Das konnten wir mit dem Graphen, in dem alle Städte direkt verbunden sind, modellieren und sind auf die astronomisch hohe Abschätzung für die Laufzeit gekommen.

Die Analyse hat uns also eine eventuell aufwendige Implementierung erspart. Glücklicherweise gibt es im allgemeinen Problemlöseschema auch Pfeile zurück, die uns ermöglichen, einen der vorhergehenden Schritte nochmals anzugehen, um eine verbesserte Lösung zu bekommen. In diesem Fall erkennt man, dass durch eine veränderte Modellierung der Lösung durch Ausprobieren sicher keine Verbesserung erzielt werden kann: Die Analyse lässt sich auf alle entsprechenden Lösungen übertragen.

Im Verlauf des Kapitels haben wir die „intuitive Lösung“ mit Hilfe der Methode der Ameisen gemeistert. Manchmal lässt allerdings bei der Problemlösung ein entsprechender Geistesblitz auf sich warten. Eine Alternative besteht daher darin, das neue Problem auf bereits (meistens von anderen) gelöste Probleme zurückzuführen. Im Zweifel führt Ausprobieren verschiedener bekannter Algorithmen oft zum Erfolg! Im Fall unseres Wegeproblems ist das die Breitensuche. Ich überlasse es Ihnen, das Thema zu recherchieren. Es wird recht schnell offensichtlich, wie man Aufgaben löst, bei denen alle Wege zwischen den Orten gleich lang sind. Als Anregung für die Übertragung auf allgemeine Landkarten schauen Sie sich bei Bedarf Abbildung 1.34 an. Hier sind alle Strecken auf die Länge 1 normiert, indem beim Graphen (mit ganzzahlig gerundeten Entfernungen) entsprechend viele Zwischenknoten eingefügt wurden. Führen Sie hierauf eine ganz normale Breitensuche aus, dann kommen Sie auf das gleiche Ergebnis wie beim Experimentieren mit den Ameisen vorne im Kapitel und (hoffentlich) auch auf Dijkstras Idee. Ein Ergebnis mit eingefärbten Knoten statt Zahlen sehen Sie in Abbildung 1.35.

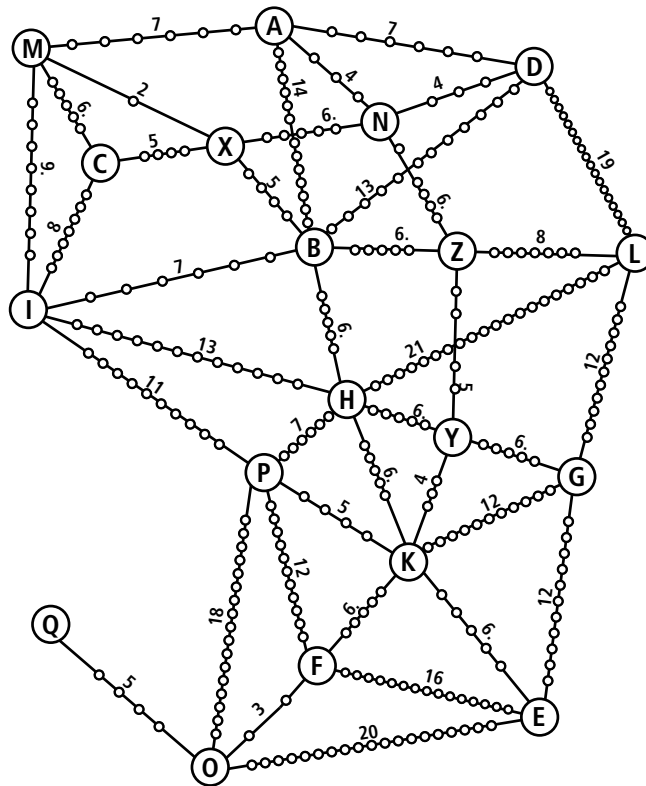


Abbildung 1.34
Karte, auf der durch Zwischenknoten alle Wegstrecken die Länge 1 besitzen. Die Gesamtstrecke könnte weggelassen werden, ist aber zum Vergleich angegeben.

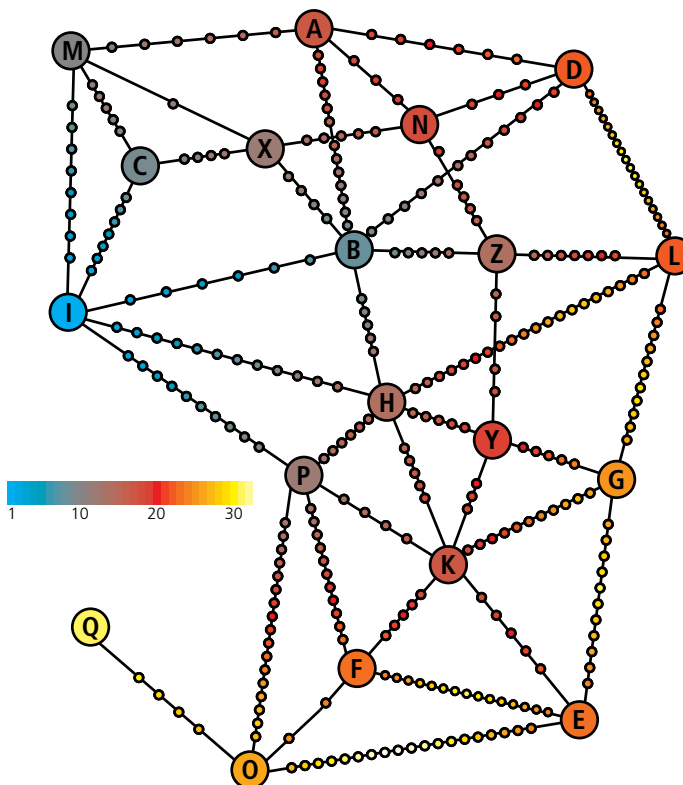


Abbildung 1.35
Abstände als Farben

Bei der anschließenden Analyse ermitteln Sie auch hier die sogenannte quadratische Laufzeit. Dass der originale Dijkstra-Algorithmus sogar noch schneller läuft – in sogenanntem $O(n \cdot \log n)$ – hängt damit zusammen, dass man noch etwas in Bezug auf die verwendeten Datenstrukturen optimieren kann. Das möchte ich hier im Buch jedoch nicht weiter vertiefen, Sie können es selbst unter dem Stichwort „Prioritätswarteschlange“ recherchieren. Zusammen werden wir das allgemeine Problemlöseschema weiter anwenden, indem wir doch noch einmal grundlegender darüber nachdenken, ob sich unsere Lösung weiter verbessern lässt.

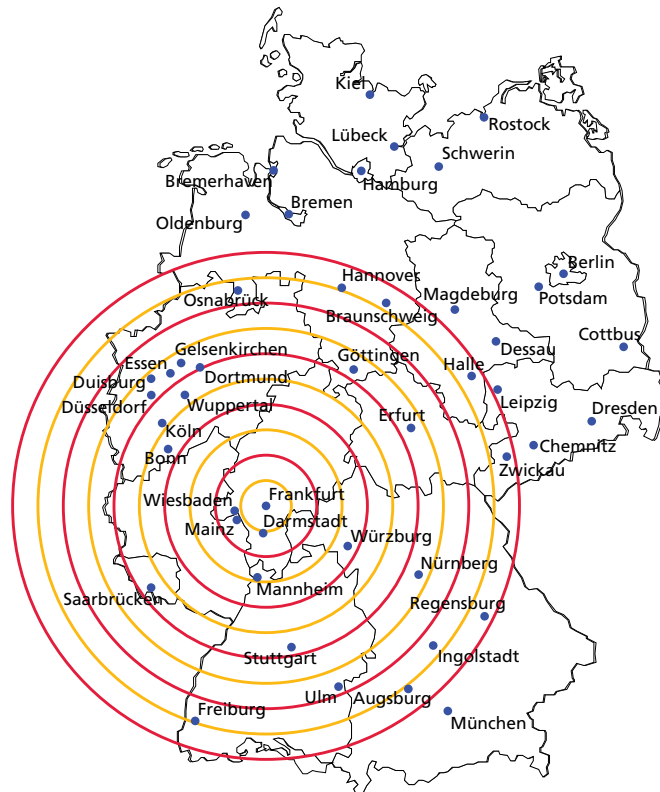
Stellen Sie sich vor, Sie nutzen den Dijkstra-Algorithmus zur Bestimmung des kürzesten Weges zwischen zwei Städten in einem Navigationssystem. Wenn Sie die Route von Frankfurt am Main nach München bestimmen, wird das Verfahren – wie man das von einer Breitensuche erwartet – etwa kreisförmig um Frankfurt herum nach dem Ziel suchen. Abbildung 1.36 zeigt das. Viel Rechenaufwand wird also hineingesteckt, um zum Beispiel den Weg über Hannover zu optimieren, obwohl Hannover ganz sicher nicht auf der Route nach München liegt!

Wenn wir im Problemlöseschema zurück gehen, finden wir eventuell eine leicht verbesserte Lösung: Um die beiden Städte wird ein Rechteck, eine „Bounding-Box“ gezeichnet und die Suche nach dem kürzesten Weg begrenzen wir auf Strecken innerhalb dieses Rechtecks. Gegebenenfalls erweitert man das Kästchen noch großzügig, um auch die Autobahnauffahrt zu finden, für die man zunächst ein kurzes Stück in die „falsche Richtung“ fahren muss. Abbildung 1.37 zeigt ein Beispiel.

An dieser Stelle können wir zunächst formal darüber nachdenken, zu welchem Schritt des Problemlöseschemas wir für diesen Vorschlag zurückgehen müssen. Ist es eine neue, kreative Problemlösung? Effektiv nutzen wir allerdings Informationen, die wir

Abbildung 1.36

Dijkstra nutzt recht uneingeschränkt die Breitensuche und erkundet den Graphen daher ohne Bevorzugung einer Richtung.



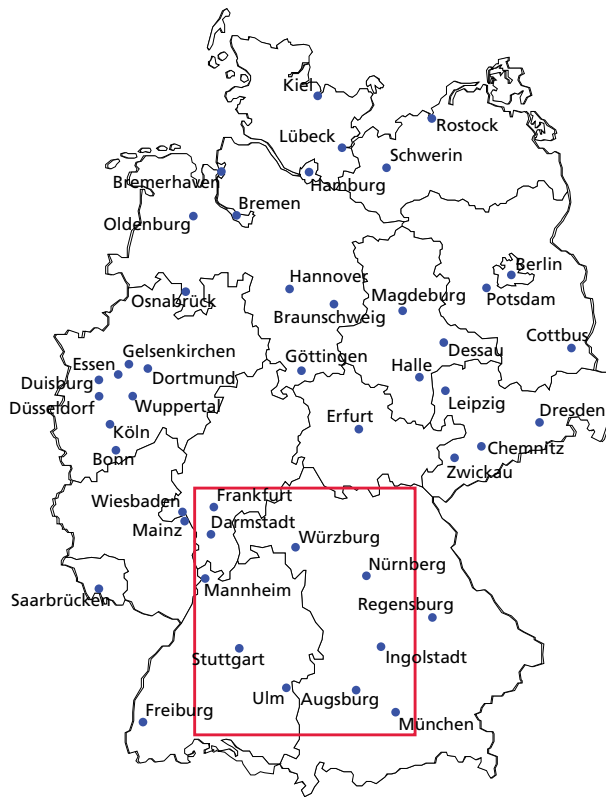


Abbildung 1.37

Ein gedachtes Rechteck um Start und Ziel grenzt die Suche ein, indem nun alle Orte außerhalb des Rechtecks nicht mehr in das Verfahren einbezogen werden.

bereits im Abstraktionsschritt zuvor weggelassen hatten: die geographischen Positionen der Orte! Diese benötigen wir natürlich, um zu bestimmen, ob ein Ort innerhalb oder außerhalb des Rechtecks liegt. Daher fangen wir hier sogar noch weiter vorne an!

Die Methode der Bounding-Box könnte dazu führen, die beste Strecke gar nicht zu finden: Woher wissen wir, dass es nicht günstiger ist, ein paar Kilometer in Gegenrichtung zu fahren, um dann eine schnurgerade Autobahn zu erreichen, auf der man den Umweg leicht wieder hereinholt? In einigen Fällen könnte die Bounding-Box sogar verhindern, dass man überhaupt eine Strecke findet. Die nebenstehende Abbildung stellt einen Teil Süditaliens dar. Möchte man von Lecce nach Catanzaro navigieren, findet man innerhalb der rot eingezeichneten Bounding-Box keinen Weg (außer man schwimmt).



Besser ist, zwar den Fokus der Suche in Richtung des Ziels zu lenken, aber trotzdem noch alle Optionen offenzuhalten. Wie so oft in der Informatik ist es auch hier sinnvoll, sich selbst bei der manuellen Suche nach einer günstigen Fahrtroute auf die Finger zu schauen. Wenn wir vom Start ausgehen und den nächsten Ort ansteuern, sind uns solche Orte am liebsten, die uns näher zum Ziel bringen. Wir könnten daher vielleicht die Länge der direkten Strecke zum Ziel (Luftlinie) als eine Art „Lustfaktor“ in den Algorithmus einbeziehen: Je kürzer die Luftlinie, desto williger steuern wir einen Ort an, werden aber die anderen Orte auch nicht ignorieren.

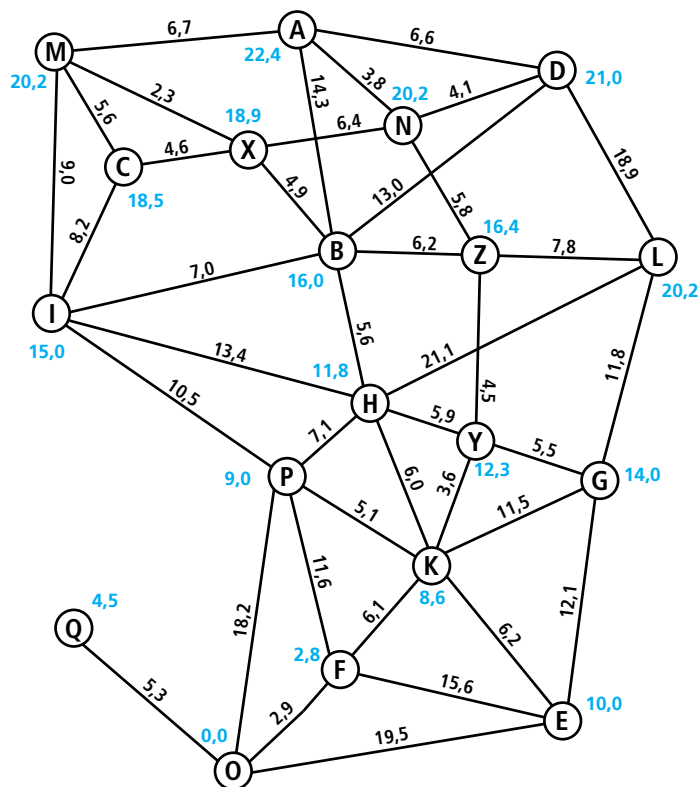
Im Problemlöseschema haben wir der neuen Anforderung nach dem Versuch mit der Bounding-Box bereits Rechnung getragen und die für die Bestimmung der Luftlinie nötigen Informationen hinzugenommen: die genaue geographische Position der Städte.

Nun lässt sich die Länge der Luftlinie zum Ziel sehr einfach aus den geographischen Koordinaten berechnen. Abbildung 1.38 zeigt den Graphen, in den mit hellblauer Schrift an jedem Knoten diese Luftlinienentfernung zum Ziel eingetragen ist. Es handelt sich erneut um unser Standardbeispiel von Imstadt nach Oppenheim. Wie kommt man nun auf eine weitere Verbesserung für einen bereits bekannten Algorithmus? Man führt ihn durch und prüft in jedem Schritt, ob sich dieser in Bezug auf eine Verbesserung in der geplanten Weise eignet.

Der erste Schritt ist die Ermittlung aller Nachbarknoten und deren Entfernungen von der Startstadt aus. Hier fällt es uns schwer, etwas einzusparen. Aber wir können auch stringenter denken: Eigentlich wollen wir ja zum Ziel, die Nachbarstädte sind daher wahrscheinlich nur Wegpunkte. Und wir wissen, welche minimale Entfernung jeder Wegpunkt zum Ziel hat. Also können wir aus der Summe der tatsächlich ermittelten Entfernung zum Wegpunkt und der minimalen Entfernung des Wegpunktes zum Ziel eine minimale Strecke zum Ziel bestimmen für den Fall, dass wir den entsprechenden Wegpunkt benutzen. Abbildung 1.39 stellt die entsprechenden Berechnungen mit violetter Summe dar.

Im Prinzip funktioniert unser neuer Algorithmus – in der Fachliteratur bekannt als A-Stern – wie „Dijkstra“. Allerdings bestimmen wir den nächsten zu besuchenden Knoten ein klein wenig anders: Gemäß der neuen Idee, betrachten wir nun nicht mehr nur die ermittelte Entfernung von der Startstadt, sondern die Summe aus dieser Entfernung und der Luftlinie zum Zielort. Diese Summe stellt quasi die minimal zurückzulegende Strecke dar, die man benötigt, um vom Start zum Ziel zu kommen, wenn man die entsprechende Stadt als Teilziel wählt.

Abbildung 1.38
Landkarte als Graph mit
zusätzlichen Knotenmarkie-
rungen für die Luftlinie zum
Ziel Oppenheim



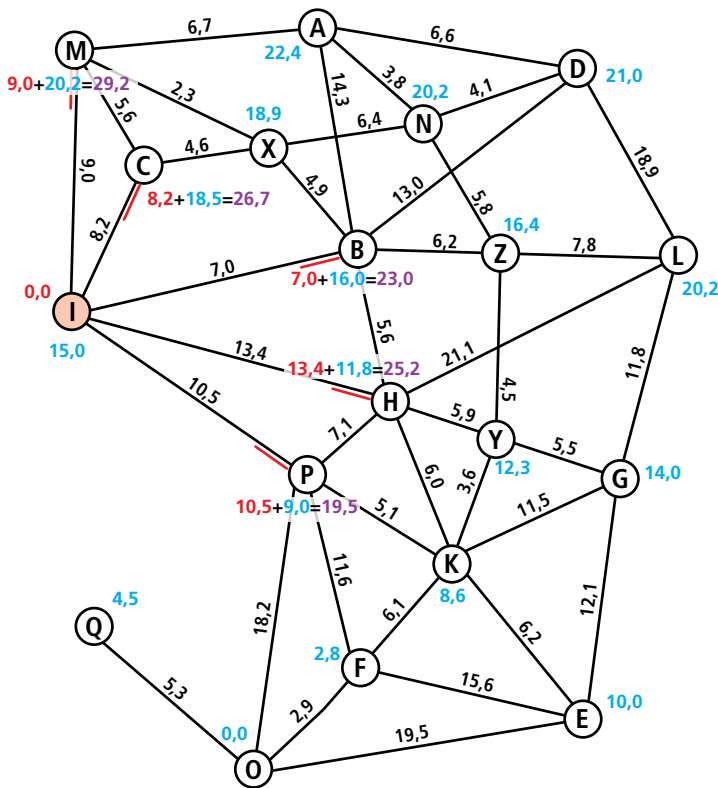


Abbildung 1.39

Neben den tatsächlich ermittelten Strecken zu den Orten vom Start aus, können wir auch die Summe (violett) aus tatsächlicher Strecke vom Start (rot) und minimaler Strecke zum Ziel (blau) ermitteln. Das ist gleichzeitig eine minimale Strecke zum Ziel bei Nutzung des Ortes als Teilziel.

Das Blockdiagramm des neuen Algorithmus nach Abbildung 1.40 ist praktisch identisch mit dem, das Sie aus Abbildung 1.16 bereits als Dijkstra-Algorithmus kennen. Den einzigen Unterschied habe ich zur Verdeutlichung rot markiert. Bei einer Implementierung muss die hier zur Verdeutlichung in den Graphen aufgenommene blaue Luftlinie genau wie die violette Summe natürlich nicht als eigene Knotenmarkierung gespeichert werden, sondern kann mit dem Satz des Pythagoras aus den Koordinaten der Orte bei Bedarf ganz einfach berechnet werden.

Hier wird auch ein ganz wesentlicher Vorteil der sehr allgemeinen, abstrakten Beschreibung eines Algorithmus deutlich, die in unserem Fall die Form des Blockdiagramms hat: Verbesserungen und Veränderungen sind so meistens nur sehr geringfügig. Es wird sofort offensichtlich, an welchen Stellen die Programmierer auch die Implementierung überarbeiten müssen.

Modellbildung

Ein informatisches Modell beschreibt genau die Teile einer Problemstellung übersichtlich, die zur Lösung des Problems nötig sind. Das gilt insbesondere auch für die Modellierung der Problemlösung als Algorithmus oder als Datenstruktur. Modellbildung findet auf vielen unterschiedlichen Ebenen statt: Selbst die Implementierung in einer gängigen Programmiersprache stellt eine Modellierung dar, da sie bestimmte Konzepte und Denkweisen impliziert.

Abbildung 1.40
A-Stern Algorithmus als
Blockcode

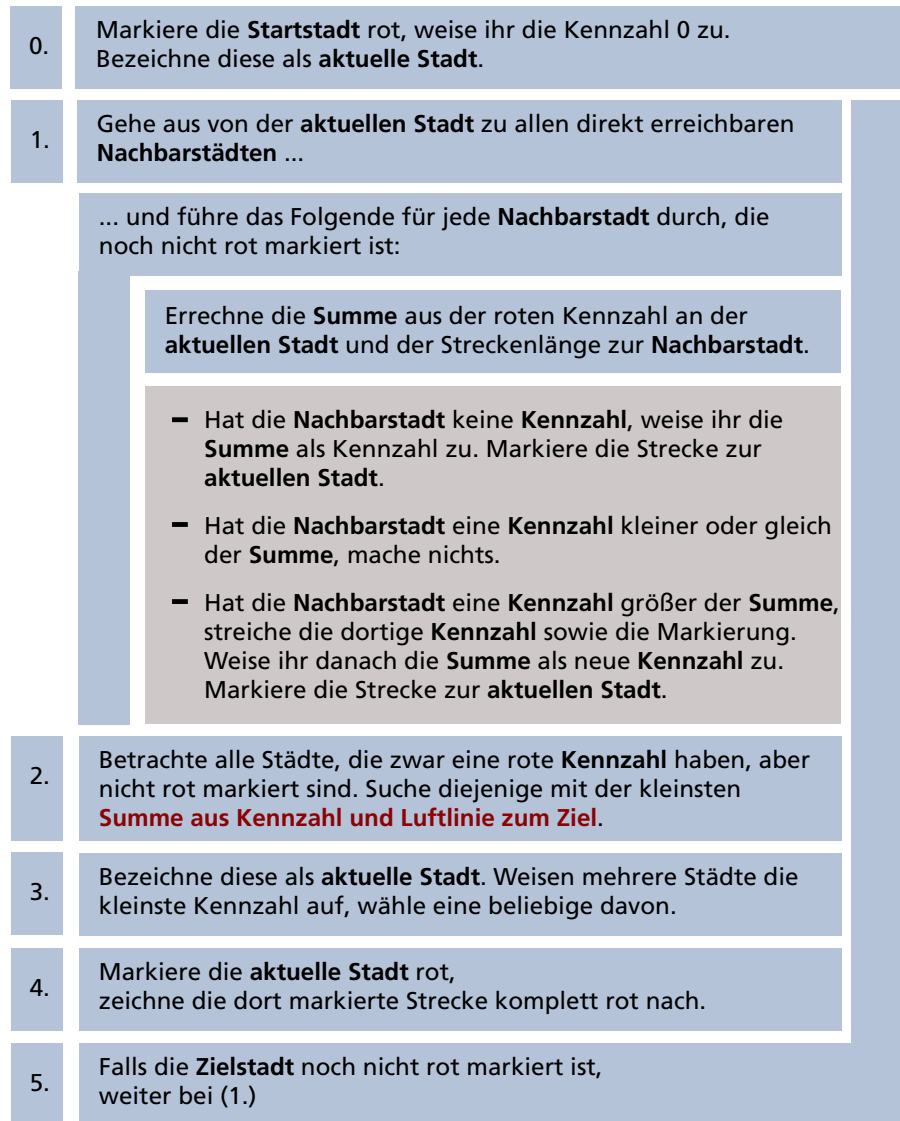


Abbildung 1.41 zeigt den nächsten Schritt des Algorithmus zum Nachvollziehen: Knoten **(P)** hat die kleinste violette Markierung und wird als Nächstes rot markiert, nun ist allerdings die tatsächliche Strecke dorthin bestimmt und sowohl blaue als auch violette Markierungen werden unerheblich. Für die Nachbarn werden die roten Kennzahlen ermittelt, daraus ergeben sich wiederum violette Summen.

Die Strategie wird nun konsequent verfolgt. Abbildung 1.42 stellt einen weiteren Schritt dar. Knoten **(B)** wird rot markiert. Beachten Sie, dass sich für Knoten **(H)** dadurch die Bewertungen verändern.

Nun möchte ich Ihnen aber gar nicht den Spaß verderben, die restlichen Schritte des Algorithmus selbst nachzuvollziehen – ganz selbstständig oder mit Hilfe der weiteren Abbildungen bis 1.46.

Abbildung 1.41
Nächster Schritt von A-Stern

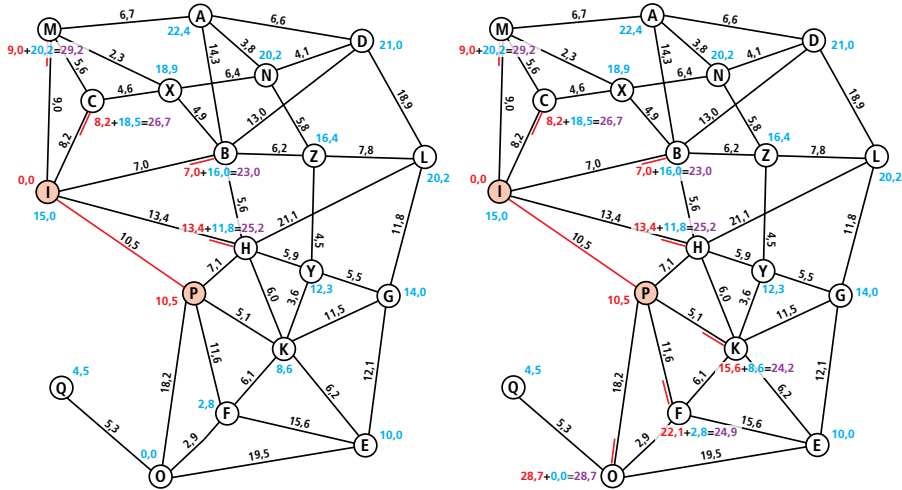


Abbildung 1.42
A-Stern

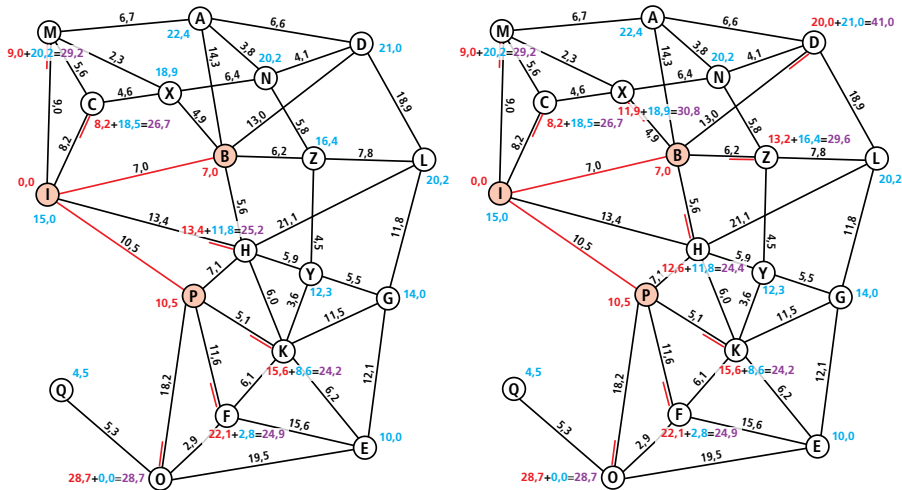


Abbildung 1.43
A-Stern

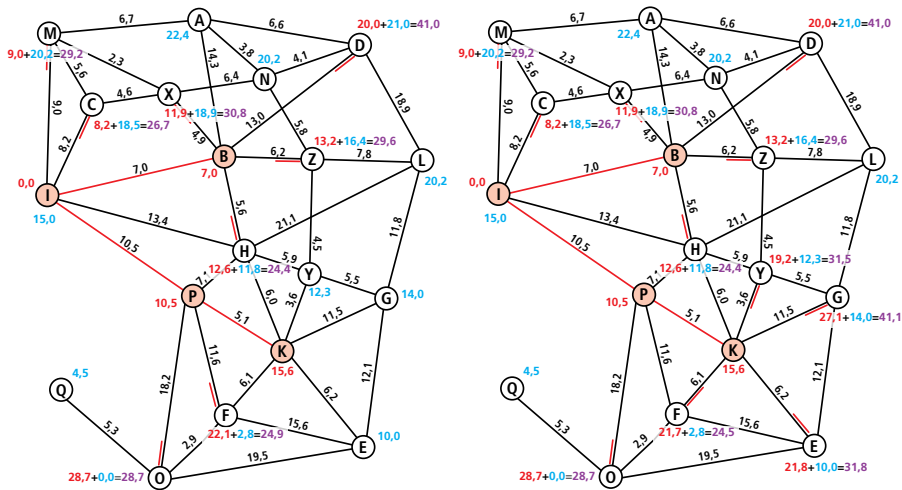


Abbildung 1.44
A-Stern

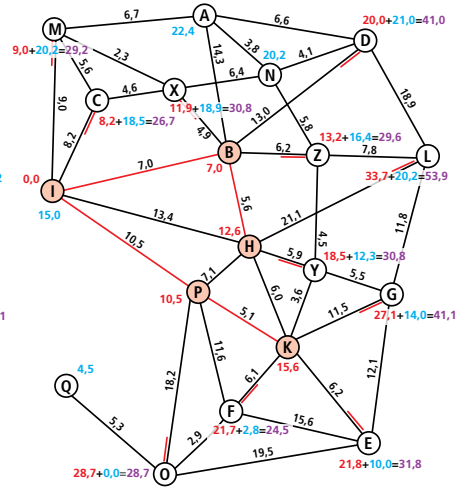
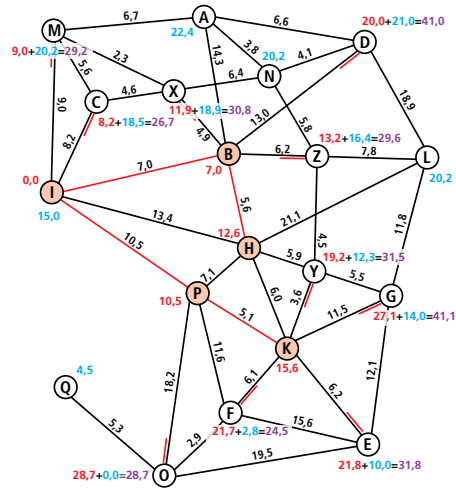
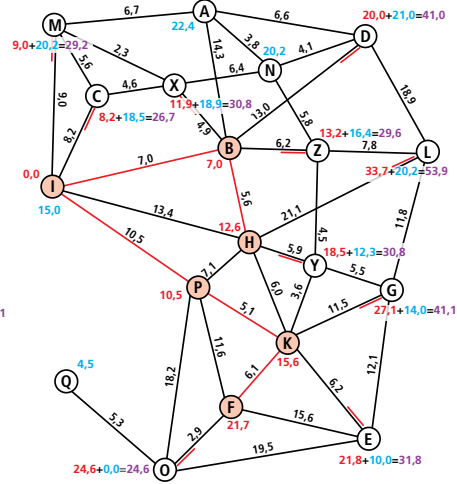
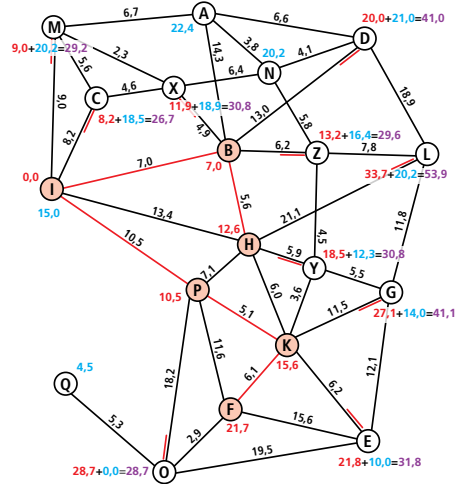


Abbildung 1.45
A-Stern



Unser Wunsch nach besserer (Ausführungs-)Geschwindigkeit ist mit A-Stern erfüllt worden: Statt in alle Richtungen gleichermaßen zu suchen, bevorzugt der neue Ansatz die Richtung zum Ziel. Im Beispiel kommt der Algorithmus auch zum gleichen Ergebnis wie Dijkstra. Die Frage ist allerdings, ob er auch immer das Optimum findet, denn unser Argument mit dem Ameisenprinzip gilt leider nicht mehr, da wir ja die Besuchsreihenfolge verändert haben. Der Analyseschritt der allgemeinen Problemlösestrategie ist also teilweise noch offen, solange die Frage nach der Optimalität nicht geklärt ist.

In Abbildung 1.47 ist der „erkundete“, also rot markierte Teilgraph zur Verdeutlichung gelb umrandet. Bemerken Sie, dass alle Nachbarknoten, also alle potentiellen Teilziele eine rote und damit auch eine violette Markierung tragen? Diese entspricht der **minimal** zurückzulegenden Strecke, wenn wir diesen Knoten als Teilziel nutzen. Und jede dieser Zahlen ist größer als die tatsächlich ermittelte Strecke zum Ziel. Wäre eine der Zahlen kleiner, hätten wir den entsprechenden Knoten gemäß unserem Algorithmus ja ebenfalls rot markiert!

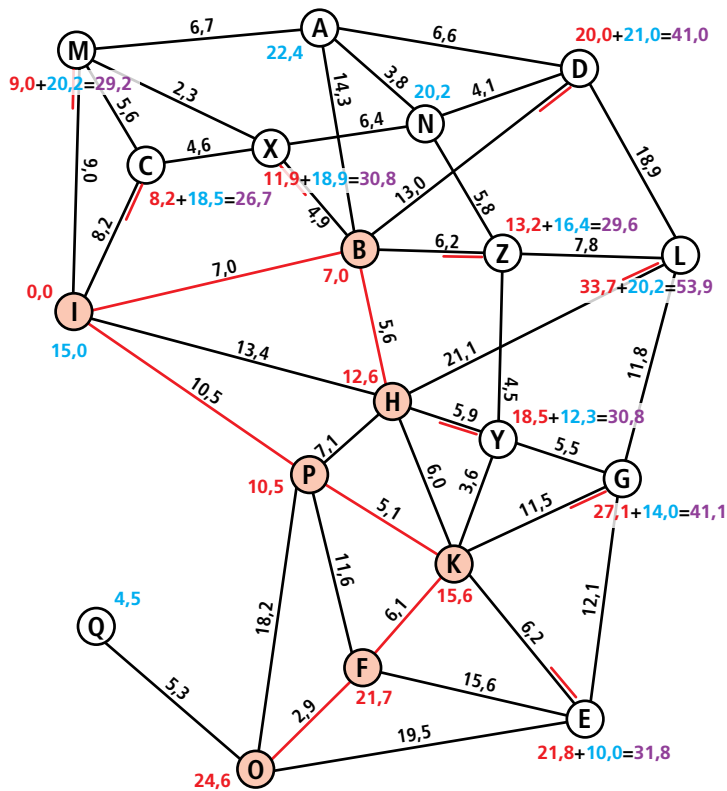


Abbildung 1.46
 Letzter Schritt des A-Stern Algorithmus: Das Ziel Openheim wurde erreicht in 24,6 km.

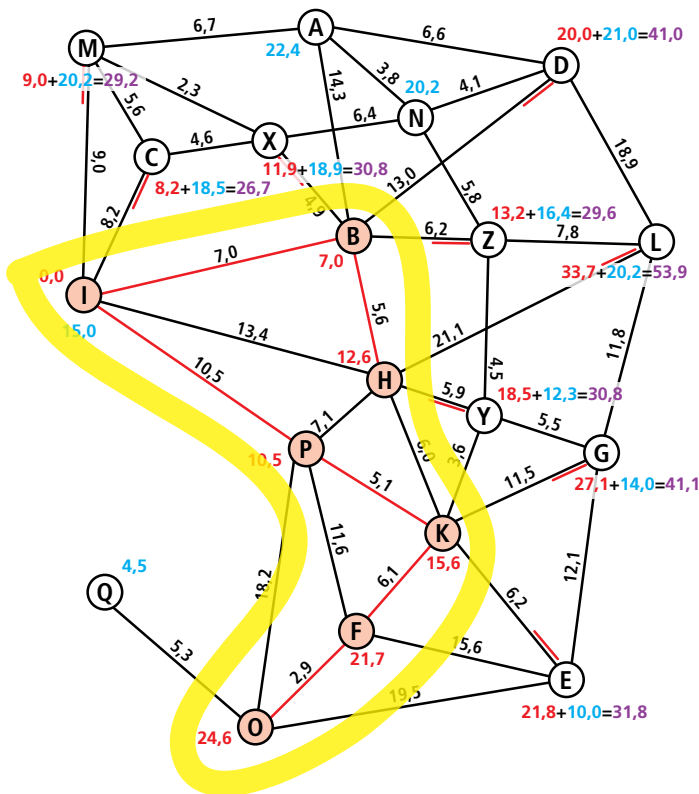


Abbildung 1.47
 Alle Knoten in der Nachbarschaft des erkundeten Pfades wurden als Teilziele bewertet und nicht berücksichtigt.

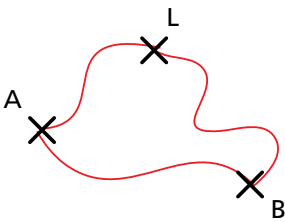
Damit ist nachgewiesen, dass alle „Abwege“ eine Verschlechterung bedeutet hätten und damit unsere gefundene Strecke – wie bei Dijkstra – die kürzeste ist.

A-Stern ist ein sehr schönes Beispiel dafür, dass wir in der Informatik oft einen bekannten Algorithmus nur leicht zu verändern brauchen, um ihn für einen bestimmten Zweck deutlich zu verbessern. Effektiv haben in diesem Fall wenige Zeichen Code ausgereicht. A-Stern wurde übrigens schon 1968 von den Peter Hart, Nils Nilsson und Bertram Raphael veröffentlicht, fand aber erst in den 1990ern erste echte Anwendungen – in Computerspielen.

Das Thema „Bestimmung des kürzesten Weges“ ist damit aber immer noch nicht am Ende: Auch die von A-Stern mit der Luftlinie als minimale Entfernung zum Ziel erreichte Steigerung der Laufzeit des Verfahrens reicht immer noch nicht aus, damit bekannte Portale effizient Millionen von Routen pro Sekunde berechnen können. Selbst wenn das Argument „dann stellen wir eben noch ein paar zusätzliche Server hin“ auf den ersten Blick einen gewissen Charme hat: Jede weitere Steigerung der Effizienz führt bei Millionen von Anfragen gleich auch zu einer spürbaren Entlastung unserer Umwelt, weil erhebliche Mengen Strom und damit CO₂ eingespart werden können.

Für Enthusiasten lässt sich daher noch nachvollziehen, wie man weiter vorgeht, um auch A-Stern weiter zu verbessern. Knackpunkt ist die erwähnte „Luftlinie“. Damit unsere Argumentation für die Optimalität von oben noch greift, muss dieser Wert auf jeden Fall kleiner oder gleich der tatsächlich zu fahrenden Strecke zwischen zwei Orten sein. Die Luftlinie hat diese Eigenschaft, ist aber oft keine wirklich gute untere Schranke. Denken Sie etwa an das Beispiel Lecce–Catanzaro aus Abbildung 1.40. Die Luftlinie läuft mitten durch das Mittelmeer, ist also sehr viel kürzer als die tatsächlich zurückzulegende Strecke drumherum. A-Stern würde in diesem Fall immer noch sehr viel Sucharbeit in die falsche Richtung investieren. Wie bekommt man aber eine bessere Abschätzung nach unten?

Andrew Goldberg, Haim Kaplan und Renato Werneck arbeiten dafür mit wenigen Leuchtturmpunkten („Landmarks“), zu denen sie die tatsächliche Entfernung vorberechnen. Während eine Tabelle mit der zurückzulegenden Strecke von jedem Punkt der Erde zu jedem anderen die Kapazität jeder Datenbank sprengen würde, ist dies von jedem Punkt der Erde zu 10 bis 20 vordefinierten „Leuchttürmen“ durchaus problemlos.



Die Idee dafür sei hier nur skizziert: Für drei beliebige Punkte bzw. Orte der Landkarte – wir nennen sie A, B und L – gilt die Dreiecksungleichung. Der zurückzulegende Weg zwischen zwei beliebigen der Punkte ist immer maximal so lang wie die Summe der beiden übrigen Wege. Der Beweis ist sehr einfach: Nehmen wir zum Beispiel an, der Weg |AB| sei länger als |LA|+|LB|. Dann könnten wir von A nach B einfach über L fahren und damit genau |LA|+|LB| zurücklegen, was die Annahme widerlegt.

Für die Skizze am Rand gelten also bezüglich der kürzesten Strecken zwischen den drei Punkten A, B und L alle Dreiecksungleichungen:

$$|AB| \leq |LA| + |LB|$$

$$|LA| \leq |LB| + |AB|$$

$$|LB| \leq |AB| + |LA|$$

Die unteren beiden kann man umformen in

$$|AB| \geq ||LA| - |LB||$$

Für die kürzeste mögliche Strecke zwischen zwei beliebigen Punkten A und B ist also die Differenz der kürzesten Strecken von A und B zu einem Leuchtturm L eine untere Schranke. Rechnet man diese untere Schranke für alle möglichen Leuchttürme aus, ist das Maximum dieser Schranken immer noch eine untere Schranke, die normalerweise aber schon recht gut ist.

Resümee

Anhand der Lösung des Wegeproblems haben wir wichtige Werkzeuge der Erkenntnisgewinnung in der Informatik kennen gelernt. Abstraktion und Modellbildung – das gezielte Weglassen irrelevanter Information und die übersichtliche Darstellung der relevanten – sind die wichtigsten Voraussetzungen, um eine Aufgabe zu lösen. Wenn man durch Intuition, Analogiebildung oder Verwendung bekannter Algorithmen einen Ansatz gefunden hat, hilft es wiederum, diesen formal zu beschreiben, um durch Analyse herauszufinden, ob er für die zur Verfügung stehenden Ressourcen wie Rechenzeit und Speicherplatz auch funktioniert.

Ein allgemeines Problemlöseschema mit den aufgeführten Schritten hilft bei der Bewältigung recht komplexer informatischer Probleme, weil es eine Ankerlinie darstellt, an der man sich entlanghangeln kann. Selbstverständlich muss man trotzdem noch sehr viel Kreativität und logisches Denkvermögen einsetzen, was die meisten Menschen glücklicherweise auch besitzen. Sie müssen sich nur bei der Lösung „beobachten“ und die Vorgehensweise formalisieren – schon ist der Algorithmus zu Papier gebracht und danach auch verhältnismäßig schnell implementiert.

Eine einmal gefundene Lösung kann dann auf viele Aufgaben angewendet werden. So kann man mit Dijkstra nicht nur den kürzesten Weg, sondern allgemein den „besten“ Weg berechnen, wobei man beliebige Qualitätskriterien annehmen kann. Dabei modellieren wir unsere Welt immer explizit und implizit. Die angenommenen Zeiten für die fahrbaren Geschwindigkeiten und die Zuschläge für die Ortsdurchfahrten beeinflussen das Ergebnis in ganz erheblicher Weise. Stimmen sie nicht, kann unser restliches Lösungsverfahren perfekt funktionieren – eine realistische Lösung wird nicht herauskommen.

Fast immer lässt sich ein Verfahren noch weiter verbessern, etwa durch eine neue Lösungsstrategie oder – wie im Fall A-Stern – durch ein verändertes Modell der Welt.

In den folgenden Kapiteln wird uns die hier erprobte „Informatiker-Denke“ immer wieder begegnen.

Abbildung 1.K1

Abstrakte Karte als Grundlage
für unsere Überlegungen zur
Lösung der Aufgabe

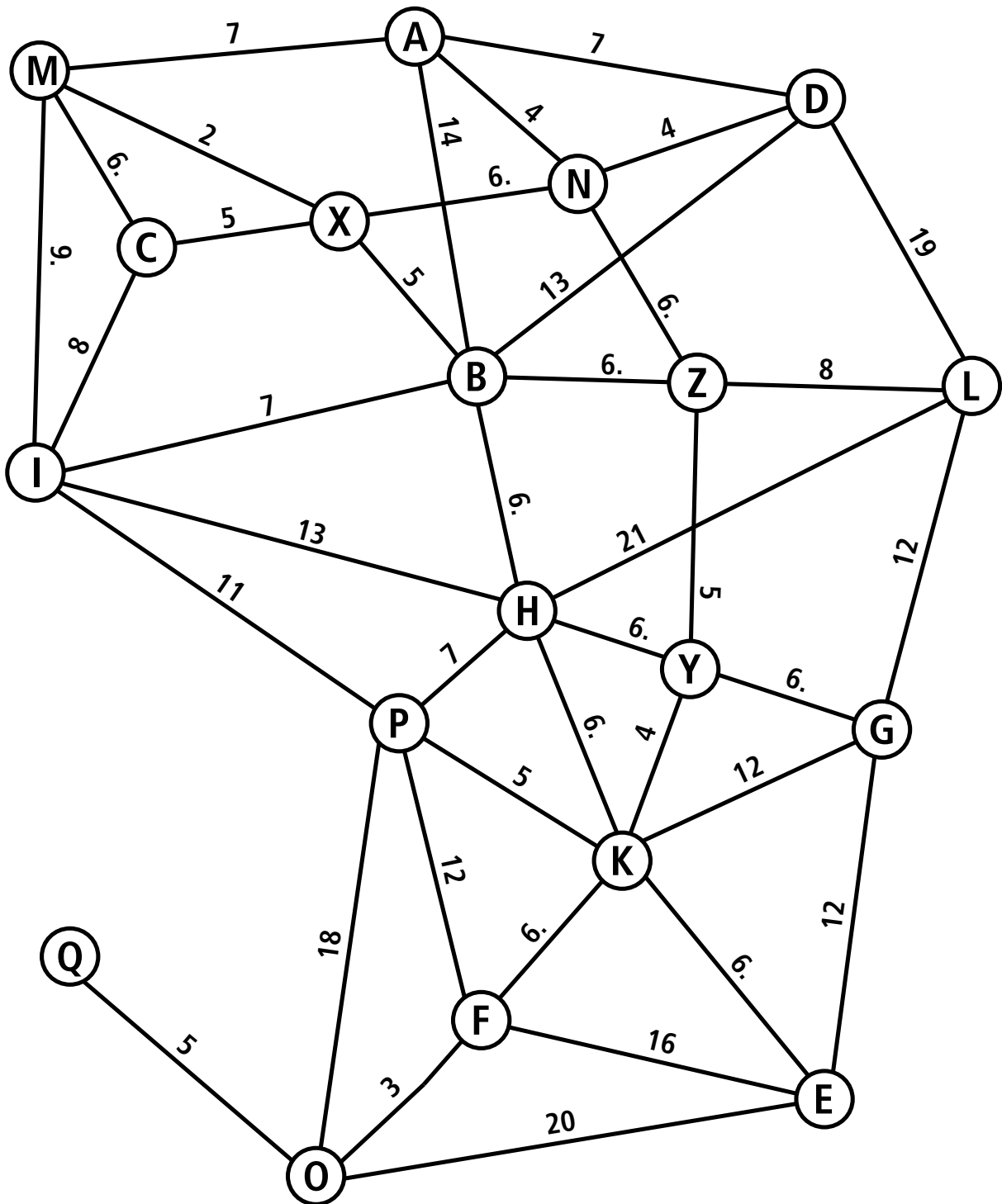
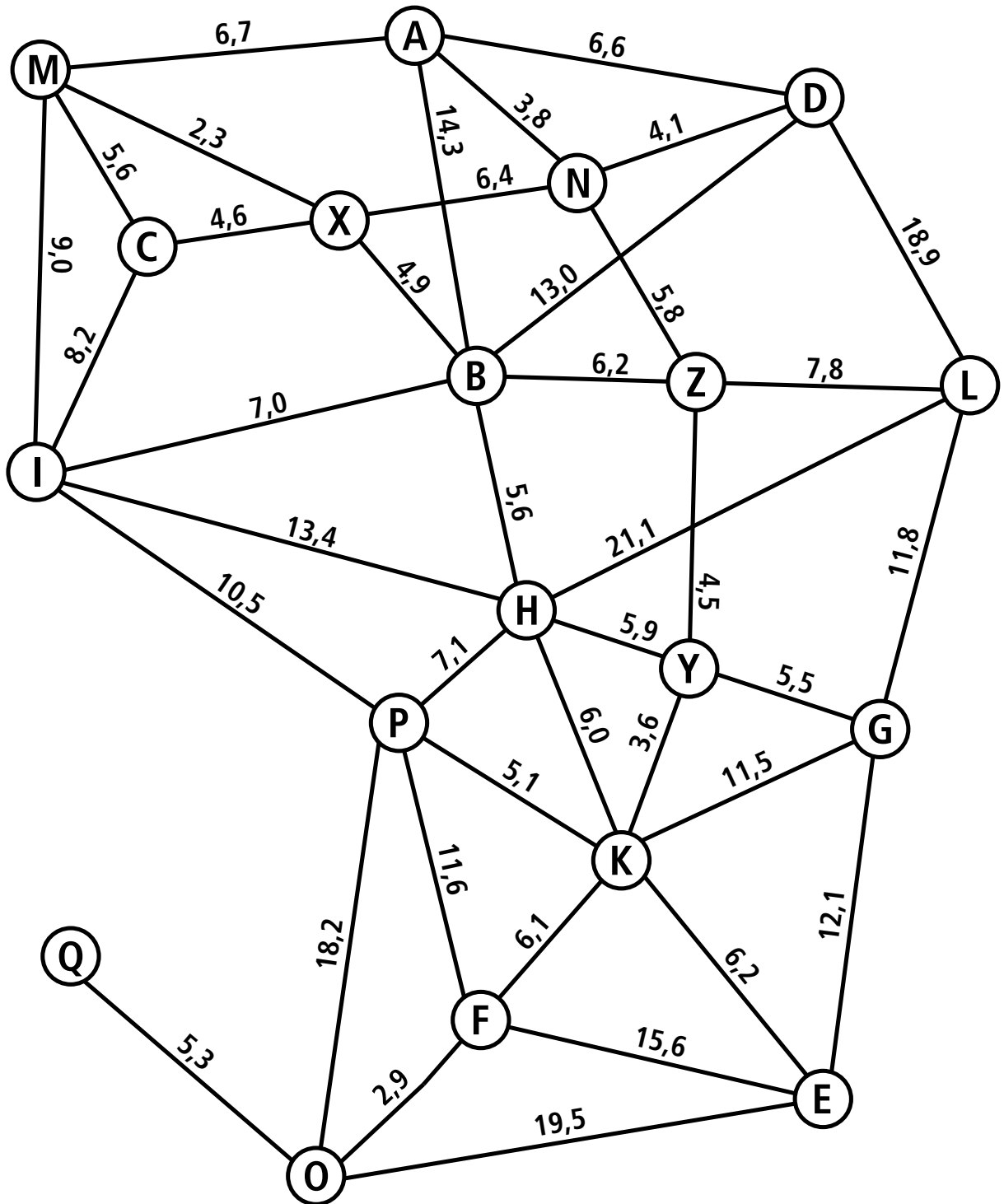


Abbildung 1.K2
Abstrakte Karte mit genaue-
ren Wegstrecken





AIRSEC-IAESTO

REDANTION

KUNSTSTOFF-102

102

ELEKTROTECHNIK SCHER-WEIZEN

STUDENTENHEIM

HRZ-TELEFONZENTRALE

HAUSHALTSSTELLE

HAUSHALTSSTELLE

PERSONALRAT

HAUSHALTSSTELLE

ZENTR. TECHN. GLENN

Lichtkabin

2. Ordnung muss sein!

Einführung

Ordnung ist das halbe Leben. Schön ist jedoch, wenn man nicht das halbe Leben damit zubringen muss, sie zu halten! Wie gut, dass es heute Computer gibt, die einen darin unterstützen.

Wie aus dem deutschen Kunstwort „Informatik“ schon ersichtlich wird, hat das Geschäft eines Informatikers sehr viel mit Information zu tun: Diese soll nicht einfach nur irgendwo abgelegt werden, sondern man möchte sie auch möglichst schnell wieder auffinden.

Das Grundproblem unterscheidet sich dabei nicht von der Frage, die auch für das Arbeitszimmer, die Küche und andere Bereiche des Lebens gilt: „Wer Ordnung hält, ist nur zu faul zum Suchen“ oder, etwas vornehmer ausgedrückt, „Der Aufwand, Ordnung zu schaffen, muss dadurch gerechtfertigt werden, dass man daraus erhöhten Komfort zieht“ – zum Beispiel mehr Platz, schnelleres Auffinden von Dingen usw.

Normalerweise versucht man daher, das Chaos erst gar nicht aufkommen zu lassen – jeden neuen Gegenstand also gleich an den richtigen Platz zu setzen. In verschiedenen Situationen funktioniert das jedoch nicht: Post kommt meistens als ungeordneter Stapel an, Spielkarten werden gemischt ausgegeben usw. Hier wird eine vollständige Sortierung notwendig. In diesem Kapitel beschäftigen wir uns damit, wie Computer die Aufgabe lösen, eine unsortierte Menge von Daten so zu organisieren, dass der Zugriff auf einen einzelnen Datensatz sehr schnell geht.

Um das wiederum experimentell nachzuvollziehen, eignen sich Spielkarten sehr gut: Jeder hat Erfahrung damit, hat selbst schon einmal die zugeteilten Karten auf der Hand sortiert, um dann Doppelkopf, Rommé oder Canasta präziser und schneller spielen zu können. In Abbildung 2.K1 am Ende des Kapitels finden Sie Kopiervorlagen für ein paar ganz spezielle Spielkarten zum Ausschneiden, die statt Kreuz, Karo, Ass und Dame normale Zahlen enthalten, denn dies sind die Größen, mit denen ein Computer meistens rechnen muss: Geburtsdaten und Sozialversicherungsnummern für Rentenauszahlung, Matrikelnummern von Studierenden für die Prüfungsverwaltung, Kontonummern und viele mehr. Schneiden Sie die Karten aus – Mischen ist erst einmal nicht notwendig!

Tipp:

Die Vorlagen für die Bastelmaterialien können Sie auch im Internet herunterladen. Dort finden Sie zusätzlich Bezugsquellen für fertige Bastelbögen auf Pappe:

www.abenteuer-informatik.de

Das Sortierproblem

Bei allen Aufgaben, die man mit dem Computer lösen möchte, ist ein guter Ansatz, sie erst einmal manuell zu erforschen, um ein Gefühl dafür zu bekommen und die Problemstellung zu erfassen. Wie wir beim Auffinden des kürzesten Weges im letzten

Kapitel gesehen haben, ist dabei oft die Problemgröße relevant. Das ist in unserem Fall die Anzahl zu sortierender Objekte.

Machen Sie daher einen ersten Versuch:

Nehmen Sie etwa die Hälfte der blau/grün/rot bedruckten Karten und sortieren Sie sie nach der blauen Zahl. Ergebnis sollte eine zusammenhängende Reihe von Karten (ohne Lücken) auf Ihrem Schreibtisch sein. Falls hier nicht genug Platz ist, dürfen selbstverständlich auch mehrere Reihen untereinander die Sortierung ergeben.

Eigentlich war das aber nur die Übung für den eigentlichen Versuch: Nehmen Sie nun 120 dieser Karten (einschließlich der gerade sortierten) und mischen Sie erneut. Bilden Sie dann sechs Stapel zu jeweils 20 Karten, die Sie bereitlegen. Sie benötigen außerdem eine Uhr mit Sekundenangabe oder eine Stoppuhr: Messen Sie die Zeit, die Sie zum Sortieren des ersten Stapels nach den blauen Zahlen benötigen, und schreiben Sie das Ergebnis auf.

Danach sammeln Sie die Karten wieder ein, nehmen einen weiteren Stapel hinzu, so dass es jetzt 40 Karten sind. Sortieren Sie nun nach den roten Zahlen. Auf diese Weise sparen Sie das Mischen zwischendurch, weil die roten Zahlen unsortiert sind, wenn die blauen sortiert sind, und umgekehrt. Notieren Sie auch hier die benötigte Zeit. Wiederholen Sie das mit 60, 80, 100 und 120 Karten.

Stellen Sie nun die Ergebnisse graphisch dar: Nehmen Sie ein Stück kariertes Papier und übertragen Sie die nebenstehende Abbildung! Machen Sie dann für jeden Ihrer Messwerte ein Kreuz über der entsprechenden Anzahl von Karten (20, 40, 60, ...) in der Höhe, die der benötigten Zeit zum Sortieren entspricht, und verbinden Sie die Kreuze mit Linien!

Vielleicht haben Sie ja auch noch willige „Opfer“ in Ihrer Umgebung, mit denen Sie den gleichen Versuch durchführen können: Lassen Sie sie zunächst ebenfalls eine Anzahl von Karten (ungefähr 60) sortieren, um sich an den Vorgang zu gewöhnen, und messen Sie dann die Zeit zum Sortieren von 20, 40, 60 usw. Karten. Übertragen Sie die Ergebnisse möglichst jeweils mit einer eigenen Farbe in Ihr Diagramm. Ein Beispiel dafür sehen Sie in der Abbildung 2.1.

Nehmen Sie sich nach jedem Versuch auch kurz Zeit, um die Strategie des Sortierers mit ein paar Stichworten festzuhalten.

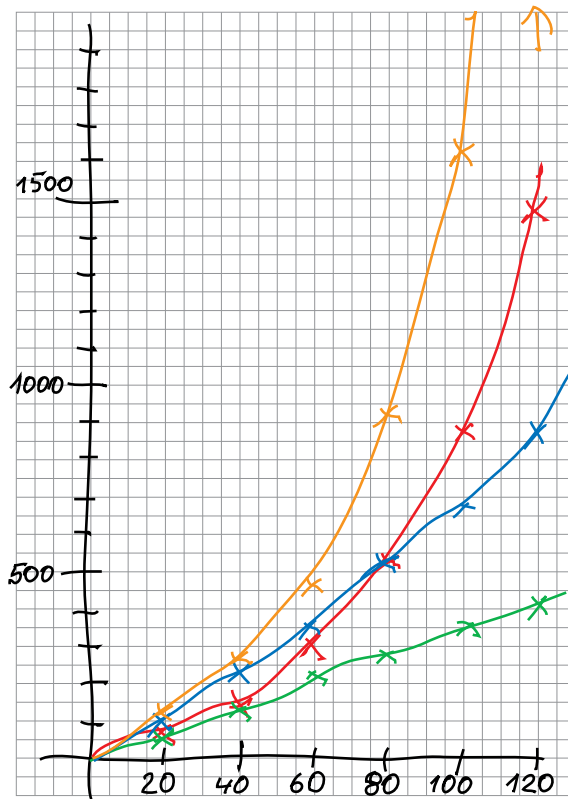
Wie können wir das Diagramm interpretieren? Die erste Erkenntnis ist, dass die Zeit zum Sortieren mit der Anzahl der Karten im Allgemeinen ansteigt. Das ist jetzt noch keine bahnbrechende Entdeckung, denn das kennen wir von allen möglichen alltäglichen Vorgängen: Das Spülen von 20 Tellern dauert auch länger als das von zehn Stück. Aber vielleicht gibt es ja noch andere Faktoren für den Zeitbedarf? Allgemein ist für die Informatik die Identifizierung einer Problemgröße sehr wichtig.

Die Problemgröße

Ein wichtiger Vorgang bei der Lösung von Aufgaben aus der Informationstechnik ist das Bestimmen der sogenannten Problemgröße. Sie stellt ein Maß dar, wie schwierig bzw. umfangreich die Aufgabe bzw. das Problem ist. Daher hängt von der Problemgröße entscheidend der zu leistende Aufwand ab.

In vielen Fällen ist die Problemgröße recht offensichtlich – wie die Anzahl von Städten einer Landkarte für die Bestimmung eines kürzesten Weges. In anderen Fällen

Abbildung 2.1
Graph Anzahl der Karten vs.
Zeit in Sekunden.



müssen wir erst einmal darüber nachdenken, und die Problemgröße hängt nicht nur von der Aufgabe selbst ab, sondern auch davon, was sich im Allgemeinen bei gleich bleibendem Aufgabentyp an der tatsächlichen Aufgabenstellung ändert:

Beim Sortieren von Objekten ist ein wesentlicher Faktor die Anzahl der Objekte! Das konnten wir in unserem kleinen Experiment feststellen. Aber auch die Größe der zu sortierenden Elemente könnte als Problemgröße dienen: Es ist sicherlich aufwendiger, 100-stellige Zahlen zu sortieren als 6-stellige ...

Daher müssen wir uns überlegen, was in einer Aufgabenstellung aus der Realität normalerweise feststeht und was variabel ist. Die Größe der zu sortierenden Elemente ist dabei meistens recht stabil: Geburtsdaten, Kontonummern, Namen usw. haben eine recht definierte Länge. Wenn wir als IT-Unternehmen ein neues Verfahren für die Sortierung von Daten anbieten, könnten wir also mit dem Slogan „Wir sortieren Kontonummern mit der doppelten Länge in der gleichen Zeit wie unsere Konkurrenten“ sicherlich weniger punkten als mit der Aussage „Wir sortieren die doppelte Anzahl von Kontonummern in der gleichen Zeit wie unsere Konkurrenten.“

Für das Sortieren nehmen wir daher im Folgenden immer die Anzahl der Objekte als Problemgröße an.

Problemgrößen und Aufwand

Von der Problemgröße hängt der Aufwand zum Erfüllen der Aufgabe ab. Wie verhält sich das denn nun in unserem Sortier-Experiment? Normalerweise geht man ja

intuitiv vom sogenannten linearen Verlauf aus: Pro Objekt wird eine bestimmte Zeit benötigt – zum Spülen von 100 Tellern benötigt man also 100-mal die Zeit, die für einen einzelnen Teller notwendig ist. Unter Umständen sogar etwas weniger, weil man ja langsam Übung bekommt und so für die letzten Teller kürzer braucht.

Schauen Sie sich daraufhin einmal Ihr Diagramm bzw. Ihre Zahlen an. Benötigten Sie zum Sortieren von 40 Karten in etwa doppelt so lange wie für 20? Haben Sie 100 Karten in der fünffachen Zeit von 20 sortiert? Im Beispiel wird das von der grünen Linie ausgedrückt. Wenn Sie das wirklich geschafft haben, sind Sie schon ein Spitzen-Sortierer und heben sich von 99 % aller anderen positiv ab.

Normal ist, wenn Sie für die Sortierung von 40 Karten knapp dreimal so lange benötigen wie für 20, nicht die doppelte Zeit. Für 60 Karten ist die fünf- bis sechsfache Zeit fällig usw. Das sieht man im Beispiel in etwa an der roten und der blauen Linie. Manche Sortierer verdoppeln jeweils sogar die benötigte Zeit, wenn sie einen weiteren Stapel Karten mit sortieren. Das trifft ungefähr für die gelbe Linie zu.

Versuchen Sie selbst, dieses Phänomen zu erklären! Was ist der entscheidende Unterschied zwischen dem Sortieren von Karten und dem Spülen von Tellern?



Divide et impera

ist weder eine Erfindung der Informatiker noch etwas, das (wie man bei lateinischen Sprüchen denkt) bereits die Römer in die Welt setzten. Der Satz wird König Ludwig XI. von Frankreich (1423 – 1483) zugeschrieben, der ihn als Maxime hatte. Es handelte sich ursprünglich um eine Militärstrategie, den Gegner zu entzweien, um ihn dann leichter zu beherrschen. Die Informatik hat ihn dann für ihre friedlichen Zwecke umfunktioniert: eine Aufgabenstellung beherrschbarer zu machen, indem man sie einteilt.

Der Unterschied lässt sich auf eine sehr grundlegende Methode der Informatik zurückführen: „divide et impera“ oder zu Deutsch „teile und herrsche“. Diese Methode ist allerdings gar nicht so neu, sondern wird von fast jedem seit Jahrhunderten für die Erledigung alltäglicher Arbeiten genutzt. Lesen Sie selbst:

Das Prinzip „divide et impera“

Sehr häufig hat man im Leben Probleme zu lösen, die zu unüberschaubar und groß sind, um sie in einem Ansatz zu lösen. Vielmehr teilen wir das Gesamtproblem auf in mehrere, handhabbare Stücke, die wir lösen. Die Teillösungen werden danach nur noch zusammengefasst.

Dieses Prinzip wird auch in der Informatik sehr stark verwendet: Ein Programm zerlegt die gestellte Aufgabe zunächst in mehrere kleinere Einheiten, genannt Teilprobleme (divide = teile), und weist danach andere Programme an, diese zu lösen (impera = herrsche, befehlige). Dabei ist sehr wichtig, dass die Teilprobleme unabhängig voneinander gelöst werden können, denn sonst müssten die Programme miteinander kommunizieren, unter Umständen auf Lösungen voneinander warten, was den Aufwand wiederum sehr erhöht.

Unterschiedliche Probleme können unterschiedlich gut aufgeteilt werden. Das Tellerwasch-Problem stellt hier keine Herausforderung dar: „100 Teller waschen“ hat direkt als Teilproblem „1 Teller waschen“. Dieses wird 100-mal ausgeführt. Jeder Teller kann unabhängig voneinander gewaschen werden, der Gesamtaufwand besteht also in der Summe der einzelnen Aufwände, also 100-mal der Zeit, die zum Waschen eines Tellers benötigt wird. Man kann die Zeit sogar durch Parallelisierung optimieren: Stellen wir 100 Tellerwäscher ein, schaffen wir das Problem in der Zeit, die man zum Waschen eines einzelnen Tellers benötigt ...

Wie sieht das aber beim Sortieren von Karten aus? Probieren wir, auch dieses Problem auf die gleiche Weise aufzuteilen. Wir gehen davon aus, pro Arbeitsschritt eine (un-

sortierte) Karte auf die Hand zu nehmen, die wir dann in die Folge bereits sortierter Karten einordnen.

„100 Karten sortieren“ hat direkt als Unterproblem „eine Karte sortieren“. Wie lange benötigen wir aber, um eine Karte korrekt zu sortieren? Das hängt sehr stark von der Anzahl Karten ab, die bereits auf dem Tisch liegen!

Die erste Karte geht sehr schnell, wir können sie einfach hinlegen. Bei der zweiten Karte müssen wir bereits die Entscheidung treffen, ob sie links oder rechts angelegt wird – der Aufwand hat sich vergrößert.

Befinden sich bereits 99 Karten auf dem Tisch, müssen wir die neue Karte mit sehr vielen der bereits sortierten Karten vergleichen, um die richtige Position herauszufinden.

Hier liegt der große Unterschied: Der Aufwand zum Lösen eines Teilproblems ist nicht mehr unabhängig von der Problemgröße, wie das beim Tellerwaschen der Fall war! Je mehr Karten wir sortieren, desto länger dauert im Mittel das Sortieren einer Karte. Daher steigt der Aufwand mit jeder Karte, die zu unserer Sortierung hinzukommt, einerseits um den Faktor „die Anzahl der Karten ist höher, daher muss die Anzahl der Sortiervorgänge erhöht werden“, aber zusätzlich auch um den Faktor „jeder Sortiervorgang dauert im Mittel etwas länger“.

Eine der wesentlichen Aufgaben eines Informatikers ist, diese Erhöhung des Zeitaufwands zu erforschen und dieses Wissen zu nutzen, um die verwendeten Programme zu verbessern, so dass auch hohe Problemgrößen noch bewältigt werden können.

Sortieren auf Computerisch ...

Egal, welche verschiedenen Strategien Sie und Ihre Versuchspersonen bisher angewandt haben – diese sind sicherlich in der einen oder anderen Weise auch in der Informatik bekannt, es existieren also Sortieralgorithmen dafür.

Allerdings fällt es schwer, diese zu erkennen, wenn man jemandem beim Sortieren von Karten zuschaut. Ein Mensch hat einfach zu viele Freiheitsgrade, muss beim Agieren keine strikten Regeln einhalten. Hier hilft oft, einfach Computer zu spielen:

Überlegen Sie, welchen Beschränkungen ein Computer unterworfen ist, wenn er in seinem Speicher Zahlen sortieren soll. Wie könnte man diese Beschränkungen in eine Art „Spiel“ umsetzen, so dass man mit ähnlichen Voraussetzungen die Karten sortiert? Wenn Sie ungefähr wissen, wie der Speicher eines Computers funktioniert, versuchen Sie zunächst, die Frage selbst zu beantworten, ansonsten lesen Sie bitte einfach weiter.



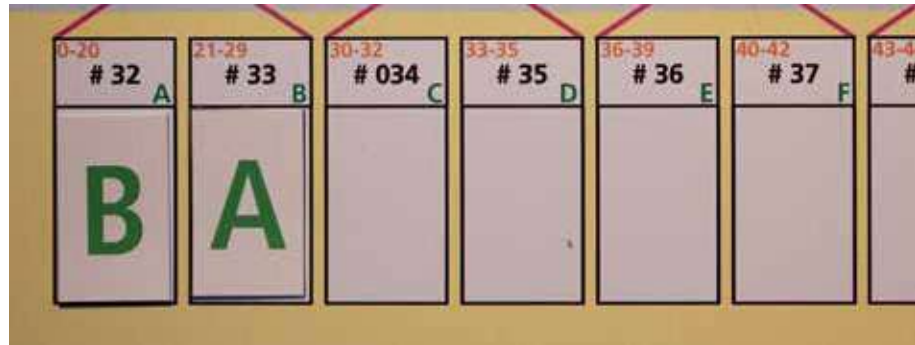
Der größte Unterschied zwischen Mensch und Maschine besteht beim Sortieren in der Speicherung: Der menschliche Kartensortierer besitzt quasi überall „Speicher“ – auf der Hand, als Stapel auf dem Tisch, ausgelegt auf dem Tisch usw. Beim Computer ist der Speicher streng in Positionen organisiert und jede Position, genannt Speicherstelle oder Speicheradresse, kann genau einen Wert bzw. in unserem Spiel eine Karte beinhalten.

Das können wir glücklicherweise als Experiment simulieren. Die Doppelseite mit Abbildung 2.K2 ist Ihr eigener Computerspeicher für die Karten aus der Kopiervorlage. Zunächst kommen wir mit dem gelben Bereich mit den Speicherstellen #32 bis #63 aus. Wenn Sie wollen, können Sie aber den Bogen mehrfach kopieren, um einen noch viel längeren Bereich zu erhalten! Die grünen Buchstaben auf den ersten Speicherstellen sollen genauso wie die roten Zahlenbereiche erst einmal unbeachtet bleiben!

Bringen Sie nun ein paar Karten in den neuen Speicher. Für Sie gilt selbstverständlich die Spielregel: Karten dürfen nur genau auf Speicherpositionen liegen, nicht dazwischen. Legen Sie zwei beliebige Karten auf die Adressen #32 und #33.

Den Inhalt der Speicheradressen können Sie auslesen – einfach darauf schauen. Eine Besonderheit gibt es allerdings beim Schreiben! Was passiert, wenn Sie einen zusätzlichen Wert auf Position #33 schreiben (also eine Karte auf diese legen)? Bei unserem

Abbildung 2.2
Papierspeicher mit unsortierten Karten



Papierspeicher passiert nicht viel: Eine Speicheradresse kann prinzipiell als Stapel beliebig viele Karten fassen. Nimmt man die oberen Karten wieder weg, kommen die unteren wieder zum Vorschein.

Ein Computer funktioniert anders. Jede Adresse kann genau einen Wert speichern. Gibt man einen neuen Wert hinein, überschreibt man damit den alten Inhalt. Wir müssen also noch als Spielregel festlegen, dass auf jeder Speicheradresse nur eine Karte liegen darf: Sobald eine neue Karte auf einer Position abgelegt wird, kommt die eventuell bereits dort liegende Karte auf den Ablagestapel und ist aus dem Spiel.

Probieren Sie das versuchsweise anhand eines sehr einfachen Experiments: Die Speicheradressen #32 und #33 sind momentan jeweils mit einer Karte belegt. Tauschen Sie den Inhalt dieser beiden Positionen aus und halten Sie sich dabei genau an die Spielregeln.



Haben Sie das Problem erkannt? Normalerweise würden wir für den Austausch einfach eine der Karten in die Hand nehmen, die andere verschieben und dann die Karte aus der Hand auf den verbliebenen Platz legen. Dadurch hätten wir aber die Hand als „Speicher“ genutzt – das ist gegen die Spielregel, nach der Karten nur auf Speicherpositionen des Papierspeichers liegen dürfen.

Nehmen wir dagegen eine der Karten und legen sie direkt auf die andere Position, wird diese überschrieben, die Karte mit dem anderen Wert kommt aus dem Spiel – sie ist verloren und der Austausch nicht mehr möglich.

Folgerung daraus ist, dass wir den zusätzlichen Speicherplatz („die Hand“) wirklich benötigen: Legen Sie die Karte von Speicherstelle #33 zunächst auf Position #34 ab. Danach können Sie die Karte von #32 auf #33 verschieben und jetzt erst die Karte von #34 auf #32 legen. Das entspricht unseren Spielregeln und exakt so funktioniert die Sache auch im Computer.

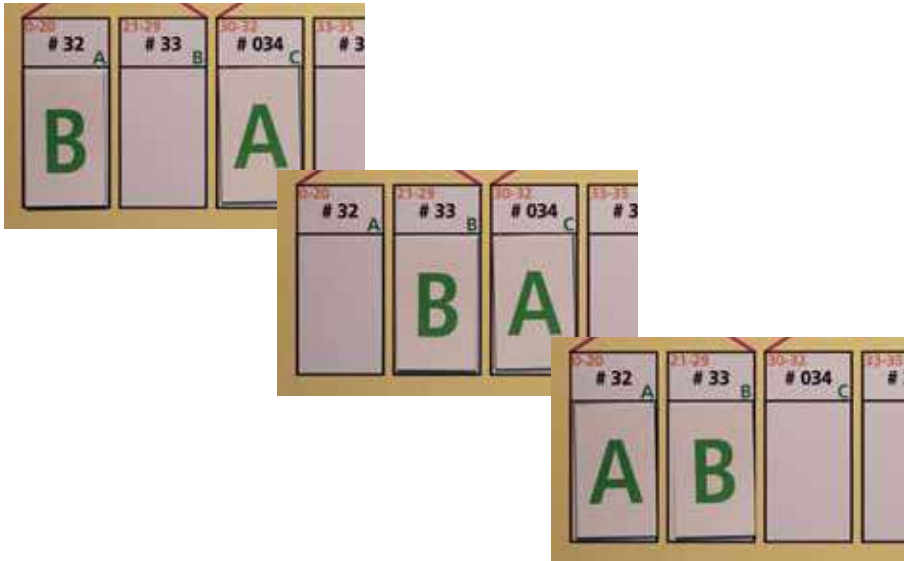


Abbildung 2.3

Kartentausch nur mit Hilfe einer zusätzlichen Speicherstelle

Für viele Vorgänge wird ein Zwischenspeicher benötigt. Dafür besitzen die meisten Computer auch spezielle Vorkehrungen – Speicheradressen, die besonders schnell und komfortabel abgefragt werden können. Wie setzen wir das im Papierspeicher um? Wo haben wir im echten Leben einen komfortablen Zwischenspeicher für Karten? Wie wäre es mit folgender Erweiterung der Spielregel: **Sie dürfen nun maximal eine Karte in der Hand behalten, während Sie die anderen Karten im Speicher verschieben.**

Zeit für unsere nächsten Schritte als Computer-Simulator. Wählen Sie zufällig sechs der grünen Spielkarten (mit einfachen Buchstaben) aus und legen Sie diese unsortiert auf die Adressen #32 bis #37. Abbildung 2.4 zeigt ein Beispiel.

Nun wollen wir die Folge alphabetisch sortieren. Eine Möglichkeit dafür ist, von vorne Ordnung zu schaffen. **A** muss also ganz nach vorne. Nehmen Sie daher **A** in die Hand, Ihren legalen Zwischenspeicher, und schieben Sie nacheinander **T** auf die Position #36, **S** auf #35, **E** auf #34 und **L** auf #33. Nun ist #32 leer, wir können **A** aus dem Zwischenspeicher dort ablegen. Das Verfahren können wir für **D** wiederholen, also **D** aufnehmen, **LEST** von hinten nach vorne je um eine Position verschieben und dann **D** auf der freien Position #33 ablegen. Abbildung 2.5 zeigt, wie die Folge jetzt sein sollte.



Abbildung 2.4

Unsortierte Karten im Speicher

Abbildung 2.5
Nach den ersten Sortierschritten



Abbildung 2.6
Die sortierte Folge

Nun müssen nur noch **L** und **E** getauscht werden und die Folge ist perfekt sortiert, wie Abbildung 2.6 zeigt.

War das eine korrekte Vorgehensweise, wenn man einen Computer simulieren möchte? Betrachten Sie für die Antwort erst einmal nur den allerersten Schritt: **A** muss ganz nach vorne.

Zwei Erkenntnisse haben wir hier genutzt, die wir als Menschen sofort haben, die der Computer aber nicht ohne Weiteres besitzen kann:

1. **A** ist im Speicher die Karte mit dem alphabetisch kleinsten Buchstaben.
2. Die Karte mit dem kleinsten Buchstaben liegt auf Position **#36**.

Woher wissen wir das? Weil wir natürlich alle Karten gleichzeitig sehen und bereits unser Unterbewusstsein spontan die Situation für uns analysiert.

Ein Computer kann zwar auf seinen gesamten Speicher zugreifen, aber im Normalfall immer nur gleichzeitig eine Adresse anschauen. Es ist für ihn daher im Normalfall ein aufwendiger Prozess, herauszufinden, welche der vorhandenen Karten denn nun ganz nach vorne muss.

Wir fügen das auch noch unseren Spielregeln hinzu. Dazu benötigen wir die Pfeile aus den Bastelbögen. Wenn wir eine Speicherstelle auslesen wollen, muss immer ein Pfeil auf die entsprechende Adresse zeigen. Alle anderen Adressen gelten als unsichtbar.

Ich verspreche: Jetzt haben wir wirklich alle Regeln festgelegt, um Computer zu spielen, und dürfen mit der Umsetzung des ersten Sortierverfahrens beginnen.

Alles, was Sie hier erforschen, einschließlich der offenbar ungünstigeren Vorgehensweisen, ist übrigens tatsächlich in heutigen Computersystemen wiederzufinden. Besonders in Bereichen, in denen an korrekten Programmen viel Geld hängt, folgt man gerne dem Informatik-Motto „never change a running system“, also „Verändere nie ein System, wenn es (doch) läuft.“ Daher finden sich in den Programmen oft noch Prinzipien, die an anderen Stellen bereits durch neue Ideen ersetzt wurden.

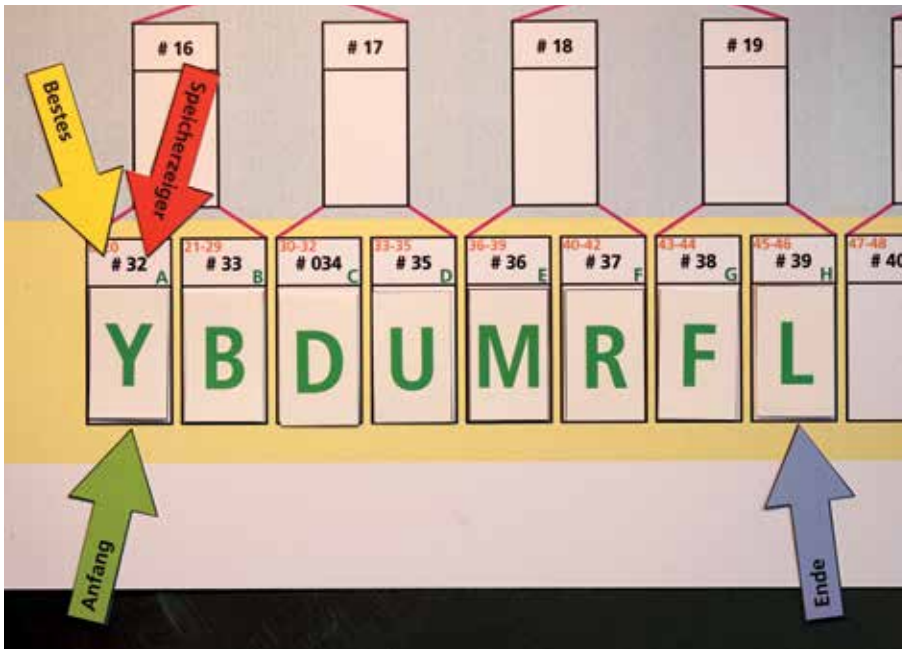


Abbildung 2.7
Sortierfolge mit Zeigern

Selection-Sort

Aufgabe ist, die Folge von sechs Buchstabenkarten im Speicher von Anfang bis zum Ende zu sortieren. Abbildung 2.7 zeigt die Aufstellung mit den zur Verfügung stehenden Zeigern „Speicherzeiger“ sowie „Anfang“, „Ende“ und „Bestes“.

Wie bereits oben dargestellt, ist ein immer wiederkehrendes Prinzip der Informatik die Unterteilung eines großen Problems in kleinere, handhabbare Portionen. Eine Unterteilung des Sortierproblems könnte also lauten:

Sortieren von n Karten:

- Sortiere die Karte mit dem kleinsten Buchstaben an den Anfang
- Sortiere die restlichen $n - 1$ Karten

Diese Vorgehensweise nennt man auch „Sortieren durch Auswahl“ oder mit englischem Fachbegriff „Selection-Sort“, weil immer eine Karte ausgewählt und korrekt einsortiert wird.

Versuchen Sie nun, diese Vorschrift zu konkretisieren. Spielen Sie mit den Materialien und formulieren Sie einen allgemeinen Algorithmus für Selection-Sort. Die Zeiger helfen Ihnen dabei, Positionen zu markieren. Sie dürfen sie im Algorithmus benutzen.

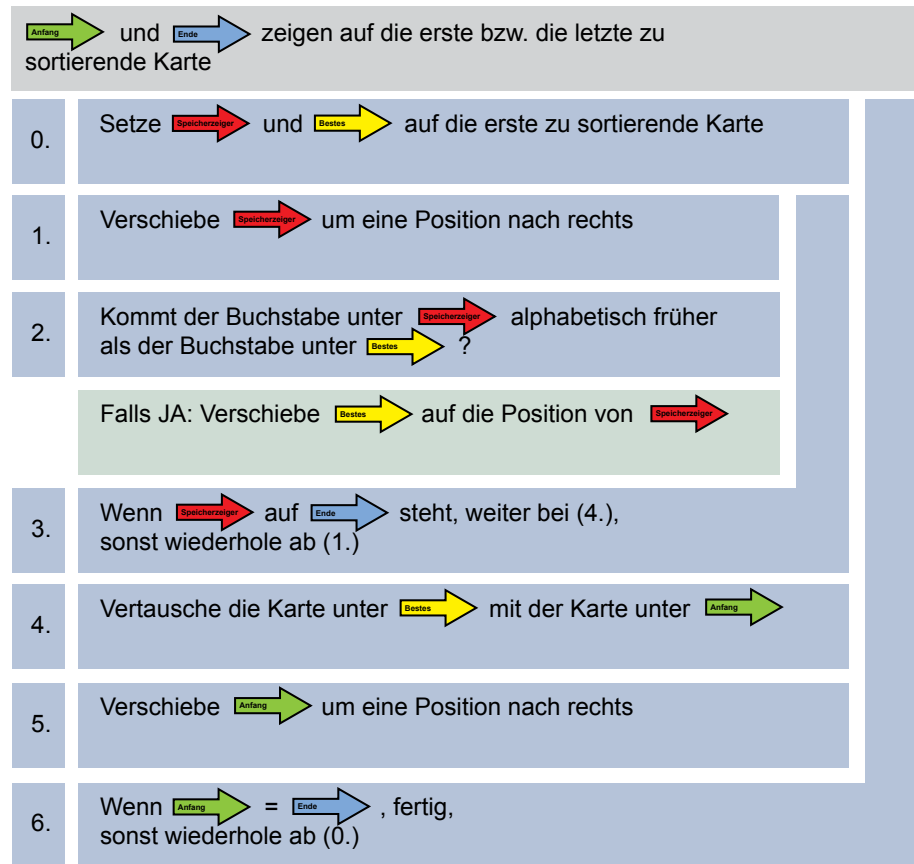
Sie können eine aus dem ersten Kapitel bekannte Schreibweise verwenden oder einfach die Vorschrift so aufschreiben, wie es für Sie selbst am besten nachvollziehbar ist.



Ein Vorschlag ist die ausformulierte Vorschrift nach Abbildung 2.8.

Abbildung 2.8

Die hier verwendete Notation für Algorithmen ist eine vereinfachte Form der von Isaac Nassi und Ben Schneiderman entwickelten modularen Darstellungsart. Im deutschen Sprachraum ist sie unter den Bezeichnungen „Nassi-Shneiderman-Diagramm“ oder „Struktogramm“ bekannt.



In den Anweisungen (1.) bis (3.) wird der kleinste Buchstabe gesucht und gefunden. Dieser muss ganz an den Anfang und wird daher getauscht mit der Karte, die den Platz dort belegt (4.).

Nun weiß man, dass die Karte an der ersten Position korrekt liegt, dass diese also nie mehr verschoben werden muss: Sie gehört nicht mehr zum zu sortierenden Bereich. Daher rücken wir bei (5.) den Anfangszeiger um eine Position nach rechts und wiederholen das Sortieren so lange, bis die unsortierte Folge nur noch aus einer einzelnen Karte besteht. Eine Karte ist aber sowieso immer sortiert, daher sind wir fertig!

Ich will das Vorgehen hier gar nicht weiter erklären. „Begreifen“ Sie lieber den Algorithmus, indem Sie mehrere zufällige Folgen von Karten sortieren. Nehmen Sie hierfür auch gerne die Karten mit den echten Zahlen. Sie können sowohl die rote als auch die blaue Reihe nutzen – wenn Sie das abwechselnd tun, sparen Sie sich das Mischen zwischendurch.

Besser durch Blubberblasen?

Damit wir noch etwas zum Spielen haben, erkläre ich gleich ein weiteres, sehr bekanntes Sortierverfahren, das wegen seiner Einfachheit besonders in älteren Programmen recht häufig eingesetzt wird.

Die generelle Idee ist dabei: Gehe die Reihe mit Karten von Anfang bis Ende durch. Vergleiche dabei immer zwei direkt nebeneinander liegende Karten. Sind diese richtig sortiert, mache gar nichts, ansonsten vertausche beide.

Können Sie wieder das grundlegende Prinzip der Informatik erkennen? Jede Vertauschung stellt etwas mehr Ordnung her, bis alle kleinen Fortschritte zusammen die gesamte Arbeit vollbracht haben.

Versuchen Sie das anhand einer zufälligen Reihe, wie in Abbildung 2.9.



Abbildung 2.9
Unsortierte Folge

Nach dem Durchgehen der Reihe ist die Folge bereits etwas mehr sortiert (Abbildung 2.10).



Abbildung 2.10
Folge nach einem Durchlauf
des Verfahrens

Überlegen Sie, was genau Sie über die Folge nach dem ersten Durchlauf sagen können und ob man diese Aussage so formulieren kann, dass sie für jede beliebige Kombination von Karten gilt, nachdem Sie den Algorithmus einmal durchgeführt haben.



Überlegungen dieser Art müssen Informatiker oft anstellen: das Schließen auf eine allgemeine Regel aufgrund eines Beispiels.

In diesem Fall sind nach dem ersten Durchlauf zwei Karten korrekt sortiert: **C** und **Q**. Ist das aber immer so? Nein! Nimmt man als Ausgangsfolge zum Beispiel **PEQCGD**, kommt nach dem ersten Durchlauf **EPCGDQ** heraus, wie in Abbildung 2.11 dargestellt. **C** steht also nicht am Anfang, aber wiederum **Q** am Ende.

Wenn Sie darüber nachdenken, muss das auch so sein: Der Reihe nach wird durch das Vertauschen zweier Karten die größere der beiden immer nach rechts gebracht. Die größte Karte wechselt daher auf die Position ganz rechts, wo sie hingehört.

Abbildung 2.11
Unsortierte Folge



Auch dieses Sortiervfahren bringt also pro Durchlauf eine Karte an die richtige Stelle. Wenn wir es so oft durchführen, wie Karten vorhanden sind, müssen also danach alle Karten sortiert sein.

Versuchen Sie nach diesem Prinzip einen Algorithmus zu formulieren, in ähnlicher Weise wie oben Selection-Sort. Sie können wieder die Zeiger Start, Ende und Speicherzeiger verwenden, um den Bereich zu markieren, der noch unsortiert ist.



Eine mögliche, in der Praxis häufig verwendete Formulierung sehen Sie in Abbildung 2.12: Wenn in einer Anweisung „Wiederhole Folgendes ...“ vorkommt, bezieht sich „Folgendes“ auf den eingerückten Text darunter.

Der Algorithmus nennt sich übrigens „Bubblesort“, weil man sich vorstellt, dass die Karten durch den immerwährenden Tausch an die korrekte Position kommen, so wie Luftblasen im Wasser an die Oberfläche steigen.

Gut, besser, bester?

Das Finden verschiedener Algorithmen zur Lösung eines Problems ist für Informatiker sehr wichtig! Mindestens genauso wichtig ist jedoch, unter allen möglichen Algorithmen den besten zu finden.

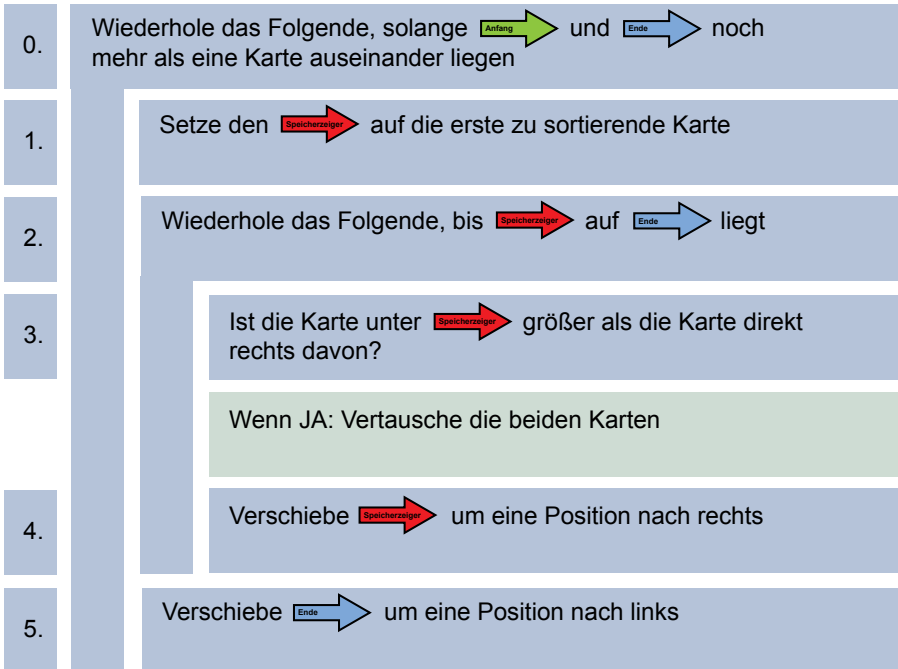
Für einen guten Algorithmus kann es selbstverständlich sehr viele verschiedene Kriterien geben, zum Beispiel wie viel Programmieraufwand man benötigt oder wie elegant ein entsprechendes Computerprogramm aussieht. Für manche Anwendungen gilt sogar, dass ein unübersichtliches Programm sehr viel besser ist als ein übersichtliches, weil es von Konkurrenten schwerer abgeschrieben werden kann.

Normalerweise untersuchen wir jedoch immer die Zeit, die ein Algorithmus zur Lösung einer bestimmten Problemgröße benötigt – je schneller, desto besser. Genau wie bei den Menschen gibt es selbstverständlich auch Computer, die mehr oder weniger zügig arbeiten, und so kann das gleiche Verfahren auf unterschiedlichen Rechnern unterschiedlich viel Zeit in Anspruch nehmen. Daher müssen wir eine vergleichbare, nur auf den Algorithmus, nicht auf die Maschine zutreffende Basis schaffen:

Hierfür legen wir einfach fest, welche Schritte des Algorithmus im Computer Zeit kosten. Man spricht hier auch von Operationen. Diese werden dann bei der Durchführung einfach gezählt.

 und  zeigen auf die erste bzw. die letzte zu sortierende Karte

Abbildung 2.12
Ein mögliches Struktogramm für Bubblesort



Da normalerweise Zugriffe auf den Speicher immer besonders zeitintensiv sind, können für unseren Papierspeicher folgende Schritte ausgewählt werden:

- Verschieben einer Karte auf eine neue Speicherposition
- Vergleich zweier Karten

Selbstverständlich gibt es noch viele weitere Operationen, wie zum Beispiel Verschieben eines Zeigers, Wenn-dann-Anweisung usw. Wie bereits oft in diesem Buch verwenden wir jedoch das Prinzip der Abstraktion, um uns das Leben so einfach wie möglich zu machen – in diesem Fall, indem wir nur die relevanten Operationen auswählen.

Elementaroperationen

Operationen bzw. Arbeitsschritte, die wir bei der Bestimmung der Laufzeit eines Algorithmus für wichtig bzw. zeitkritisch erachten. Welche Operationen hier ausgewählt werden, ist nicht global definiert, sondern vom jeweiligen Anwendungsfall abhängig. Oft handelt es sich um Speichervorgänge (Lesen und Schreiben im Speicher).

Benchmark-Tests

Wir beschäftigen uns hier damit, die Laufzeit von Algorithmen durch theoretische Betrachtungen abzuschätzen. Ein anderer Ansatz ist, es einfach auszuprobieren. Das nennt man „Benchmarking“ (von engl. Benchmark = Maßstab). Hier werden die zu testenden Programme (oder auch ganze Systeme) mit Testdaten laufen gelassen. Der benötigte Zeitaufwand zum Lösen der Aufgaben wird gemessen und verglichen.

Preisfrage: Wie viele unserer Elementaroperationen benötigt das weiter oben beschriebene Verfahren zum Tausch zweier Karten im Papierspeicher?

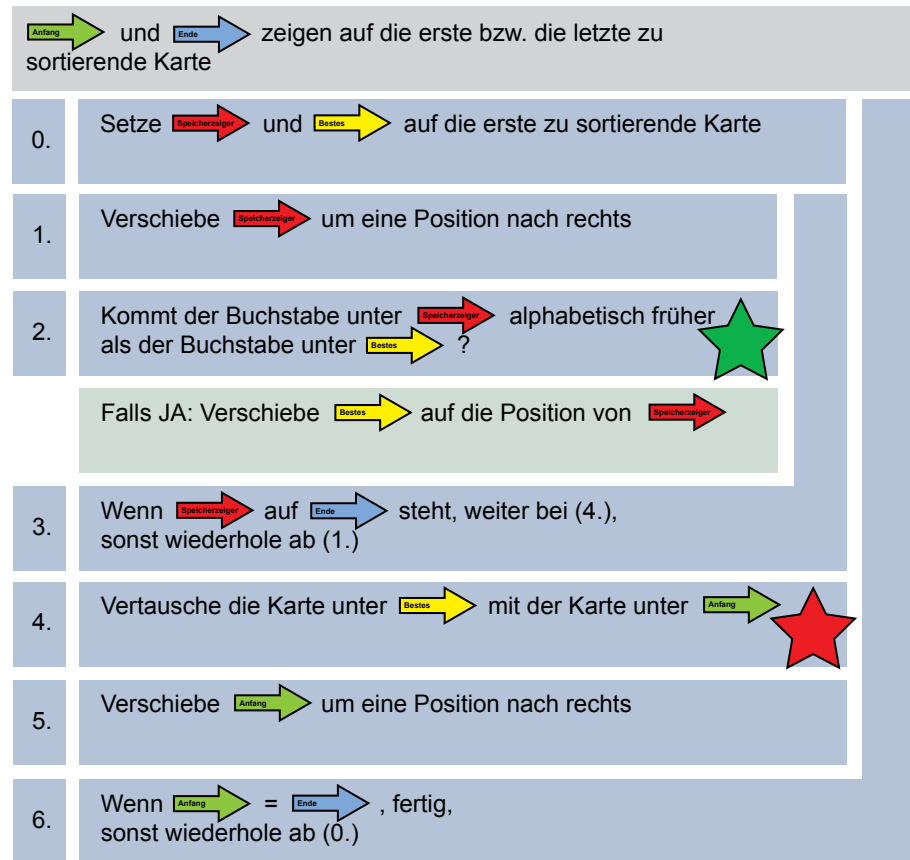


Um die Karten auf den Speicherpositionen A und B zu tauschen, sind drei Verschiebeoperationen nötig:

- Verschiebe Karte von A auf Zwischenspeicher
- Verschiebe Karte von B nach A
- Verschiebe Karte von Zwischenspeicher nach B

Wenn wir auf diese Weise die bisher aufgeschriebenen Algorithmen für Selection-Sort und Bubblesort analysieren, können wir an den mit farbigen Sternen markierten Stellen Elementaroperationen identifizieren. Grün steht für einen Vergleich (1 Elementaroperation), Rot für den Austausch zweier Speicherpositionen (3 Elementaroperationen). Die Abbildungen 2.13 und 2.14 zeigen die Struktogramme.

Abbildung 2.13
Selection-Sort mit markierten
Elementaroperationen



Jetzt sind Sie an der Reihe: Überprüfen Sie experimentell, welches Sortierverfahren das bessere ist, indem Sie verschiedene Kartenanzahlen mit beiden Verfahren sortieren. Notieren Sie sich dann (zum Beispiel mit einer Strichliste) jedes Mal den Vergleich zweier Karten und (dreifach) den Austausch zweier Karten. Wenn Sie die bunten Sortierkarten nutzen, können Sie abwechselnd die roten und blauen Zahlen nutzen, um dazwischen nicht immer wieder mischen zu müssen.



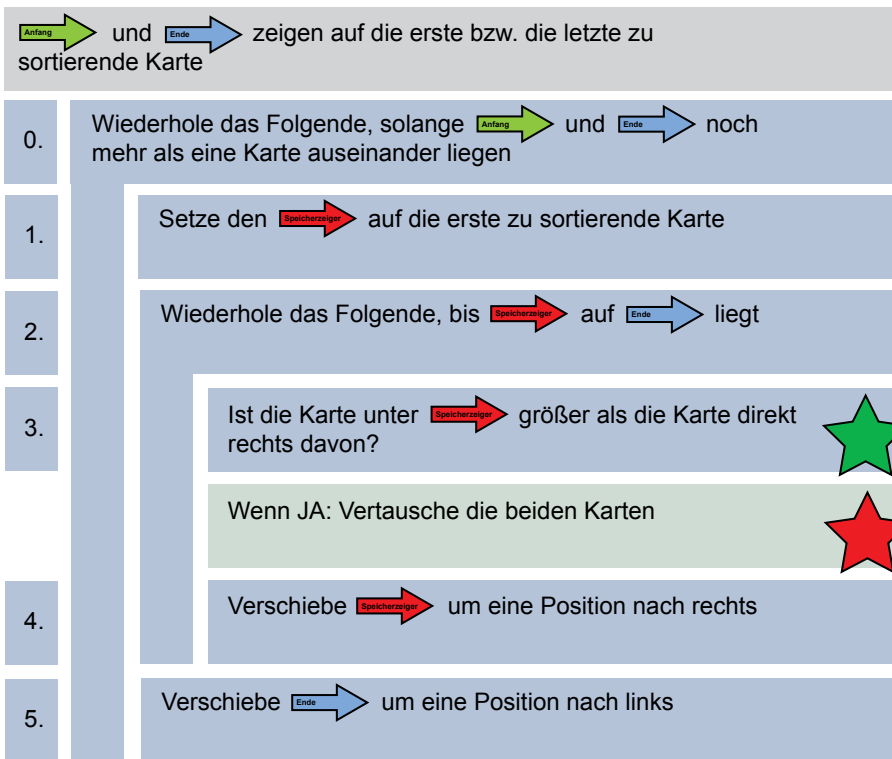


Abbildung 2.14
Bubblesort mit markierten
Elementaroperationen

Selbstverständlich ist das Ergebnis davon abhängig, wie gut die Karten jeweils gemischt waren. Wenn man zufällig eine bereits sortierte Folge als Ausgangssituation hat, dann benötigt man zum Beispiel gar keine Vertauschungen. Bis auf solche Spezialfälle kann man an der entstehenden Tabelle jedoch recht gut ablesen, wie die Rechenzeit in Abhängigkeit der Problemgröße ansteigt.

Bei meinen Versuchen mit bis zu 35 Karten bin ich auf Ergebnisse nach der Tabelle auf der nächsten Seite gekommen. Dabei habe ich in gesonderten Spalten auch den „Aufwand pro Karte“ vermerkt, indem ich die Anzahl der Elementaroperationen nochmals durch die Anzahl der Karten geteilt habe.

Wie kann man dies interpretieren? Genau wie beim komplett manuellen Sortieren, wie wir es ganz am Anfang durchgeführt haben, steigt der Aufwand pro Karte mit der Anzahl der Karten. Man sagt, die Verfahren sind nicht linear, was bedeutet, dass sich der Aufwand nicht mit einem konstanten Faktor aus der einfachen Problemgröße ableiten lässt.

Weiterhin kann man ablesen, dass Bubblesort fast durchgehend schlechter ist als Selection-Sort: Der Aufwand pro Karte ist bei Bubblesort ungefähr doppelt so groß wie bei Selection-Sort.

Der folgende Abschnitt (bis zur nächsten Überschrift) erfordert etwas mathematisches Vorwissen. Daher ist es nicht schlimm, wenn Sie ihn gegebenenfalls nicht komplett verstehen oder gleich ganz überspringen möchten.

Insgesamt wächst der Aufwand pro Karte bei beiden Verfahren etwa linear zur Problemgröße n . Bei Selection-Sort kann man ihn etwa mit $0,6 \cdot n$ abschätzen. Bei Bubblesort sind es etwa $1,3 \cdot n$.

Nur einige der vielen bekannten Sortieralgorithmen:

Bogosort
Bubblesort
Cocktailsort
Combsort
Gnomesort
Heapsort
Insertionsort
Introsort
Mergesort
Quicksort
Selection-Sort
Shellsort
Smoothsort
Stooogesort
Tournament-Sort

Tabelle

Aufwand für das Sortieren verschiedener Kartenzahlen

Karten (=Problem- größe)	Selection- Sort	Bubblesort	Selection- Sort pro Karte	Bubblesort pro Karte
2	4	4	2,0	2,0
3	9	12	3,0	4,0
4	15	6	3,8	1,5
5	22	22	4,4	4,4
6	30	33	5,0	5,5
7	39	57	5,6	8,1
8	49	79	6,1	9,9
9	60	111	6,7	12,3
10	72	96	7,2	9,6
11	85	127	7,7	11,5
12	99	201	8,3	16,8
13	114	183	8,8	14,1
14	130	241	9,3	17,2
15	147	273	9,8	18,2
16	165	309	10,3	19,3
17	184	403	10,8	23,7
18	204	366	11,3	20,3
19	225	441	11,8	23,2
20	247	535	12,4	26,8
21	270	594	12,9	28,3
22	294	534	13,4	24,3
23	319	616	13,9	26,8
24	345	756	14,4	31,5
25	372	828	14,9	33,1
26	400	787	15,4	30,3
27	429	861	15,9	31,9
28	459	1014	16,4	36,2
29	490	1006	16,9	34,7
30	522	1074	17,4	35,8
31	552	1194	17,8	38,5
32	586	1251	18,3	39,1
33	620	1188	18,8	36,0
34	653	1335	19,2	39,3
35	690	1426	19,7	40,7

Für den Gesamtaufwand T müssen wir natürlich noch mit der Anzahl der Karten n multiplizieren und kommen auf

$$T_{\text{Selection-Sort}}(n) \approx 0,6 \cdot n^2 \text{ sowie}$$

$$T_{\text{Bubblesort}}(n) \approx 1,3 \cdot n^2.$$

Wie sehr können wir eigentlich den Faktoren 0,6 bzw. 1,3 vertrauen? Dazu können Sie ein weiteres Experiment durchführen: Nehmen wir an, die Sortierung würde auf einem speziellen Computer durchgeführt, der die Operation „vertauschen“ hardwareunterstützt durchführen kann. Daher koste diese nun – wie der Vergleich – nur noch eine Elementaroperation. Wiederholen Sie nun das Experiment für beide Sortierverfahren Selection-Sort und Bubblesort!

Während sich bei Selection-Sort am Ergebnis wahrscheinlich nicht viel verändert hat, ist sicherlich Bubblesort deutlich „besser“ geworden: Selection-Sort nutzt ja genau eine Vertausch-Operation pro Karte. Bubblesort nutzt hingegen, abhängig von der gegebenen Vorsortierung, deutlich häufiger die Vertauschung, weshalb sich der Faktor durch die neue Annahme deutlich verbessert. Bei mir ergaben sich:

$$T_{\text{Selection-Sort}}(n) \approx 0,5 \cdot n^2 \text{ sowie}$$

$$T_{\text{Bubblesort}}(n) \approx 0,7 \cdot n^2.$$

Daraus lässt sich eine ganz entscheidende Erkenntnis ableiten: Konstante Faktoren sind im Allgemeinen sehr stark abhängig von den verwendeten Rechner-systemen und von den Annahmen, die wir für die notwendigen Rechenzeiten treffen. Letztlich nehmen wir hier auch wiederum eine Modellierung vor. Wenn wir diese Modellierung gewissenhaft durchführen, verändert sich in der Regel der höchste Exponent hinter der Problemgröße n nie.

Hinzu kommt, dass konstante Faktoren durch die Wahl eines schnelleren Computers immer ausgeglichen werden können. Wären zum Beispiel 1.000.000 Karten zu sortieren, bräuchte mit den ursprünglichen Annahmen Selection-Sort ungefähr 600 Milliarden Elementaroperationen, Bubblesort 1.300 Milliarden. Selbstverständlich würde man in der Praxis den schnelleren Algorithmus einsetzen. Andererseits kann das Manko jederzeit durch Einsatz eines etwa doppelt so schnellen Computers ausgeglichen werden. Für bestimmte Gegebenheiten mag sogar Bubblesort besser geeignet sein: Es werden immer nur direkt benachbarte Speicherstellen miteinander verglichen. Denkt man

daran, dass die Daten auf einer Festplatte liegen, geht also der Vergleich ohne große Bewegung des Schreib-/Lesekopfes vonstatten. Bei Selection-Sort hat man diesen Vorteil nicht.

Wesentlich ist allerdings, wie stark der Zeitaufwand anwächst, wenn die Problemgröße anwächst! Dieses Verhältnis spiegelt sich im höchsten Exponenten und lässt sich nicht durch Wahl eines stärkeren Rechners verändern – genauso wenig wie durch die Feinjustage des Algorithmus!

Wenn wir einen besseren Algorithmus finden möchten, ist es also wesentlich, dieses Verhältnis zwischen Problemgröße und Aufwand zu verbessern. Konstante Faktoren spielen eine untergeordnete Rolle! Um Algorithmen miteinander zu vergleichen, lässt man daher normalerweise konstante Faktoren weg. Mit dieser Betrachtungsweise sind beide Sortierverfahren gleichwertig, sie haben quadratische Laufzeit. Wer es formal nicht so genau nimmt, sagt zu ihnen „quadratische Sortierverfahren“.

Nicht nur in der Informatik, auch in anderen Wissenschaften sucht man immer möglichst einfache Beschreibungsmöglichkeiten. Eine kurze, prägnante Formel ist immer vorzuziehen gegenüber einem komplizierten und langen Ausdruck. Wichtig ist, dass alle wesentlichen Aspekte berücksichtigt wurden.

Wie unterscheidet sich denn nun ein Sortierverfahren, das zu einer anderen Kategorie gehört? Nehmen wir an, es gäbe eines, das kubisch, also proportional zu Problemgröße-hoch-3 funktioniert. Das würde für die 1.000.000 Karten bereits etwa 1 Trillion Elementaroperationen benötigen, wäre also nicht nur etwas schlechter, sondern würde gleich um Größenordnungen länger benötigen, die Aufgabe zu lösen. Glücklicherweise haben wir ja bereits bessere Verfahren kennen gelernt.

Schön wäre allerdings, wenn wir ein Verfahren hätten, das noch weniger Rechenschritte benötigt. Hier kann ich vorwegnehmen: Die Elite der „normalen“ Sortierverfahren läuft proportional zu $n \cdot \log_2 n$ mit n als Problemgröße. Damit lassen sich 1.000.000 Karten ungefähr mit einem Vielfachen von 10.000 Elementaroperationen sortieren, was wiederum um Größenordnungen besser ist als Bubble- oder Selection-Sort.

Die Aufwandsabschätzung

Mit der Aufwandsabschätzung wird in der Informatik oft die Qualität von Algorithmen bestimmt. Diese Abschätzung gibt an, wie stark die notwendige Rechenzeit in Bezug zur Problemgröße anwächst. Konstante Faktoren werden hier nicht berücksichtigt. Auf diese Weise kann für kleine Problemgrößen die tatsächliche Rechenzeit eines „schlechteren“ Algorithmus unter der des besseren liegen.

Mit der hohen Kunst der Aufwandsabschätzung wollen wir nun einen noch besseren Sortieralgorithmus finden.

nlogn oder die Kür des Sortierens

Im folgenden Abschnitt werde ich der Kürze halber immer n als Bezeichnung der Problemgröße verwenden, ohne das jedes Mal erneut zu erwähnen. Das ist der unter Informatikern übliche Buchstabe dafür.

Weitere Quantensprünge?

Die hier herausgearbeiteten Prinzipien für Aufwandsabschätzung gelten für herkömmliche Computer, die immer genau eine Elementaroperation auf einmal durchführen können (oder mit mehreren Prozessorkernen eine genau definierte Zahl von Elementaroperationen auf einmal).

Manche Forscher versuchen jedoch, Computer zu bauen, die völlig anders – auf Basis von Wahrscheinlichkeiten – operieren. Für unsere konventionellen Berechnungen würde das dann so aussehen, als ob diese Computer bei höheren Problemgrößen auch eine höhere Zahl von Elementaroperationen gleichzeitig verarbeiten könnten. Nach diesem Prinzip arbeiten zum Beispiel Quantencomputer. Andere Forscher legen jedoch Ergebnisse vor, nach denen Quantencomputer in sinnvollen Größenordnungen nie funktionieren werden.

Im letzten Abschnitt haben wir experimentell nachvollzogen, warum die bisher durchgesprochenen Sortiervverfahren eine quadratische Aufwandsabschätzung (also n^2) besitzen. Man kann das jedoch auch anders begründen: Ein Sortierschritt der Verfahren sorgt jeweils dafür, dass ein einzelnes Element korrekt positioniert wird (am Ende der Folge). Wir benötigen also n Sortierschritte, bis alle Karten korrekt liegen.

Betrachten wir nun einen einzelnen Sortierschritt: Hier wird die gesamte Liste durchgegangen – bei Selection-Sort, um das größte Element zu finden, bei Bubblesort, um jeweils Paare in die korrekte Reihenfolge zu bringen. Im ersten Schritt müssen wir also n Karten anfassen, im zweiten Schritt eine weniger und so weiter, bis im letzten Schritt nur eine Karte betrachtet wird. Im Durchschnitt sind also pro Schritt etwa $\frac{n}{2}$ Karten anzufassen (was einem Vielfachen von n Operationen entspricht).

Insgesamt gibt es also

- n Durchläufe
- mit je n Operationen.

Multipliziert ergibt das einen Aufwand von $\frac{n^2}{2}$ Operationen. Da wir konstante Faktoren nicht betrachten wollten, fassen wir das zu einer Aufwandsabschätzung von n^2 zusammen.

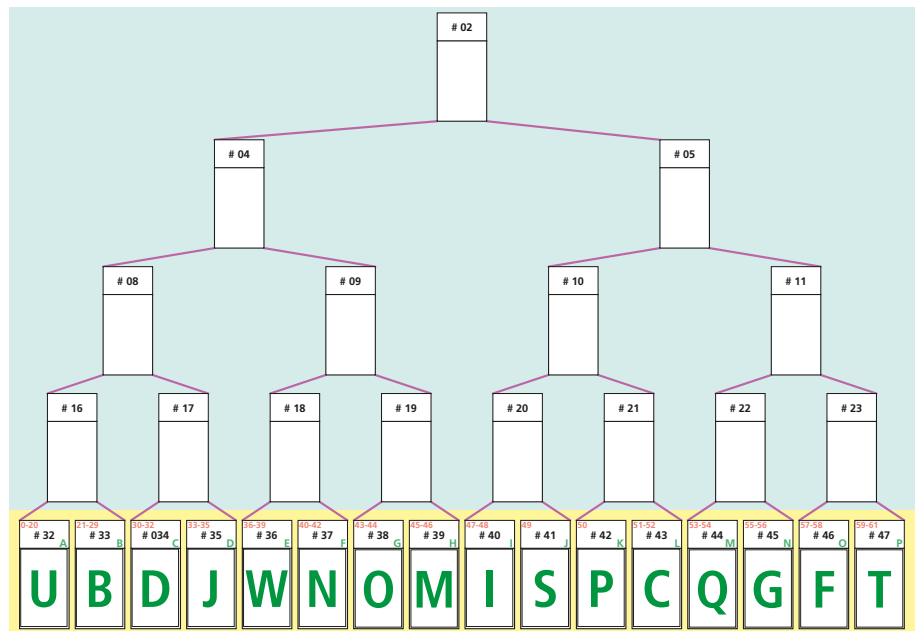
Widmen wir uns der nächsten großen Aufgabe: Ein besseres Sortiervverfahren soll gefunden werden!

Das können wir offenbar erreichen, indem wir

- entweder die Anzahl der Durchläufe reduzieren oder
- den Aufwand pro Durchlauf reduzieren.

Wie schon im letzten Kapitel können wir hier auch wieder alltägliche Situationen als Vorbild nutzen – in diesem Fall ein im Sport typischer Wettkampf im K.-o.-Verfahren. Allerdings lassen wir in unserem Fall nicht Sportler, sondern Karten gegeneinander

Abbildung 2.15
Die Karten treten paarweise gegeneinander an.



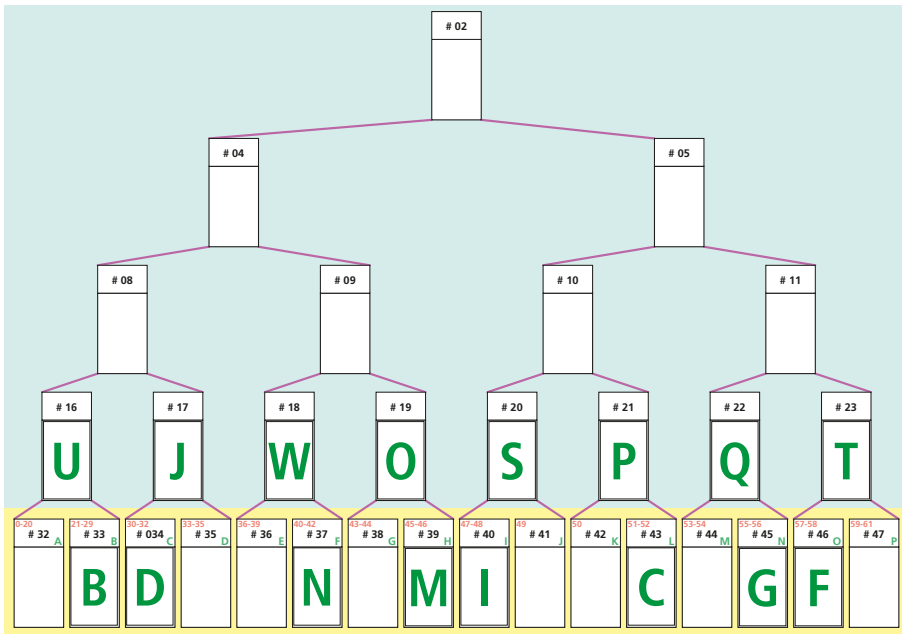


Abbildung 2.16
Das Achtelfinale wurde aus-
getragen.

antreten. Verwenden Sie den Papierspeicher für den Wettkampf – diesmal einschließ-
lich der Speicherstellen #01 bis #31. Legen Sie aber zunächst die unsortierte Folge auf
die bereits bekannten Speicheradressen #32 bis #47, also die Hälfte des Papierspei-
chers mit #02 als oberstem Element. Ein Beispiel sehen Sie in Abbildung 2.15.

Daraufhin spielen wir die erste K.-o.-Runde aus. Die jeweils größere Karte eines Paa-
res „gewinnt“ und steigt eine Runde auf, wie in Abbildung 2.16. Im Sport entspräche
das dem Achtelfinale, dessen acht Sieger um den Einzug ins Viertelfinale spielen.

Als Nächstes rücken die jeweils größeren Karten des Viertelfinales auf die vier höhe-
ren Felder vor, quasi als Halbfinalisten. Abbildung 2.17 zeigt das.

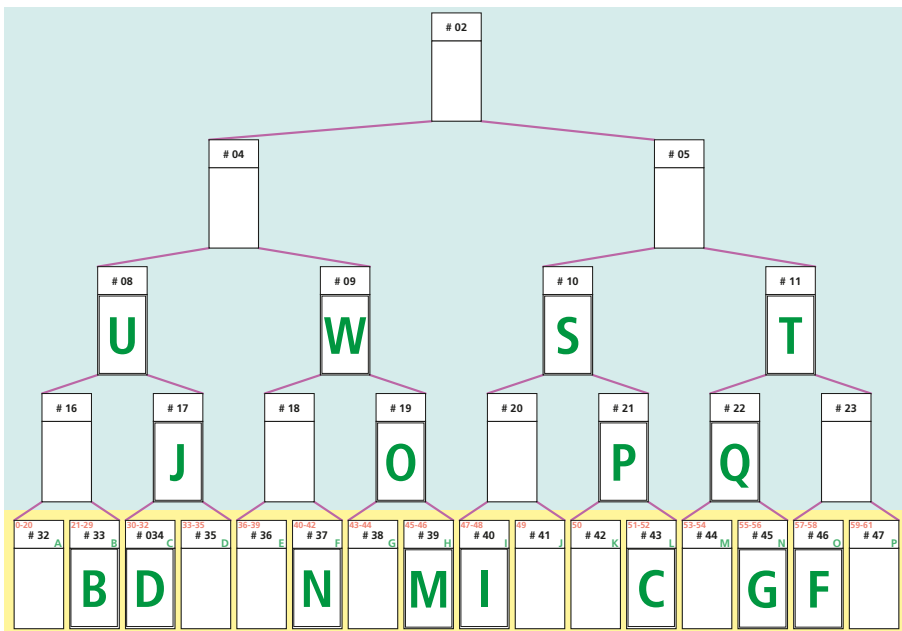
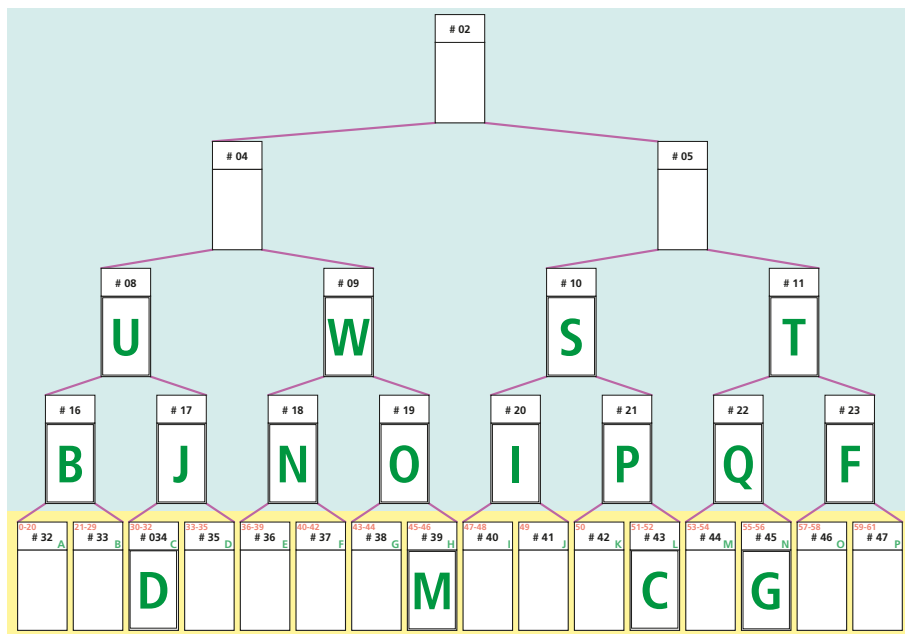


Abbildung 2.17
Die Sieger des Achtelfinales
stehen im Viertelfinale

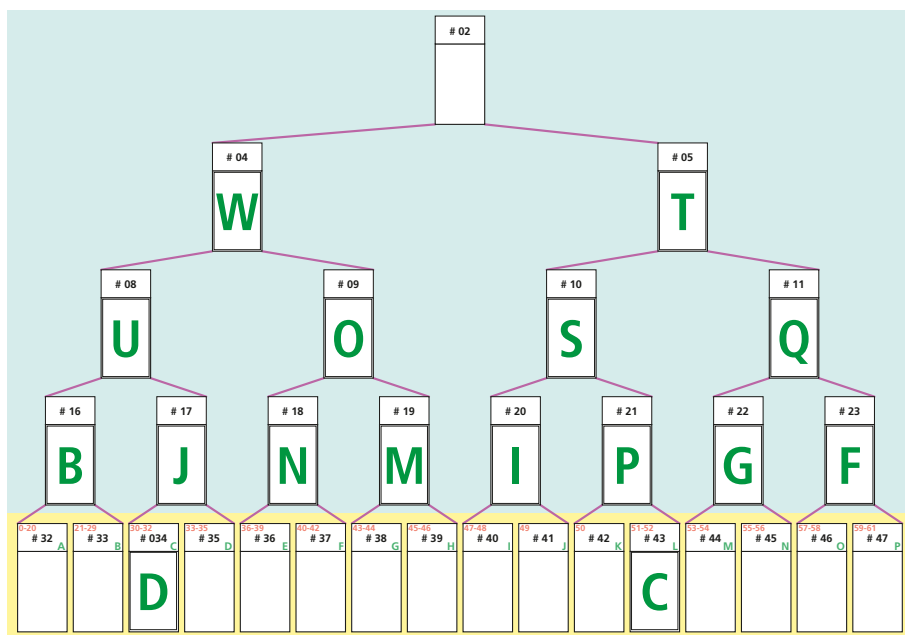
Abbildung 2.18
Belegung des Viertelfinales
mit nachgerückten Kandida-
ten des Achtelfinales



Im Sport wären die ausgeschiedenen Mannschaften bereits nach Hause gefahren. Beim Sortieren wäre es allerdings ungünstig, die „Verlierer“ gar nicht mehr weiter zu berücksichtigen. Daher werden nun die frei gewordenen Plätze des Viertelfinales aufgefüllt: Von unten rücken die entsprechenden Karten nach. In Abbildung 2.18 sehen Sie, wie zum Beispiel **B** von #33 nach #16 aufgestiegen ist.

Nun wird nach dem gleichen Schema das Halbfinale ausgespielt. Es gewinnen W und T, sie rücken ins Finale auf. Beachten Sie, dass wiederum die frei gewordenen Plätze des Halbfinals belegt werden. Dafür stehen nun jeweils zwei Kandidaten zur Verfügung und es wird „ausgespielt“, wer aufsteigt. In diesem Fall kommt **O** auf #09 und **Q**

Abbildung 2.19
Vor dem Finale



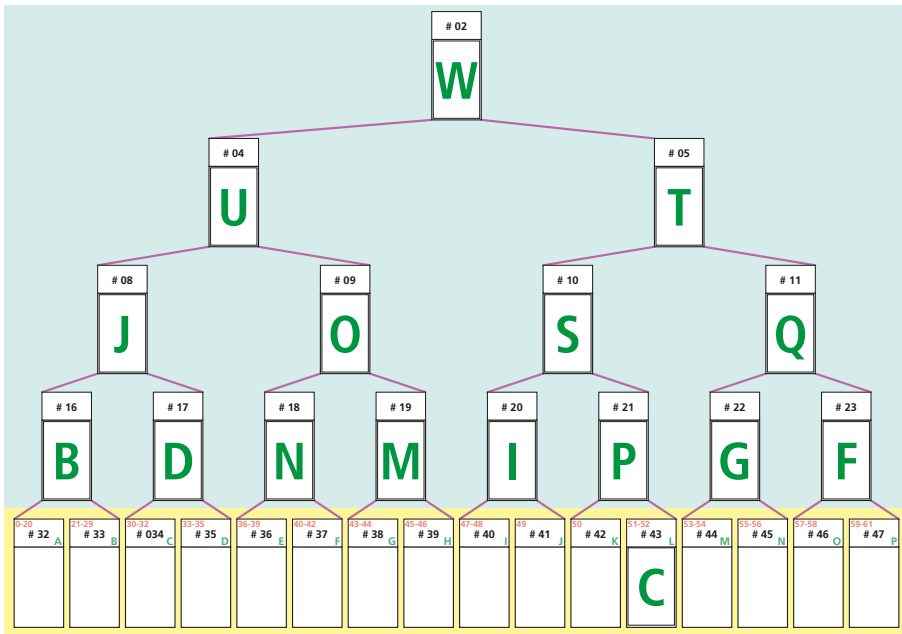


Abbildung 2.20
Fertig ausgespieltes K.-o.-
System

auf #11. Falls möglich, werden danach noch die verwaisten Plätze des Viertelfinales von unten aufgefüllt. Abbildung 2.19 zeigt das Ergebnis.

Nun kommt nur noch das Finale – **W** gewinnt und kommt auf das Siegerfeld #02. Die entstehende Lücke wird selbstverständlich sofort von den jeweils nächstplatzierten Karten der darunterliegenden Ebenen ausgefüllt, die dadurch entstandene Lücke auch usw. In diesem Fall rückt die Karte **U** nach, dann **J** und **D**. Abbildung 2.20 zeigt das fertige K.-o.-System. Spielt man mit dem kompletten Papierspeicher, dauert es bis zum Finale eine Runde länger und das Siegerfeld ist die rot markierte Position #01.

Was bringt uns das Ganze jetzt? Durch das K.-o.-System haben wir garantiert die größte Karte gefunden, denn die liegt auf dem Siegerfeld. Sie gehört an das Ende der sortierten Liste. Dort schieben wir sie daher auch hin. In diesem Fall ist das **W**.

Der Siegerplatz ist nun vakant und selbstverständlich rücken die nächsten Karten nach, indem nacheinander das Finale sowie je eine Partie des Halbfinals und Viertelfinales ausgetragen werden, bis kein Kandidat zum Nachrücken von unten mehr da ist.

Versuchen Sie selbst, die Situation zu ermitteln, nachdem W an die Endposition der sortierten Liste verschoben wurde.



Ihr Resultat können Sie mit Abbildung 2.21 vergleichen. Genau das gleiche Prinzip verwenden Sie nun, um immer wieder den nächsten Sieger auf die letzte freie Position der fertigen Sortierung zu verschieben und das Siegerfeld neu zu füllen, indem Sie die Karten darunter nachrücken lassen. Probieren Sie es aus! Zur Kontrolle habe ich Ihnen in Abbildung 2.22 noch ein letztes Zwischenergebnis dargestellt.

Sie werden jetzt vielleicht überlegen, wozu der ganze Aufwand eigentlich gut ist, denn wir haben ja bereits zwei gut funktionierende Sortierverfahren kennen gelernt. Was ist anders am K.-o.-Verfahren, das übrigens nach dem englischen Wort für Wettbewerb

Abbildung 2.21

Situation nach dem „Auscheiden“ des Siegers und Nachrücken der nächsten Kandidaten

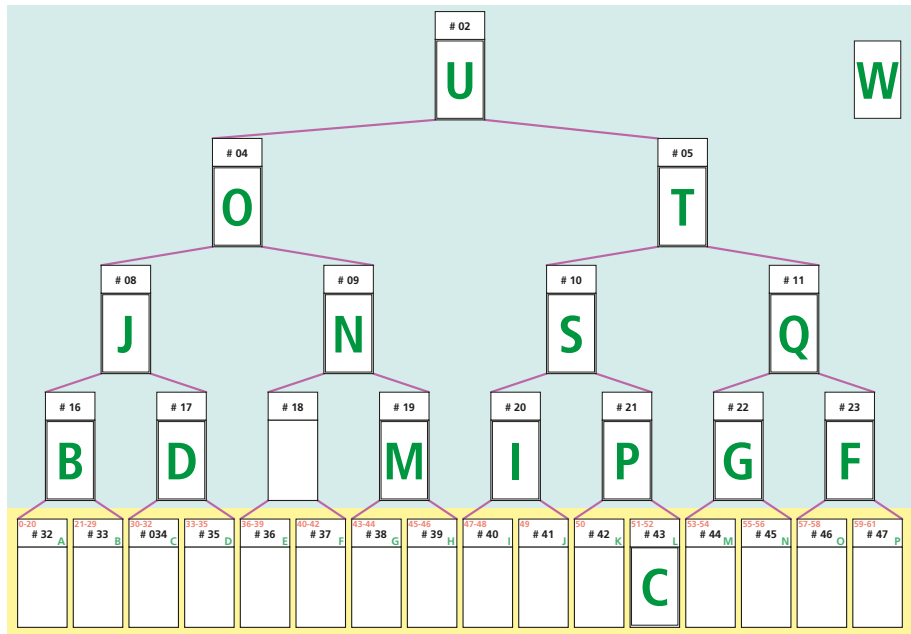
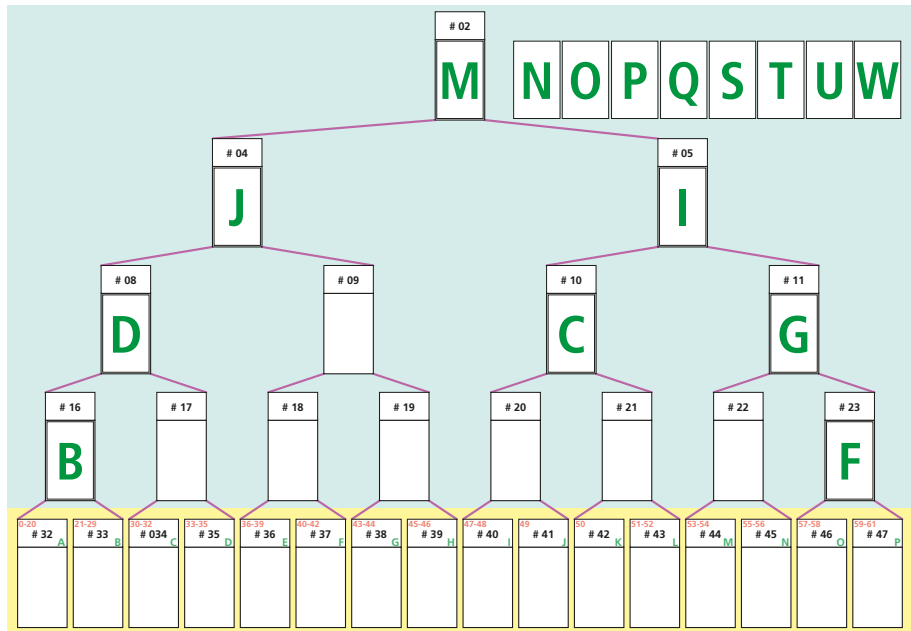


Abbildung 2.22

Die Hälfte der Karten ist nun sortiert.



„Tournament-Sort“ genannt wird? Für diese Analyse benötigen wir erst einmal die genaue formale Fassung des Algorithmus.

Im Aufstellen formaler Algorithmen sind Sie ja inzwischen bereits Meister. Wollen Sie das ein weiteres Mal unter Beweis stellen, indem Sie für Tournament-Sort eine Ablaufvorschrift erstellen?



Meinen Vorschlag finden Sie in Abbildung 2.23.

0.	Gehe die leeren Ebenen von unten nach oben durch und fülle jedes Feld mit der größeren Karte der beiden zugehörigen Wettbewerber		
1.	Fülle dabei weiter unten entstehende Lücken sofort wieder mit einer der Karten darunter auf:		
2.	bei zwei Wettbewerbern mit der größeren Karte	bei nur einem Wettbewerber mit diesem	bei keinem Wettbewerber bleibt das Feld leer
3.	Mache Folgendes so lange, bis keine Karte mehr im Siegerfeld liegt		
4.	Verschiebe die Siegerkarte auf das erste freie Feld am Ende der sortierten Liste		
5.	Fülle dabei weiter unten entstehende Lücken sofort wieder auf wie oben beschrieben		

Versuchen Sie nun, diesen Algorithmus genau so zu analysieren, wie wir dies bereits oben getan haben. Wie viele Durchläufe benötigt er für die Sortierung? Wie hoch ist der Aufwand pro Durchlauf?

Kleiner Hinweis: Im Prinzip besteht der Algorithmus aus einer Art Vorbereitungsschritt und der eigentlichen Sortierung. Beides können wir auch getrennt voneinander betrachten.



Im ersten Teil wird die K.-o.-Tabelle vorbereitet, die erste Karte auf dem Siegerfeld ermittelt. Hierfür muss jedes (anfangs leere) Feld im oberen Bereich der K.-o.-Tabelle mit einer Karte belegt werden. Wie viele leere Felder gibt es übrigens für eine K.-o.-Tabelle mit n Sortierkarten?

Auf der untersten Ebene im gelben Bereich befinden sich n Felder mit den unsortierten Karten. Auf der untersten K.-o.-Ebene sind es halb so viele, da je eine von zwei Karten gewinnt. Es folgen wiederum halb so viele, also ein Viertel usw., bis nur noch ein Feld, das Siegerfeld, in der obersten Ebene steht.

Insgesamt gibt es also $\frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \dots + 1$ Felder auf den Siegerebenen.

Einer mathematischen Rechenregel nach ergibt diese sogenannte geometrische Reihe ungefähr wieder n , also $\frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \dots + 1 \approx n$.

Sie können sich das einfach auch durch Ausprobieren klarmachen: Legen Sie eine Anzahl Karten in einer Reihe aus und dann für jedes Feld der K.-o.-Tabelle eine Münze darüber – also in der Reihe oberhalb eine Münze für je zwei Karten, in der Reihe darüber wieder eine Münze für je zwei Münzen darunter usw.

Zählen Sie die Anzahl der Münzen. Je nachdem, ob die Anzahl der Karten gerade ist bzw. wie oft sie durch zwei teilbar ist, ist sie etwas kleiner oder etwas größer als die Zahl der Münzen, aber immer ungefähr gleich.

Die gesamte Anzahl der Siegerplätze eines K.-o.-Systems ist immer etwa gleich groß wie die Anzahl der Wettbewerber.

Wir haben nun also herausgefunden, dass jedes Feld der leeren K.-o.-Tabelle gefüllt werden muss und dass es insgesamt ungefähr n Felder gibt! Nennen wir das einmal n Durchläufe.

Beim Füllen eines Feldes müssen gegebenenfalls weiter unten frei werdende Felder mit Nachrückern bestückt werden. Maximal passiert das so oft, wie es Ebenen in der Tabelle gibt. Man nennt die Anzahl der Ebenen auch die Höhe der Tabelle oder einfach h .

Der Aufwand unseres Tournament-Sorts wird also bestimmt von

- n Durchläufen mit jeweils
- h Verschiebeoperationen

oder Aufwand = $n \cdot h$.

Es wäre nun schön, wenn wir die Höhe irgendwie ausrechnen könnten. Ermitteln Sie zunächst durch Probieren, welche Höhe eine Tabelle mit 8, 9, 10, 15, 16, 32 und 64 Karten besitzt.



8 Karten ergeben 4 Ebenen. Bei 9, 10, 15 oder 16 Karten sind es immer 5 Ebenen. Die K.-o.-Tabelle mit 32 Karten hat 6 Ebenen, wie auch einfach am Bastelbogen ablesbar ist. 64 Karten können auf 7 Ebenen sortiert werden.

Ebene	Anzahl Karten
1	1
2	2
3	4
4	8
5	16
6	32
7	64
8	128
...	...
h	2^{h-1}
$h+1$	2^h

Klar wird dies, wenn man bedenkt, dass jede Ebene immer halb so viele Karten enthält (ggf. muss aufgerundet werden) wie die darunterliegende Ebene. Die oberste Ebene enthält eine Karte. Umgekehrt kann also jede untere Ebene (maximal) doppelt so viele Karten enthalten wie die darüber liegende Ebene.

Wenn wir die oberste Ebene mit der Nummer 1 versehen, ergibt sich für das Fassungsvermögen also die Tabelle am Rand.

Eine Tabelle mit 2^h Karten besitzt nach dieser Tabelle also $h+1$ Ebenen!

Da wir die Anzahl der Karten mit n bezeichnet haben, gilt also nun der Zusammenhang $n \approx 2^h$.

Das Ungefähr-Zeichen verwende ich hier, weil Gleichheit nur gelten würde, wenn die Anzahl der Karten einer „glatten“ Zweierpotenz, also 2, 4, 8, 16, ... entspräche.

Mathematiker sehen nun sofort, dass man die Formel mit dem Logarithmus auflösen kann:

$$\log_2 n \approx h$$

Für alle anderen: Sie konnten sich experimentell eine Vorstellung davon machen, wie die Anzahl der Ebenen mit der Anzahl der Karten wächst: Je mehr Ebenen bereits vorhanden sind, desto mehr Karten kann man hinzufügen, ohne die Anzahl der Ebenen zu erhöhen. Mit anderen Worten: Je mehr Karten vorhanden sind, desto langsamer wächst die Anzahl der Ebenen. Dieses Verhältnis wird mathematisch vom Logarithmus ausgedrückt!

Kommen wir jetzt zu unserer Betrachtung zurück, wie viel Rechenzeit der Algorithmus in Bezug zur Anzahl der Karten benötigt.

Oben hatten wir bereits zum Aufbau der K.-o.-Tabelle einen Aufwand von $n \cdot h$ bestimmt. Die Höhe h der Tabelle können wir nun mit $\log_2 n$ angeben. Daher haben wir für diesen ersten Teil des Algorithmus einen Aufwand von $n \cdot \log_2 n$ oder einfach $n \log n$.

Im zweiten Teil des Algorithmus wird jeweils die Siegerkarte in die sortierte Liste verschoben und dann das leere Feld wieder gefüllt. Das passiert so oft, wie Karten vorhanden sind, also n -mal. Jedes Mal muss das leere Feld gefüllt werden und die Anzahl der Verschiebungen entspricht wiederum maximal der Höhe, also $\log_2 n$. Wieder ergibt sich daher insgesamt ein Aufwand von $n \cdot \log_2 n$. Man könnte jetzt sagen, der Aufwand beider Teile zusammen beträgt $2 \cdot n \cdot \log_2 n$, aber konstante Faktoren lassen wir ja einfach weg, daher kommen wir auf den gewünschten Algorithmus mit $n \log n$ Aufwand. Sie können diese viel bessere Laufzeit auch experimentell nachvollziehen:

Sortieren Sie unterschiedliche Kartenzahlen mit Tournament-Sort. Wenn der vorhandene Papierspeicher für Ihren Eifer nicht ausreicht, können Sie leicht mit Haftnotizen auf einer Tischplatte Ihre eigene Version mit mehr Platz bauen. Zählen Sie anhand einer Strichliste jeweils, wie viele Verschiebungen und wie viele Vergleiche Sie durchführen mussten.



In der Tabelle auf der nächsten Seite sind zum Vergleich nochmals die Werte von Selection-Sort aufgeführt, also des besseren der beiden zuerst betrachteten Sortieralgorithmen. Außerdem sind ein paar Werte für recht hohe Kartenanzahlen eingetragen, die ich allerdings nicht per Hand, sondern per Computer ermittelt habe. Diese zeigen deutlich, dass sich ein Verfahren mit einem geringeren Aufwand besonders bei hohen Problemgrößen auszahlt – bei sehr kleinen Kartenzahlen ist Tournament-Sort sogar schlechter!

Bei Selection-Sort steigt der Aufwand pro Karte linear mit der Gesamtanzahl Karten, bei Tournament-Sort lediglich logarithmisch. Sie sehen, dass daher auch große Mengen mit Tournament-Sort noch verarbeitet werden können. Bei bereits 500 Karten benötigt dieser Algorithmus nur noch etwa 7 % der Elementaroperationen von Selection-Sort für die gleiche Aufgabe.

Was steckt dahinter?

Dieses Kapitel stand voll im Zeichen dessen, was in der Informatik Aufwandsabschätzung oder auch Komplexität genannt wird. Bereits im letzten Kapitel war es notwendig, die Qualität zweier Algorithmen gegeneinander abzuwiegen. Dort ging es jedoch hauptsächlich darum, die gegebene Aufgabe des Routenplaners überhaupt zu bewältigen. Einer der Algorithmen – Brute-Force – versagte dabei bereits für kleine Beispiele.

In diesem Kapitel haben Sie nun mehrere Algorithmen kennen gelernt, die auf den ersten Blick gleichwertig sind, da sie für überschaubare Beispiele alle recht gut funktionieren.

Computer werden jedoch oft gerade in Bereichen eingesetzt, die für uns Menschen nicht mehr überschaubar sind. Der Zentralrechner einer Bank muss nicht 30, 500 oder 10.000 Datensätze sortieren, sondern teilweise Milliarden von Transaktionen in die korrekte Reihenfolge bringen. Hierbei ist zwingend erforderlich, die dafür verwendeten Algorithmen nicht intuitiv zu beurteilen, sondern anhand genauerer Maßstäbe.

Im Allgemeinen wird die Abschätzung der Komplexität als Maßstab verwendet!

Tabelle

Aufwand für das Sortieren verschiedener Kartenzahlen

n	Selection	Tournament	Selection pro Karte	Tournament pro Karte
2	4	5	2,0	2,5
3	9	12	3,0	4,0
4	15	16	3,8	4,0
5	22	28	4,4	5,6
6	30	33	5,0	5,5
7	39	41	5,6	5,9
8	49	48	6,1	6,0
9	60	68	6,7	7,6
10	72	73	7,2	7,3
11	85	80	7,7	7,3
12	99	90	8,3	7,5
13	114	97	8,8	7,5
14	130	110	9,3	7,9
15	147	115	9,8	7,7
16	165	126	10,3	7,9
17	184	161	10,8	9,5
18	204	168	11,3	9,3
19	225	179	11,8	9,4
20	247	190	12,4	9,5
25	372	244	14,9	9,8
30	522	289	17,4	9,8
50	1.372	577	27,4	11,5
100	5.247	1.368	52,5	13,7
150	11.622	2.300	77,5	15,3
200	20.497	3.096	102,5	15,5
250	31.872	3.931	127,5	15,7
300	45.747	5.197	152,5	17,3
350	62.122	6.091	177,5	17,4
400	80.997	7.073	202,5	17,7
450	102.372	7.943	227,5	17,7
500	126.247	8.869	252,5	17,7

Wie bereits oben dargestellt, ist dabei oft gar nicht so wichtig, ob ein Algorithmus doppelt so lange benötigt wie ein anderer. Wesentlich ist, wie viel mehr Rechenzeit ein Algorithmus benötigt, wenn die Problemgröße anwächst. Genau dieses Verhalten macht sich nämlich besonders stark bemerkbar, wenn wir mit sehr großen Datenmengen arbeiten.

Wenn wir die Verfahren auf diese Weise analysieren, bekommen wir heraus, welches für unsere Problemstellung am besten geeignet ist. Die Sache hat aber auch noch einen weiteren Vorteil: Wenn wir die Analyse durchführen, machen wir uns besonders deutlich, an welchen Stellen des jeweiligen Verfahrens Rechenzeit verbraucht wird. Das liefert gute Hinweise darauf, wo wir noch etwas verbessern können, um zu einem günstigeren Ergebnis zu gelangen.

Im Beispiel oben hatten wir festgestellt, dass sowohl Selection-Sort als auch Bubblesort grob gesagt n Durchläufe mit jeweils n Operationen benötigen, also insgesamt eine Komplexität von n^2 besitzen. Die Folge war, dass wir auf ein Verfahren hinarbeiten konnten, in dem die Anzahl der Durchläufe gleich blieb, aber die Anzahl der Operationen pro Durchlauf deutlich optimiert wurde: Tournament-Sort.

Einen anderen Ansatz zur Optimierung verfolgt übrigens der von Tony Hoare 1960 erfundene Quicksort-Algorithmus, der auch heute noch in vielen Formen eingesetzt wird: Hier benötigt jeder Durchlauf prinzipiell noch n Operationen, aber die Anzahl der Durchläufe wird reduziert. Auch Quicksort hat daher im Durchschnitt die Komplexität von $n \log n$.

Großes Oh

Im bisherigen Verlauf des Kapitels habe ich die Freiheit der populärwissenschaftlichen Darstellung genutzt und den Algorithmen einfach einem bestimmten Aufwand zugeordnet, also etwa n^2 oder $n \log n$.

Der wirkliche Aufwand eines Algorithmus hängt aber von vielen Faktoren ab: wie die Daten am Anfang sortiert sind, wie das Verfahren genau auf einem Computer umgesetzt wurde usw. Da dieser tatsächliche Aufwand daher gar nicht oder nur sehr schwer zu bestimmen ist, spricht man gewöhnlich von der Ordnung eines Aufwands, wenn man ihn so abschätzt, wie wir das bisher getan haben.

Bubblesort und Selection-Sort sind von der Ordnung n^2 , während Tournament-Sort und Quicksort von der Ordnung $n \log n$ sind.

Informatiker bedienen sich hierfür des Buchstabens O, abgeleitet vom griechischen Omikron, das der Mathematiker Paul Bachmann bereits 1892 verwendete, um die Ordnung von Funktionen zu definieren.

Man sagt daher genauer, Bubblesort und Selection-Sort haben eine Komplexität von $O(n^2)$, gesprochen „groß-Oh von n Quadrat“. Quicksort und Tournament-Sort haben eine Komplexität von $O(n \log n)$. Diese Notation wird Ihnen im Großteil der Informatik-Fachliteratur begegnen.

O(n) oder: Streben nach dem perfekten Algorithmus

Lange Zeit galten Tournament-Sort & Co. als die besten möglichen Sortierv Verfahren. Man kann sogar mathematisch beweisen, dass es für das Sortieren, so wie wir es machen, gar keine besseren Verfahren als solche von der Komplexität $O(n \log n)$ geben kann. Informatiker sind da jedoch recht flexibel und oft lassen sich auch die strengsten mathematischen Beweise dadurch umgehen, dass man die Voraussetzungen dafür abändert. Das haben wir im ersten Kapitel nachvollzogen, als plötzlich die geographischen Positionen der Orte (und damit ihre Entfernungen) in die Betrachtung einbezogen wurden. Bei Proxmap-Sort berücksichtigt man, dass in realen Anwendungen keine Schlüssel vorkommen, die nur verglichen werden können. Was ich damit meine, möchte ich Ihnen erläutern:

Proxmap-Sort ist eigentlich bei genauerer Betrachtung gar nichts Revolutionäres, sondern etwas, das Generationen von Menschen bereits beim Sortieren von Karten oder ähnlichen Objekten intuitiv durchführen.

Nehmen Sie ein Päckchen gewöhnlicher Spielkarten und einen großen Tisch. Ihre Aufgabe ist, die Karten nach ihren Werten zu sortieren. Die Herangehensweise nach den bisherigen Algorithmen wäre, die Karten erst einmal genauso, wie sie sind, auf dem Tisch auszulegen und dann einen der besprochenen Algorithmen durchzuführen.

Effektiv gehen aber in der Realität die wenigsten Menschen auf diese Weise vor. Vielmehr nutzen wir bereits beim Auslegen der Karten unser Wissen über deren Verteilung. Die Verteilung besagt, wie viele Karten welches Typs im Stapel sind. Normalerweise wissen wir also, dass jede Karte von As bis König in einer Farbe genau einmal vorkommt. Nehmen wir zu Beginn eine 7 auf, dann legen wir sie nicht auf die linke Seite, sondern bereits ungefähr an die Position, an die sie gehört, also etwa in die Mitte des Tisches. Ein As kommt an den Anfang, ein König ans Ende. Auf diese Weise brauchen wir im Idealfall jede Karte nur ein einziges Mal anzufassen und legen sie direkt an die korrekte Position.

Genau das Gleiche macht Proxmap: Für jede Karte wird „erraten“, wo diese in einer Liste hinkommt, und dort wird sie dann direkt abgelegt. Ist die Stelle ausnahmsweise



Tony Hoare

Er wurde am 11. Januar 1932 in Sri Lanka geboren. Er studierte Statistik und beschäftigte sich mit der automatisierten Übersetzung von Fremdsprachen. Berühmt ist er durch die Entwicklung des Quicksort-Algorithmus, der bis heute als das effizienteste vergleichsbasierte Sortierv Verfahren für die meisten realen Datenmengen gilt, sowie für das „Hoare-Kalkül“, das verwendet wird, wenn man die Korrektheit eines Algorithmus beweisen möchte.

bereits besetzt, muss man eventuell eine oder zwei Karten verschieben, um Platz zu schaffen. Im Wesentlichen kommt der Algorithmus jedoch damit aus, jede Karte nur ein einziges Mal zu betrachten.

Probieren Sie das gleich einmal mit den blauen Zahlen auf Ihren Sortierkarten aus: Mischen Sie 10 der Karten und versuchen Sie, diese direkt auf den ersten 10 Feldern Ihres Papierspeichers zu verteilen.



Das Vorgehen ist klar: Die Karten zeigen Werte zwischen 0 und 999.999. Daher liegt die Vermutung nahe, dass die erste Stelle bereits ein guter Anhaltspunkt für die Position der Karte ist.

$\log n = c$?

Den interessierten Lesern möchte ich nicht verschweigen, dass einige Informatiker mit einem Augenzwinkern behaupten, es gäbe keinen Unterschied zwischen $O(n)$ und $O(n \log n)$. Die Argumentation ist recht einfach: Unsere Erde hat geschätzt $6 \cdot 10^{49}$ Atome. Mehr Objekte werden ganz sicher nie zu sortieren sein. Der Zweierlogarithmus dieser wahrlich astronomischen Zahl beträgt trotzdem nur knapp schnöde 166. Das ließe sich doch quasi als Konstante ansehen.

Sie können sich ja schon einmal eine Argumentation zurechtlegen – nur für den Fall ...

Sie werden bemerken, dass hier trotzdem oft noch sogenannte Kollisionen auftreten: Mehrere Karten kommen eigentlich auf den gleichen Platz. Diese müssen dann untereinander mit einem der konventionellen Verfahren sortiert werden.

Mit der geringen Anzahl von 10 Karten ist die Chance sehr hoch, dass wir zufällig recht ähnliche Karten ziehen. Ein ähnliches Problem haben Meinungsforscher, wenn sie zu wenige Testpersonen haben. Ein repräsentatives Ergebnis bekommt man nur mit einer entsprechend hohen Stichprobe.

Die Statistik können wir aber zu unseren Gunsten nutzen, wenn wir ein paar mehr Speicherplätze zur Verfügung stellen, als Karten einzusortieren sind.

Verteilen Sie daher in einem weiteren Experiment 15 gemischte Karten auf den 20 vorderen Speicherplätzen #32 bis #51 des gesamten Papierspeichers und sortieren Sie dabei wieder nach der blauen Zahl.

Hinweis: Wenn Sie Muße haben, können Sie später auch den Papierspeicher noch auf beliebig viele Plätze erweitern – mit einem weiteren (z. B. kopierten) Bastelbogen oder einfach mit aufgeklebten Haftnotizen. Beschriften Sie die Speicherstellen und füllen Sie diese zu ca. 75 % mit Ihrer initial unsortierten Zahlenfolge.



Nun ist auch noch die zweite Stelle relevant: Zahlen zwischen 0 und 49.999 kommen auf #32, solche zwischen 50.000 und 99.999 auf #33 usw.

Kollisionen sind jetzt sehr selten, obwohl wir immerhin 75 % des Speichers füllen! Sie müssen am Ende die Lücken nur noch schließen, indem Sie die Karten zusammenschieben. Das kostet ebenfalls maximal eine Verschiebeoperation pro Karte.

Macht man den Speicherplatz noch größer bzw. die Anzahl zu sortierender Karten kleiner, werden Kollisionen äußerst unwahrscheinlich und es gilt tatsächlich, dass jede Karte im Durchschnitt nur zwei Mal angefasst werden muss – einmal für das Einsortieren und nochmals für das Zusammenschieben.

Damit ist der Aufwand „ $2 \cdot \text{Anzahl der Karten}$ “ oder $2n$. Da wir konstante Faktoren prinzipiell weglassen, kommen wir für Proxmap-Sort auf $O(n)$.

Alles ganz normal?

Versuchen Sie nun, genau die gleichen 15 Karten nochmals auf die Speicherplätze zu verteilen, jetzt aber aufgrund der roten Zahlen. Plötzlich funktioniert unser Verfahren nicht mehr so gut. In den allermeisten Fällen gibt es jetzt deutlich mehr Kollisionen. Woran liegt das?

Auf den ersten Blick sind die roten Zahlen genauso zufällig verteilt wie die blauen. Hier trügt jedoch der Augenschein: Die blauen Zahlen sind gleich verteilt. Das bedeutet, jede Zahl zwischen 0 und 999.999 kommt mit derselben Wahrscheinlichkeit vor. Auf diese Weise ergibt sich auch eine gleichmäßige Verteilung der Karten auf die Speicherplätze – ein Garant für wenige Kollisionen.

Die roten Zahlen sind jedoch normal verteilt (auch Gauß-verteilt genannt). Hier kommen die Zahlen im mittleren Bereich um 500.000 deutlich häufiger vor als solche am Rand des Zahlenbereichs, wie 100.000 oder 900.000. Daher kommt es um die entsprechenden Speicherstellen auch vermehrt zu Kollisionen – Proxmap-Sort kann nicht mehr optimal funktionieren.

Ein Beispiel, das dies noch klarer macht, ist die alphabetische Sortierung tatsächlicher Wörter. Die grünen Wörter auf den Sortierkarten sind repräsentativ für die deutsche Sprache. Sie sind in regelmäßigen Abständen aus dem Duden entnommen.

Die grünen Buchstaben auf den Speicherstellen 0 bis 25 können zur Proxmap-Sortierung der Wörter verwendet werden. Testen Sie aus, wie viele Kollisionen es mit 20 Karten gibt ...

Offenbar ist unsere Sprache ebenfalls nicht gleich verteilt: Selbstverständlich kommen Wörter mit den Anfangsbuchstaben A oder S viel häufiger vor als solche mit X oder Y.

Egal ob normal verteilte Zahlen oder Wörter: Proxmap-Sort scheint damit nicht recht zu funktionieren ... oder können wir den Algorithmus so modifizieren, dass er besser läuft?

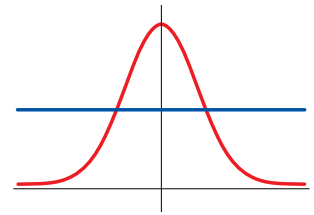
Den Teufel mit dem Belzebug austreiben? In der Informatik funktioniert es zumindest, unregelmäßig verteilte Karten mit einer unregelmäßigen Verteilung der Felder des Sortierverfahrens auszugleichen.

Sortieren Sie nochmals 15 Karten nach den roten Zahlen auf den ersten 20 Speicherplätzen – diesmal aber gemäß der roten Beschriftung der Speicherstellen.



Plötzlich funktioniert die Sache wieder mit wenigen Kollisionen. Die rote Beschriftung des Papierspeichers berücksichtigt die Verteilung der Karten und reserviert für wahrscheinlichere Intervalle auch mehr Speicherplätze als für unwahrscheinliche.

Sie merken also, dass Proxmap-Sort dann arbeiten kann, wenn die Verteilung der zu sortierenden Größen bekannt ist und man dadurch für jede Größe die Position im Speicher erraten kann. In der Praxis analysiert das Verfahren erst die zu sortierenden Daten, um die Verteilung kennen zu lernen. Damit kann der Computer recht gut raten, wo ein neuer Datensatz in den Speicher gehört.



Die **Gauß-Verteilung** bzw. **Normalverteilung** (rot) gegenüber der **Gleichverteilung** (blau). Die x-Achse bildet dabei den Wertebereich ab, die y-Achse die Wahrscheinlichkeit. Während die blaue Linie die gleiche Wahrscheinlichkeit aller Werte anzeigt, hat die rote Linie ein deutliches Maximum und fällt dann in beide Richtungen stark ab. Wegen ihrer Form wird die Gauß-Verteilung auch Glockenkurve genannt.

Proxmap – ein Sortierverfahren?

Weiter oben hatte ich geschildert, dass eigentlich bei $O(n \log n)$ Schluss ist mit der Optimierung von Sortierverfahren. Mit Proxmap-Sort erreicht man aber unter günstigen Bedingungen $O(n)$, nur eben – wie bereits erwähnt – unter veränderten Voraussetzungen. Nach den letzten Betrachtungen sollten Sie auch den Unterschied zwischen den klassischen Sortierverfahren und Proxmap erkennen. Welcher ist das?

Für die klassischen Sortierverfahren muss man lediglich eine sogenannte Ordnungsrelation zwischen zwei zu sortierenden Größen kennen: Für je zwei Karten muss klar sein, welche die größere ist bzw. ob beide als gleichwertig anzusehen sind. Dabei ist egal, wie viel größer oder kleiner die eine Karte gegenüber der anderen ist. Auch die Verteilung der Werte spielt keine Rolle – die Sortierverfahren laufen ohne weiteres Wissen darüber zuverlässig und mit gleicher Leistungsfähigkeit ab. Sie basieren ausschließlich darauf, immer wieder zwei Kartenwerte miteinander zu vergleichen.

Proxmap-Sort benötigt zusätzliche Informationen: Wie wir Menschen beim Sortieren unser Wissen über die Verteilung der Karten implizit nutzen, so versucht auch Proxmap die korrekte Position einer Karte anhand ihres Wertes zu erraten. Das setzt voraus, dass man überhaupt etwas über die Verteilung der Größen kennt. Falls dies nicht der Fall ist, kann Proxmap-Sort auch nicht sinnvoll verwendet werden.

Vielleicht überlegen Sie nun, was für Objekte man sortieren könnte, über die man keine Informationen außer der Vergleichbarkeit hat. Konstruieren lässt sich so etwas: Denken Sie an die Reihenfolge beim morgendlichen Aufstehen. „Kleiner als“ sei definiert als „früher“. Dann würde sich zum Beispiel ergeben

aufwachen < duschen < anziehen < frühstücken < schlüssel_nehmen

Ein Computer würde sich sehr schwertun, etwa dem Vorgang „anziehen“ den korrekten Platz zuzuweisen unter all den anderen Vorgängen. Eventuell kommen ja noch Tätigkeiten hinzu wie „zähneputzen“, und auch wenn die Reihenfolge irgendwie im Computerspeicher hinterlegt wird, kann man wohl keinem einzelnen Vorgang eine genaue Durchführungszeit zuweisen.

Letztlich muten solche Beispiele aber immer sehr künstlich an, denn sie enthalten Dinge, die man normalerweise auch nicht per Computer sortieren möchte. Für alle „echten“ Beispiele, die mir einfallen, lässt sich recht gut erraten, wohin ein Objekt in eine Sortierung gehört. Daher funktioniert Proxmap-Sort auch praktisch immer.



Sortieren nach Gewicht
Dosen gleichen Aussehens
werden paarweise verglichen

Ein weiteres Szenario, für das Proxmap nicht funktioniert, können Sie übrigens auch einfach basteln: Nehmen Sie eine Menge gleich aussehender, undurchsichtiger Dosen, zum Beispiel leere Farbdosen, und füllen Sie diese mit unterschiedlichen Gewichten. Ein netter Wettbewerb ist nun, die Dosen zu vermischen und danach möglichst schnell wieder dem Gewicht entsprechend zu sortieren. Dazu bleibt einem nicht viel anderes übrig, als immer zwei Dosen miteinander zu vergleichen – in der Hand oder auf einer Waage. Versuchen Sie es!

Proxmap – der Algorithmus

Vielleicht haben Sie bemerkt, dass wir bisher bei Proxmap noch nicht ganz korrekt gearbeitet haben: Kam es zu einer Kollision, konnte plötzlich unser Papierspeicherplatz



trotzdem mehrere Werte fassen. Um das Kapitel abzuschließen, erhalten Sie als Abbildung 2.24 noch einen Vorschlag für die „saubere“ Implementierung des Algorithmus.

Versuchen Sie es – das Verfahren funktioniert. Testen Sie das auch wieder mit einer Strichliste für jede Speicher- und Verschiebeoperation und ermitteln Sie dadurch die Qualität experimentell. Achten Sie darauf, dass Sie den Speicher immer nur zu etwa 75 % füllen, weil sonst Proxmap-Sort nicht mehr günstig laufen kann: Es gibt zu viele Kollisionen.

Resümee

Einerseits wissen Sie nun, wie ein Computer auf verschiedene Arten große Datenmengen sortieren kann, um dann später wieder schnell darauf zuzugreifen. Fast noch

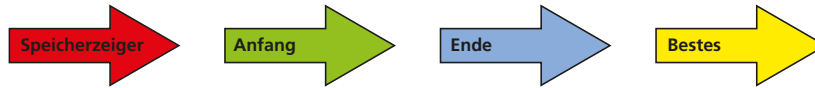
wesentlicher ist jedoch die Erkenntnis, dass man das gleiche Ziel fast immer auf sehr unterschiedliche Arten und Weisen erreichen kann.

Um zu beurteilen, welche dieser Möglichkeiten nun die beste ist, benötigt man einen Qualitätsmaßstab. Im Fall von Algorithmen ist die Ordnung der Komplexität ein solcher Maßstab. Sie gibt an, wie stark die Rechenzeit in Abhängigkeit der Problemgröße anwächst.

Oft kann man zusätzliche Informationen über die gegebenen Daten dazu verwenden, günstigere Algorithmen zu verwenden, wie im Fall von Proxmap-Sort, wo das Wissen um die Verteilung der Werte zu einem Verfahren mit der Komplexität $O(n)$ führt, während bei den klassischen, nur auf Vergleichen beruhenden Verfahren der günstigste Fall bei $O(n \log n)$ liegt.

Eventuell können Sie an dem einfachen Beispiel dieses Kapitels am besten ablesen, was den ingenieurwissenschaftlichen Teil der Informatik ausmacht: Das Sortieren, basierend nur auf Vergleichen, strahlt eine gewisse mathematischen „Schönheit“ aus, weil es universell ist: Alles, was sortierbar ist, kann sortiert werden. Trotzdem sind Informatiker damit in der Regel nicht zufrieden, sondern suchen ein Verfahren, das zwar an bestimmten, konstruierten Daten scheitert, aber in allen realen Fällen effizienter funktioniert.

Abbildung 2.K1
Kopiervorlage für Karten



187650	302782	503710	861867	212088	522107	38897	955099	96588	424261	747690	44948	581176	133986	467199	145423
aber	Ablage	absurd	Acker	Anbau	Anlage	Anteil	Arbeit	Aufruf	auftun	Ausland	Auslauf	Auster	Ausweg	Bank	Beere
276637	338210	425565	471452	544520	582278	260249	562184	444082	409121	559505	495493	163105	124890	448463	677562
488893	968277	154571	165413	9859	112092	553475	869328	885439	475340	598622	391932	539477	54218	369430	570546
Bein	bereit	Besitz	Beule	Birne	Boden	Brot	Busch	Dank	denn	Docht	Duft	Durst	ein	einmal	einzel
551904	578884	708129	343721	310313	792094	497819	762015	328439	596164	578754	551017	303040	825829	431497	686943
495973	501884	737014	92280	251907	386655	903955	272409	902289	761556	245698	79474	666701	190206	525643	555035
Ente	Erde	erneut	Erz	falsch	Fell	Filter	Flug	frei	Fuhre	Garn	gefeit	Geist	genehm	Gesetz	gewagt
313203	545038	511851	416671	196973	348971	541849	456940	206299	511274	472166	541510	527355	501389	454812	533659
417230	770853	122091	220073	799190	316190	323682	791873	801774	645869	727378	242709	285853	468831	433707	822890
gleich	Graf	Gruppe	halb	Happen	Hefe	herb	Heu	hint	hold	Idee	intern	jung	Karte	Kind	Klette
503130	493004	469414	502542	373529	286658	567975	411534	769872	587632	365762	500126	454879	504286	981432	656819
47703	689770	494105	338328	10320	230696	853090	999775	42536	601352	840530	344842	80693	173243	938664	442995
Komma	Korb	Kreuz	Kur	lang	Leber	Leute	Los	Mais	Maus	mild	Mohn	nach	Narr	nie	Oase
387491	485489	563432	716338	628007	657758	456570	261525	655509	557321	484208	408488	649042	593174	443699	955147
737455	825946	407769	194723	101144	64452	889619	179025	298597	614559	693337	901096	26685	569868	663440	544775
Ozon	Perle	Platte	Presse	Qualm	Raub	Regel	Report	Rost	Saal	schade	Schein	Schlag	Schliff	Schrank	Schutz
635066	616474	356123	394035	627000	33834	299566	429514	434058	871139	536598	746586	532031	180319	356039	285673
621512	512644	265569	1546	633803	875185	341378	922111	658614	379917	716582	208537	704238	919143	832882	406012
schwer	selbst	Sinn	sozial	Spitze	Stadt	Stein	Stock	streng	Suppe	Tausch	toll	treu	Uhr	Umkehr	Umzug
570621	304128	534281	685547	511749	437755	819903	622771	791314	345638	608609	293607	415283	739765	534179	525784
783299	813083	715258	229762	532702	450663	759881	626457	67142	585876	890751	971956	987582	371795	941393	352600
Unheil	untreu	Vehikel	Verein	Verleih	Vers	Verweis	voll	Vorteil	Wand	Weide	Wert	Winter	Wurm	Zeit	Zirkus
460043	520134	473739	491187	308833	473574	482518	468563	409135	430459	66580	715557	642427	356724	381937	491846
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
Q	R	S	T	U	V	W	X	Y	Z	0	1	3	7	12	13
18	24	31	37	40	45	51	65	66	72	73	75	78	80	84	87
92	96	98	99												

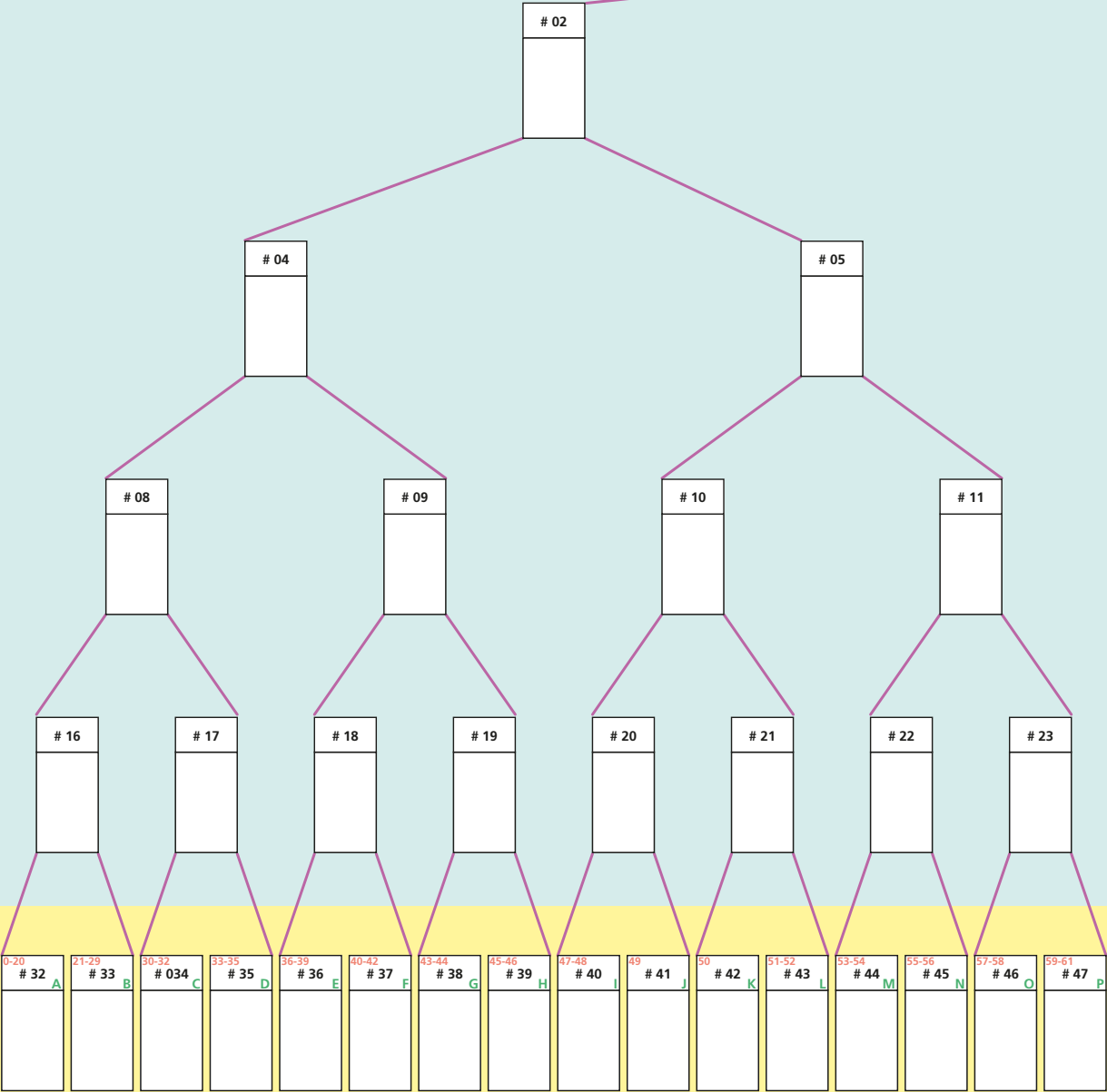
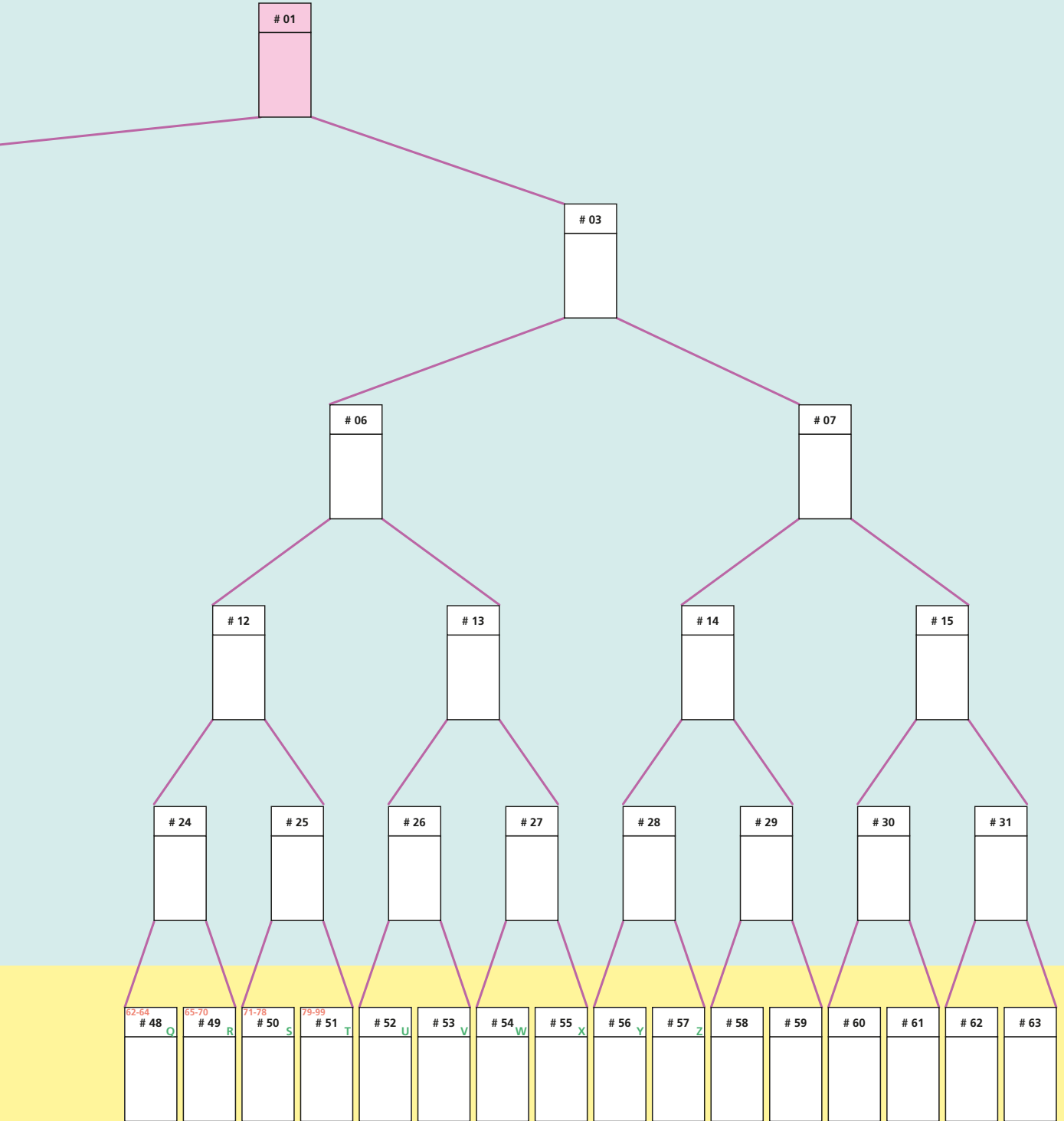


Abbildung 2.K2
Der Papierspeicher





3. Ich packe meinen Koffer und ...

Es war einmal ein tapferer Mann aus dem Volk, der rettete die Prinzessin Tausendschön vor dem Räuber Abi Lala und seinen zweiundvierzig Schergen. Leider musste sich der König nun etwas als Belohnung ausdenken: Wäre der Mann ein Prinz gewesen, hätte er ihm einfach die Hand der Prinzessin anbieten können. Für einen gewöhnlichen Bürger würde der König aber wohl oder übel seine Schatzkammer öffnen müssen.

Um den Schaden jedoch zu begrenzen, gab der König dem Mann eine Holzkiste und sprach: „Du darfst dir alles aus der Schatzkammer nehmen, was in die Kiste passt, so dass sich der Deckel noch schließen lässt.“ Er dachte, dass jener sicher so von den Schätzen geblendet wäre, dass er eher billigen Tand als wirklich wertvolle Stücke auswählen würde.

Hier hat er jedoch nicht mit unserer Beteiligung gerechnet! Die Informatik hat dem Mann aus dem Volk sogar eine eigene Kategorie von Aufgaben gewidmet, genannt „Rucksackprobleme“.

Das Rucksackproblem

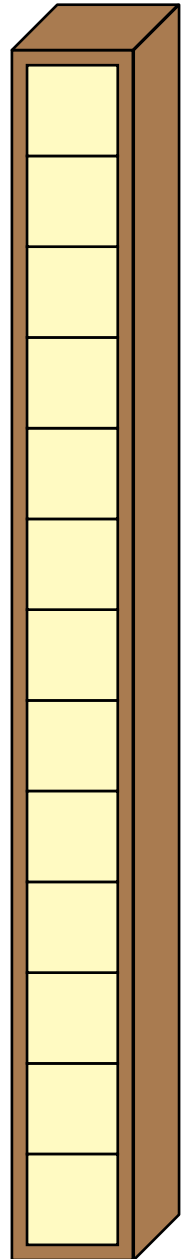
Bereits in den vorherigen Kapiteln haben wir sehr viel mit Abstraktion gearbeitet und das Problem zunächst vereinfacht, um darüber besser nachdenken zu können. Hier nehme ich diesen Schritt schon jetzt vorweg und biete Ihnen die gesammelten „abstrakten“ Schätze als Bastelbogen oder Kopiervorlage 3.K1 – rechteckig und aus einer glatten Zahl von Quadraten zusammengesetzt.

Schneiden Sie sie aus und versuchen damit, die Kiste am Buchrand möglichst optimal zu füllen. Wir gehen hier einfach davon aus, dass jedem Schatz ein Zertifikat mit einem genauen Wert (selbstverständlich einheitlich in Golddublonen bemessen) beiliegt – dieser steht als ganze Zahl neben der Abbildung.

Die sogenannten „Triviale Fälle“ sind hier übrigens bereits eliminiert: Nehmen wir an, es existierten zwei Gegenstände mit exakt der gleichen Größe, aber unterschiedlichem Wert. Dann legt man selbstverständlich den billigeren Gegenstand im Vorfeld bereits als „uninteressant“ ab. Daher steht hier nur ein Objekt pro Größe zur Wahl (eben das jeweils teuerste).

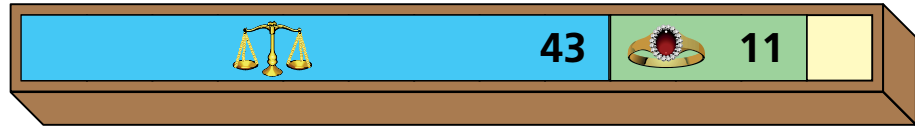
Sie sehen bereits an dem einfachen Beispiel, dass es gar nicht so einfach ist, das Optimum aus allen Möglichkeiten herauszufinden: Es passen sechs Edelsteine in die Kiste, aber auch eine Vase und zwei Ringe. Vielleicht sind aber drei Münzen wertvoller? Oder gar die goldene Waage – dann passt aber sonst nicht viel mehr in die Kiste ...

Wie auch bereits zuvor, können wir hier die Lösung durch stures Ausprobieren finden: Alle Möglichkeiten werden systematisch durchgegangen, die wertvollste Kombination wird beibehalten. Das ist für das vorliegende Beispiel machbar, aber bereits sehr zeit-



aufwendig. Überlegen Sie, wie viele Möglichkeiten Sie bereits hätten, wenn die Kiste auch nur doppelt so groß wäre ...

Abbildung 3.1
Gefüllte Schatzkiste mit
einem Wert von 54 Dublonen.
Geht das noch besser?



Über die Problematik des sturen Probierens hatten wir bereits im Kapitel über Routenplanung philosophiert, das möchte ich an dieser Stelle nicht wiederholen. Allerdings versichere ich Ihnen, dass der Aufwand zum Probieren hier mit der Größe der Kiste ebenfalls sehr stark anwächst. Die Lösungsansätze werden damit schnell unübersichtlich und sind zumindest in sinnvoller Zeit nicht zielführend.

Mehr ist weniger

Divide et impera
„Teile und herrsche“ – er-
innern Sie sich noch daran?
Wenn nicht: In Kapitel 2 wird
dieses Prinzip ausführlich
erörtert.

Statt herumzuprobieren wollen wir daher das Problem konsequent angehen. Eine entscheidende Strategie der Informatik haben Sie ja bereits mit „divide et impera“ kennengelernt: Wenn das Gesamtproblem zu groß zum Lösen ist, versuchen wir es in kleinere Pakete zu zerteilen. Das machen wir so lange, bis die Teilprobleme handhabbar klein sind.

Um das Problem in kleinere Happen zu unterteilen, müssen wir allerdings zunächst feststellen, wodurch überhaupt ein Problem dieser Art als „groß“ oder „klein“ gilt – wir bestimmen die Problemgröße. Beim Routenplaner ist das zum Beispiel die Anzahl der Orte auf einer Landkarte, beim Sortieren die Anzahl der zu sortierenden Objekte.

Ihre Aufgabe: Überlegen Sie, was beim Packen des Rucksacks die Problemgröße ist.



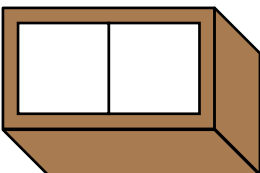
Zugegeben: Die Frage war vielleicht ein wenig unfair formuliert, aber ansonsten hätte ich schon zu viel verraten. Sie sind bestimmt trotzdem auf die korrekte Antwort gekommen.

- Ist „Größe der Kiste“ vielleicht eine Problemgröße?
- Ist „Anzahl unterschiedlicher Gegenstände“ vielleicht eine Problemgröße?

Beides ist richtig: Wie bereits oben ausgeführt, wird eine Lösung immer schwieriger, je größer die Kiste ist und je mehr Gegenstände man daher gleichzeitig einpacken muss (das gilt übrigens nur, wenn man wirklich an einer exakten Lösung interessiert ist, aber dazu mehr weiter unten). Zusätzlich steigt die Problemgröße selbstverständlich auch an, wenn man mehr unterschiedliche Gegenstände zur Wahl hat und ausprobieren muss.

Sie können Ihre Vermutung über eine Problemgröße normalerweise leicht untermauern: Überlegen Sie, welches das „kleinste Problem“ ist, wenn Sie Ihre Überlegungen zugrunde legen.

„Größe der Kiste“: Nehmen Sie eine sehr kleine Kiste, zum Beispiel die nebenstehende. Ist hier die Lösung des Problems besonders schwierig? Nein, denn es gibt überhaupt nur zwei Möglichkeiten:



- die Kiste leer lassen (und das wäre wirklich dumm) oder
- einen Edelstein hineinladen, was immerhin sechs Dublonen bringt.

„Anzahl unterschiedlicher Gegenstände“: Benutzen Sie immer die große Kiste, aber gehen Sie davon aus, in der Schatzkammer befände sich nur eine einzige Sorte Schatz, zum Beispiel Edelsteine. Auch in diesem Fall gibt es nicht viel zu entscheiden: Man lädt so viele Edelsteine in die Kiste, wie hineinpassen. Abbildung 3.2 zeigt eine solche Kiste.



Abbildung 3.2

Schatzkiste nur mit Edelsteinen, immerhin 42 Dublonen wert

Beim Sortieren von Karten war eine mögliche Vorgehensweise, mit einer einzelnen Karte anzufangen (die per Definition sortiert ist). Dann wurde die Problemgröße jeweils um eine Karte erweitert, die in die bereits sortierte Folge leicht eingefügt werden konnte.

Vielleicht finden Sie ja auf dieser Basis auch eine Lösung für das Rucksackproblem? Versuchen Sie es: Fangen Sie mit kleinen Kisten oder wenigen unterschiedlichen Objekten an und erhöhen Sie dann die Problemgröße langsam.



Die Lösung gestaltet sich schwieriger als erwartet:

Fängt man mit einer kleinen Kiste an und steigert deren Größe, kommt man recht schnell auf optimale Ergebnisse für Kisten mit 2, 3, 4 und 5 Quadraten (wir nennen diese fortan 2er-Kiste, 3er-Kiste usw. oder Kisten der Größen 2, 3, 4 usw.). Danach fängt das Rätseln an: Welche der Teillösungen müssen kombiniert werden, um eine noch größere Kiste zu packen? Auch hier können wir nur alle Möglichkeiten ausprobieren, was uns zum Ausgangspunkt unserer Betrachtungen zurückwirft.

Versuchen wir also, gleich die große 13er-Kiste zu nehmen, fangen aber zunächst an, sie ausschließlich mit Edelsteinen zu füllen, wie schon in Abbildung 3.2 gezeigt. Danach erweitern wir die Problemgröße: Auch Ringe sind nun erlaubt. Recht schnell kann man feststellen, dass vier Ringe (mit 44 Dublonen Wert) besser sind als die sechs Goldbarren (die nur 42 Dublonen wert waren).

Offenbar sind also Edelsteine nicht so günstig wie Ringe, wir können sie also für die folgenden Betrachtungen außer Acht lassen. Weiter geht es und auch die Goldtaler mit 16 Dublonen Wert sollen nun im Angebot sein. Wiederum können wir den Wert unserer Kiste steigern, indem wir die Ringe entsorgen und stattdessen drei Goldtaler einladen. 48 Dublonen sind nun das Ergebnis.

Als Nächstes nehmen wir den wertvollen Kompass hinzu, der so ausladend ist, dass er fünf Plätze ausfüllt. In unsere Kiste passen daher nur zwei Kompass mit einem Gesamtwert von 48 Dublonen. Das ist genauso viel, wie wir bereits haben. Daher bleiben wir bei den Goldtalern, oder?

Ganz so einfach ist es nicht, denn legen wir zu den zwei Kompassen noch einen Ring (was gut passt), steigt der Wert unserer Kiste sogar auf 59. Unsere Entscheidung, die Ringe als „zu billig“ außer Acht zu lassen, war also ziemlich voreilig! Auch mit diesem Lösungsansatz müssen wir im späteren Verlauf der Problemlösung immer wieder

ganz an den Anfang zurückspringen, um das Optimum herauszuholen, was effektiv auch einfach das Durchspielen aller Möglichkeiten bedeutet.

Ist hier also „divide et impera“ gar nicht zu gebrauchen? Müssen wir uns bei diesem Problem geschlagen geben?

Selbstverständlich nicht: Die Informatik hat noch ein paar Tricks auf Lager. In diesem Fall heißt der Trick: „Mehr ist weniger“, oder anders gesagt „Wenn man mehr Probleme löst, hat man letztlich weniger Arbeit.“

Wie ist das genau zu verstehen? Das Teilen von Problemen ist manchmal schwierig und man weiß nicht genau, welche Teilprobleme man genau lösen muss, um sie dann zu einer Gesamtlösung zu kombinieren. In solchen Fällen ist es manchmal hilfreich, einfach alle möglichen Teilprobleme zu lösen und danach zu entscheiden, welche man kombiniert. So etwas nennt man dynamische Programmierung.



Richard Ernest Bellman (1920 – 1984) studierte Mathematik und beschäftigte sich danach mit theoretischer Physik. Ab 1952 befasste er sich dann bei der Rand-Corporation mit Optimierungsproblemen und adaptierte ein vorher nur von Physikern verwendetes Prinzip für die Informatik, das er „dynamische Programmierung“ nannte. Heute ist ein Algorithmus zur Erstellung optimaler Binärbäume nach ihm benannt.

Dynamische Programmierung

Strategie zur Lösung komplexer Probleme, besonders zur Lösung von Optimierungsaufgaben. Man löst zunächst viele Teilprobleme und verwendet diese Bausteine – optimale Zwischenergebnisse –, um die nächstgrößeren Probleme zu lösen usw., bis das Gesamtproblem gelöst ist.

Um dieses Prinzip auf unser Rucksackproblem anzuwenden, benötigen wir viele Kisten: von einer ganz kleinen Kiste (mit überhaupt keinem Platz darin) bis zur größten Kiste mit 13 Quadraten Platz. Sie finden dies im Bastelbogen und Abbildung 3.K2 – sogar mit zwei weiteren Feldern für Ihre eigenen Experimente. Ich nenne das Gebilde im Folgenden „Maxikiste“, da die Kisten aller ganzen Größen enthalten sind.

Jetzt fangen wir ganz klein an: Gehen Sie davon aus, es gäbe nur die kleinste Sorte Schatz zur Auswahl – Edelsteine. Gemäß dem Prinzip der dynamischen Programmierung ermitteln wir nun für den anderen Typ Problemgröße – die Kistengröße – nicht nur eine Lösung, sondern gleich alle bis zur Größe 13. Die entsprechenden Kisten finden wir in der Maxikiste.

Mit nur einer Sorte Schatz ist es leicht, die Kisten entsprechend optimal zu füllen. Machen Sie das! Zur Verdeutlichung können Sie links auf Klebezetteln daneben den Wert jeder Kiste vermerken.



Abbildung 3.3 zeigt die Maxikiste. Selbstverständlich kann man in die kleinste Kiste mit der Größe 0 nichts hineingeben. Auch in die Kiste mit nur einem Quadrat Platz passt kein Edelstein – sie bleibt daher frei. Die restlichen Kisten können mit ein bis sechs Edelsteinen gefüllt werden, man stopft einfach so viele hinein, wie passen. Die größte Kiste enthält sechs Barren zu je sieben Dublonen, hat also einen Wert von 42 Dublonen.

Um es noch einmal festzuhalten: Die Maxikiste zeigt nun die optimale Lösung mit der kleinsten Problemgröße – nur die kleinsten Schätze dürfen ausgewählt werden.

Wie können wir diese Ergebnisse verwenden, um nun den nächstgrößeren Schatz einzubeziehen? Diesmal zeige ich Ihnen, was dafür zu tun ist, und wir überlegen hinterher zusammen, warum diese Strategie sinnvoll ist.

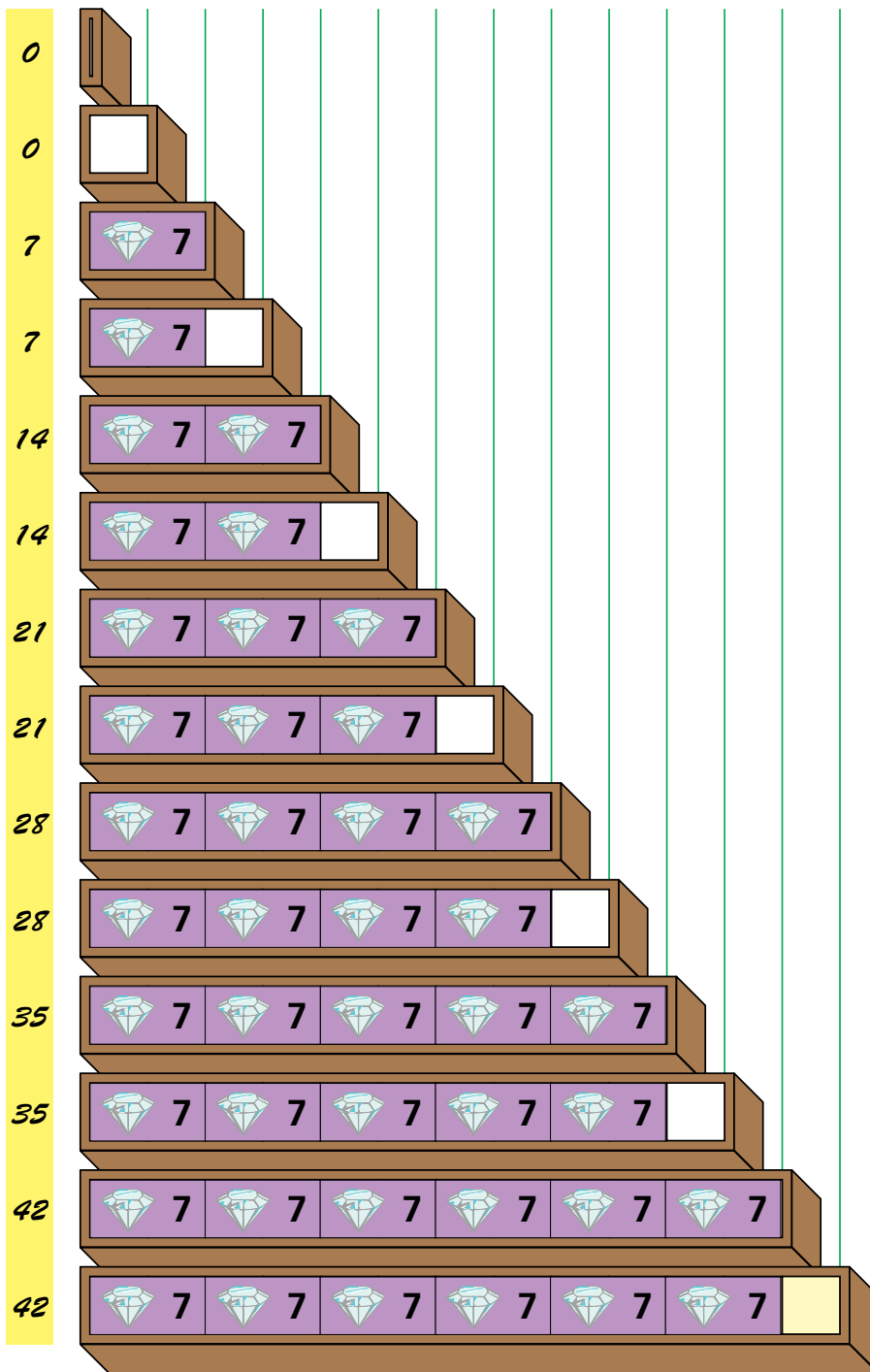


Abbildung 3.3
Lösung des Rucksackproblems bis zur Kistengröße 13, wenn nur Edelsteine zur Wahl stehen

Nehmen Sie einen Schatz „Ring“ und legen ihn rechts an die kleinste Kiste an – das ist die Kiste mit Größe 0. Sie haben jetzt sozusagen den neuen Schatz zu der Kiste hinzugelegt. In welche (größere) Kiste würden die Inhalte nun passen? Selbstverständlich in die Kiste der Größe 3, wie Sie auch ablesen können, wenn Sie die grünen Linien nach unten verfolgen. Abbildung 3.4 zeigt das.